DataScience.R

puneeth

2026-03-29

```
# ==============================================================================
# SYSTEM FOR AUTOMATIC LEARNING OF ELECTRICAL LOAD PATTERNS
# AND POWER SYSTEM SIZING — FULL ANALYSIS
# Data Science  | Reg No.: Puneeth 23BCE0191, Sanjay 23BCE0211,Sai Krishna 23BCE2005,
Harshil 23BCE0900
# ==============================================================================
# Dataset : UCI Individual Household Electric Power Consumption
# Source  : https://archive.ics.uci.edu/ml/datasets/individual+household+electric+pow
er+consumption
# Direct  : https://archive.ics.uci.edu/ml/machine-learning-databases/00235/household
_power_consumption.zip
# Citation: Hebrail, G., & Berard, A. (2012). UCI Machine Learning Repository.
# ==============================================================================

# ── 0. Install & Load Libraries ──────────────────────────────────────────────
packages <- c(
  "tidyverse",    # Data wrangling & ggplot2
  "lubridate",    # Date-time parsing
  "forecast",     # ARIMA / ETS (load BEFORE Metrics to avoid accuracy() clash)
  "tseries",      # ADF / KPSS stationarity tests
  "cluster",      # K-means + silhouette
  "factoextra",   # Cluster & PCA visualisation
  "corrplot",     # Correlation matrix heat-map
  "ggcorrplot",   # ggplot2 correlation with p-values
  "GGally",       # Pairwise scatter matrix
  "scales",       # Axis formatting helpers
  "gridExtra",    # Multi-panel plot layout
  "ggthemes",     # Extra ggplot2 themes
  "zoo",          # Rolling window functions
  "moments",      # Skewness & kurtosis
  "changepoint",  # Change-point detection (PELT)
  "viridis",      # Perceptually uniform palettes
  "randomForest", # Random Forest regression
  "Metrics"       # RMSE / MAE (load AFTER forecast)
)

installed <- rownames(installed.packages())
to_install <- packages[!packages %in% installed]
if (length(to_install) > 0)
  install.packages(to_install, repos = "https://cran.r-project.org", quiet = TRUE)

# Load in order; keep forecast::accuracy accessible via namespace
invisible(lapply(packages, library, character.only = TRUE))
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.2.0     ✔ readr     2.2.0
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ ggplot2   4.0.2     ✔ tibble    3.3.1
## ✔ lubridate 1.9.5     ✔ tidyr     1.3.2
## ✔ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflic
ts to become errors
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
##
## Welcome to factoextra!
##
## Want to learn more? See two factoextra-related books at https://www.datanovia.com/
en/product/practical-guide-to-principal-component-methods-in-r/
##
## corrplot 0.95 loaded
##
##
## Attaching package: 'scales'
##
##
## The following object is masked from 'package:purrr':
##
##     discard
##
##
## The following object is masked from 'package:readr':
##
##     col_factor
##
##
##
## Attaching package: 'gridExtra'
##
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
##
##
## Attaching package: 'zoo'
##
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
##
##
## Successfully loaded changepoint package version 2.3
```

```
##  WARNING: From v.2.3 the default method in cpt.* functions has changed from AMOC t
o PELT.
##  See NEWS for details of all changes.
##
## Loading required package: viridisLite
##
##
## Attaching package: 'viridis'
##
##
## The following object is masked from 'package:scales':
##
##     viridis_pal
##
##
## randomForest 4.7-1.2
##
## Type rfNews() to see new features/changes/bug fixes.
##
##
## Attaching package: 'randomForest'
##
##
## The following object is masked from 'package:gridExtra':
##
##     combine
##
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
##
## The following object is masked from 'package:ggplot2':
##
##     margin
##
##
##
## Attaching package: 'Metrics'
##
##
## The following object is masked from 'package:forecast':
##
##     accuracy
```

```
# Fix accuracy() namespace clash: Metrics masks forecast's accuracy()
# We'll call forecast::accuracy() explicitly throughout
cat("✅ All libraries loaded.\n\n")
```

```
## ✅ All libraries loaded.
```

```
# ── 1. Download & Load Dataset ──────────────────────────────────────────
cat("📩 Downloading UCI Household Power Consumption dataset...\n")
```

```
## 📩 Downloading UCI Household Power Consumption dataset...
```

```
url     <- "https://archive.ics.uci.edu/ml/machine-learning-databases/00235/household
_power_consumption.zip"
zipfile <- tempfile(fileext = ".zip")
datadir <- tempdir()

tryCatch({
  download.file(url, destfile = zipfile, mode = "wb", quiet = TRUE)
  unzip(zipfile, exdir = datadir)
  cat("✅ Download successful.\n\n")
}, error = function(e) stop("❌ Download failed: ", e$message))
```

```
## ✅ Download successful.
```

```
raw <- read.table(
  file.path(datadir, "household_power_consumption.txt"),
  sep = ";", header = TRUE,
  na.strings = "?", stringsAsFactors = FALSE
)
cat(sprintf("✅ Raw data: %d rows × %d columns\n\n", nrow(raw), ncol(raw)))
```

```
## ✅ Raw data: 2075259 rows × 9 columns
```

```
# ── 2. Pre-processing & Feature Engineering ─────────────────────────────
cat("🔧 Pre-processing & engineering features...\n")
```

```
## 🔧 Pre-processing & engineering features...
```

```
df <- raw %>%
  mutate(
    datetime = dmy_hms(paste(Date, Time)),
    Date     = as.Date(Date, "%d/%m/%Y"),
    across(c(Global_active_power, Global_reactive_power,
             Voltage, Global_intensity,
             Sub_metering_1, Sub_metering_2, Sub_metering_3), as.numeric)
  ) %>%
  filter(!is.na(Global_active_power)) %>%
  mutate(
    year        = year(datetime),
    month       = month(datetime, label = TRUE),
    month_num   = month(datetime),
    week        = week(datetime),
    day         = day(datetime),
    hour        = hour(datetime),
    minute      = minute(datetime),
    weekday     = wday(datetime, label = TRUE, week_start = 1),
    weekday_num = wday(datetime, week_start = 1),
    is_weekend  = ifelse(wday(datetime) %in% c(1, 7), "Weekend", "Weekday"),
    season      = case_when(
      month_num %in% c(12, 1, 2) ~ "Winter",
      month_num %in% c(3, 4, 5)  ~ "Spring",
      month_num %in% c(6, 7, 8)  ~ "Summer",
      TRUE                       ~ "Autumn"
    ),
    time_of_day = case_when(
      hour >= 6  & hour < 12 ~ "Morning",
      hour >= 12 & hour < 17 ~ "Afternoon",
      hour >= 17 & hour < 21 ~ "Evening",
      TRUE                   ~ "Night"
    ),
    energy_kwh        = Global_active_power / 60,
    apparent_power    = sqrt(Global_active_power^2 + Global_reactive_power^2),
    power_factor      = ifelse(apparent_power > 0,
                               Global_active_power / apparent_power, NA),
    reactive_ratio    = Global_reactive_power / (Global_active_power + 0.001),
    other_submetering = pmax(
      (Global_active_power * 1000 / 60) -
        Sub_metering_1 - Sub_metering_2 - Sub_metering_3, 0)
  )

cat(sprintf("✅ Clean dataset: %d rows × %d columns\n\n", nrow(df), ncol(df)))
```

```
## ✅ Clean dataset: 2049280 rows × 27 columns
```

```
# ── 3. EDA — Summary Statistics ──────────────────────────────────────────
cat("📊 Section 3: Descriptive Statistics & Normality Tests\n\n")
```

```
## 📊 Section 3: Descriptive Statistics & Normality Tests
```

```
key_vars <- c("Global_active_power", "Global_reactive_power",
              "Voltage", "Global_intensity", "energy_kwh", "power_factor")

desc_stats <- df %>%
  select(all_of(key_vars)) %>%
  summarise(across(everything(), list(
    n      = ~sum(!is.na(.)),
    min    = ~min(.,    na.rm = TRUE),
    Q1     = ~quantile(., 0.25, na.rm = TRUE),
    mean   = ~mean(.,   na.rm = TRUE),
    median = ~median(., na.rm = TRUE),
    Q3     = ~quantile(., 0.75, na.rm = TRUE),
    max    = ~max(.,    na.rm = TRUE),
    sd     = ~sd(.,     na.rm = TRUE),
    cv_pct = ~sd(., na.rm = TRUE) / mean(., na.rm = TRUE) * 100,
    skew   = ~moments::skewness(., na.rm = TRUE),
    kurt   = ~moments::kurtosis(., na.rm = TRUE)
  )))

cat("── Descriptive Statistics (transposed) ──────────────────────────────────\n")
```

```
## ── Descriptive Statistics (transposed) ──────────────────────────────────
```

```
print(as.data.frame(t(desc_stats)), digits = 4)
```

```
##                                    V1
## Global_active_power_n         2.049e+06
## Global_active_power_min       7.600e-02
## Global_active_power_Q1        3.080e-01
## Global_active_power_mean      1.092e+00
## Global_active_power_median    6.020e-01
## Global_active_power_Q3        1.528e+00
## Global_active_power_max       1.112e+01
## Global_active_power_sd        1.057e+00
## Global_active_power_cv_pct    9.686e+01
## Global_active_power_skew      1.786e+00
## Global_active_power_kurt      7.219e+00
## Global_reactive_power_n       2.049e+06
## Global_reactive_power_min     0.000e+00
## Global_reactive_power_Q1      4.800e-02
## Global_reactive_power_mean    1.237e-01
## Global_reactive_power_median  1.000e-01
## Global_reactive_power_Q3      1.940e-01
## Global_reactive_power_max     1.390e+00
## Global_reactive_power_sd      1.127e-01
## Global_reactive_power_cv_pct  9.111e+01
## Global_reactive_power_skew    1.262e+00
## Global_reactive_power_kurt    5.606e+00
## Voltage_n                     2.049e+06
## Voltage_min                   2.232e+02
## Voltage_Q1                    2.390e+02
## Voltage_mean                  2.408e+02
## Voltage_median                2.410e+02
## Voltage_Q3                    2.429e+02
## Voltage_max                   2.542e+02
## Voltage_sd                    3.240e+00
## Voltage_cv_pct                1.345e+00
## Voltage_skew                 -3.267e-01
## Voltage_kurt                  3.725e+00
## Global_intensity_n            2.049e+06
## Global_intensity_min          2.000e-01
## Global_intensity_Q1           1.400e+00
## Global_intensity_mean         4.628e+00
## Global_intensity_median       2.600e+00
## Global_intensity_Q3           6.400e+00
## Global_intensity_max          4.840e+01
## Global_intensity_sd           4.444e+00
## Global_intensity_cv_pct       9.604e+01
## Global_intensity_skew         1.849e+00
## Global_intensity_kurt         7.601e+00
## energy_kwh_n                  2.049e+06
## energy_kwh_min                1.267e-03
## energy_kwh_Q1                 5.133e-03
## energy_kwh_mean               1.819e-02
## energy_kwh_median             1.003e-02
## energy_kwh_Q3                 2.547e-02
## energy_kwh_max                1.854e-01
## energy_kwh_sd                 1.762e-02
## energy_kwh_cv_pct             9.686e+01
## energy_kwh_skew               1.786e+00
```

```
## energy_kwh_kurt              7.219e+00
## power_factor_n               2.049e+06
## power_factor_min             5.559e-01
## power_factor_Q1              9.520e-01
## power_factor_mean            9.637e-01
## power_factor_median          9.934e-01
## power_factor_Q3              9.997e-01
## power_factor_max             1.000e+00
## power_factor_sd              5.810e-02
## power_factor_cv_pct          6.029e+00
## power_factor_skew           -2.059e+00
## power_factor_kurt            6.997e+00
```

```
cat("\n── Missing Value Report ──────────────────────────────────────────────
\n")
```

```
##
## ── Missing Value Report ──────────────────────────────────────────────
```

```
miss_df <- data.frame(
  Variable = names(df),
  Missing  = colSums(is.na(df)),
  Pct      = round(colSums(is.na(df)) / nrow(df) * 100, 2)
) %>% filter(Missing > 0)
if (nrow(miss_df) == 0) cat("  No missing values remaining.\n") else print(miss_df)
```

```
##   No missing values remaining.
```

```
cat("\n── Shapiro-Wilk Normality Test (n = 5000 sample) ──────────────────────
\n")
```

```
##
## ── Shapiro-Wilk Normality Test (n = 5000 sample) ──────────────────────
```

```
for (v in key_vars[1:4]) {
  s   <- sample(na.omit(df[[v]]), 5000)
  res <- shapiro.test(s)
  cat(sprintf("  %-28s W=%.4f  p=%.2e  → %s\n", v, res$statistic, res$p.value,
              ifelse(res$p.value < 0.05, "NON-NORMAL ❌", "NORMAL ✅")))
}
```

```
##   Global_active_power          W=0.8144  p=3.48e-60  → NON-NORMAL ❌
##   Global_reactive_power        W=0.8903  p=8.67e-51  → NON-NORMAL ❌
##   Voltage                      W=0.9937  p=4.81e-14  → NON-NORMAL ❌
##   Global_intensity             W=0.8014  p=1.87e-61  → NON-NORMAL ❌
```

```
cat("\n── Kruskal-Wallis Test: Power across Seasons ──────────────────────────
\n")
```

```
##
## ── Kruskal-Wallis Test: Power across Seasons ──────────────────────────────
```

```
kw <- kruskal.test(Global_active_power ~ season, data = df %>% sample_n(20000))
cat(sprintf("  χ²=%.2f, df=%d, p=%.2e → %s\n", kw$statistic, kw$parameter, kw$p.valu
e,
            ifelse(kw$p.value < 0.05,
                   "Significant seasonal difference ✅",
                   "No significant difference")))
```

```
##   χ²=1741.66, df=3, p=0.00e+00 → Significant seasonal difference ✅
```

```
cat("\n── Kruskal-Wallis Test: Power across Time-of-Day ────────────────────────
\n")
```

```
##
## ── Kruskal-Wallis Test: Power across Time-of-Day ──────────────────────────
```

```
kw2 <- kruskal.test(Global_active_power ~ time_of_day, data = df %>% sample_n(20000))
cat(sprintf("  χ²=%.2f, df=%d, p=%.2e → %s\n", kw2$statistic, kw2$parameter, kw2$p.va
lue,
            ifelse(kw2$p.value < 0.05, "Significant ✅", "Not significant")))
```

```
##   χ²=1376.72, df=3, p=3.32e-298 → Significant ✅
```

```
# ── 4. Aggregations ──────────────────────────────────────────────────────────
cat("\n🔢 Section 4: Aggregations\n")
```

```
##
## 🔢 Section 4: Aggregations
```

```
daily <- df %>%
  group_by(Date, is_weekend, season) %>%
  summarise(
    total_kwh    = sum(energy_kwh,              na.rm = TRUE),
    mean_power   = mean(Global_active_power,    na.rm = TRUE),
    peak_power   = max(Global_active_power,     na.rm = TRUE),
    mean_voltage = mean(Voltage,                na.rm = TRUE),
    sd_voltage   = sd(Voltage,                  na.rm = TRUE),
    mean_pf      = mean(power_factor,           na.rm = TRUE),
    mean_react   = mean(Global_reactive_power,  na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(Date) %>%
  mutate(
    rolling_avg_7  = rollmean(total_kwh, 7,  fill = NA, align = "right"),
    rolling_avg_30 = rollmean(total_kwh, 30, fill = NA, align = "right"),
    rolling_sd_7   = rollapply(total_kwh, 7, sd, fill = NA, align = "right"),
    day_of_year    = yday(Date),
    week_of_year   = week(Date),
    year           = year(Date),
    month          = month(Date, label = TRUE),
    log_kwh        = log1p(total_kwh)
  )

monthly <- df %>%
  group_by(year, month, month_num, season) %>%
  summarise(
    total_kwh  = sum(energy_kwh,           na.rm = TRUE),
    mean_power = mean(Global_active_power, na.rm = TRUE),
    peak_power = max(Global_active_power,  na.rm = TRUE),
    min_power  = min(Global_active_power,  na.rm = TRUE),
    mean_pf    = mean(power_factor,        na.rm = TRUE),
    .groups = "drop"
  )

hourly_profile <- df %>%
  group_by(hour, is_weekend, season) %>%
  summarise(
    mean_power = mean(Global_active_power, na.rm = TRUE),
    sd_power   = sd(Global_active_power,   na.rm = TRUE),
    p90_power  = quantile(Global_active_power, 0.90, na.rm = TRUE),
    .groups = "drop"
  )

seasonal_hour <- df %>%
  group_by(season, hour) %>%
  summarise(mean_power = mean(Global_active_power, na.rm = TRUE), .groups = "drop")

tod_summary <- df %>%
  group_by(time_of_day, season) %>%
  summarise(
    mean_power = mean(Global_active_power, na.rm = TRUE),
    total_kwh  = sum(energy_kwh,           na.rm = TRUE),
    .groups = "drop"
  )
```

```
cat("✅ Aggregations complete.\n")
```

```
## ✅ Aggregations complete.
```

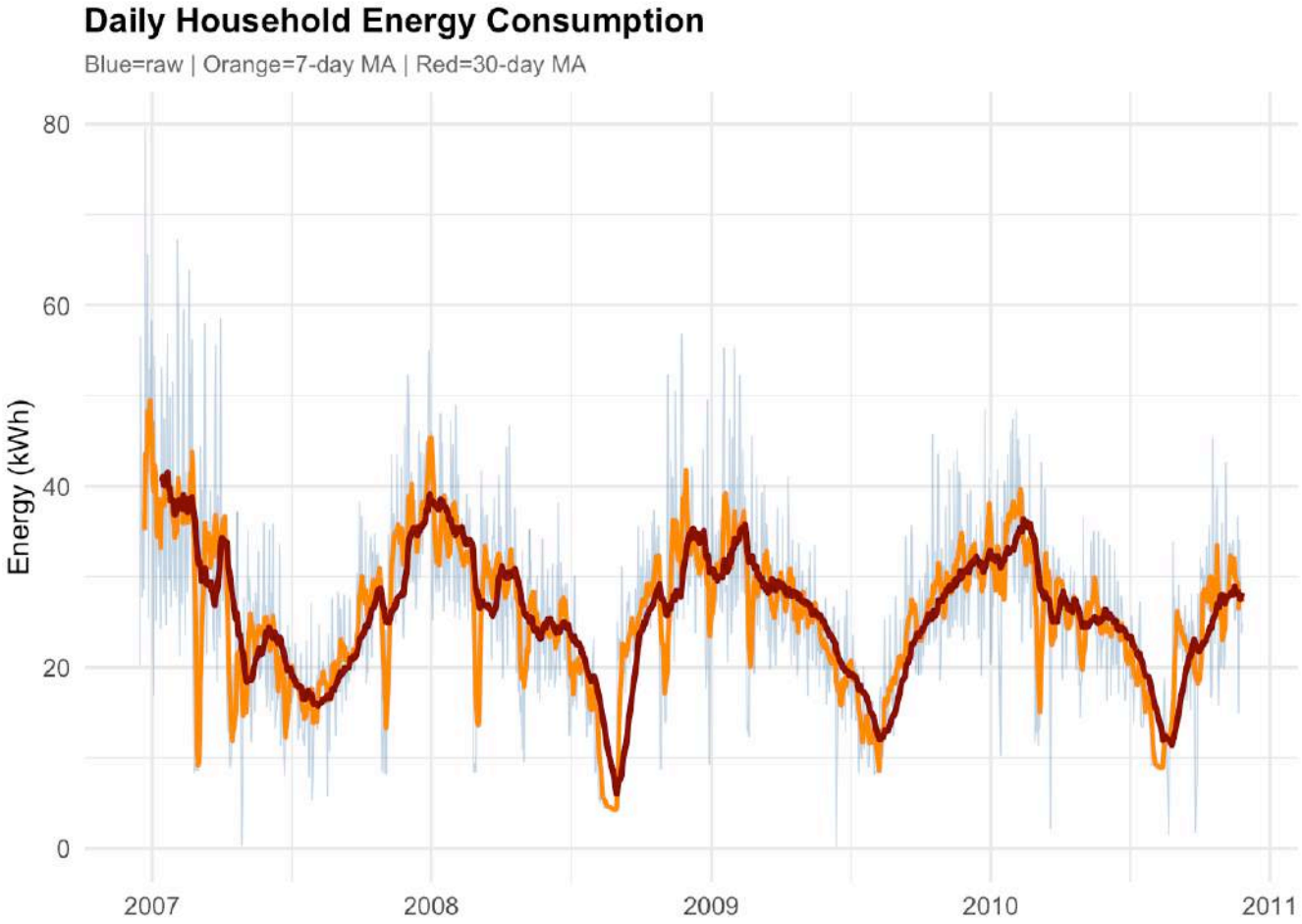
```
# ── 5. Visualisations ──────────────────────────────────────────────────────
cat("\n🎨 Section 5: Visualisations (12 plots)\n")
```

```
##
## 🎨 Section 5: Visualisations (12 plots)
```

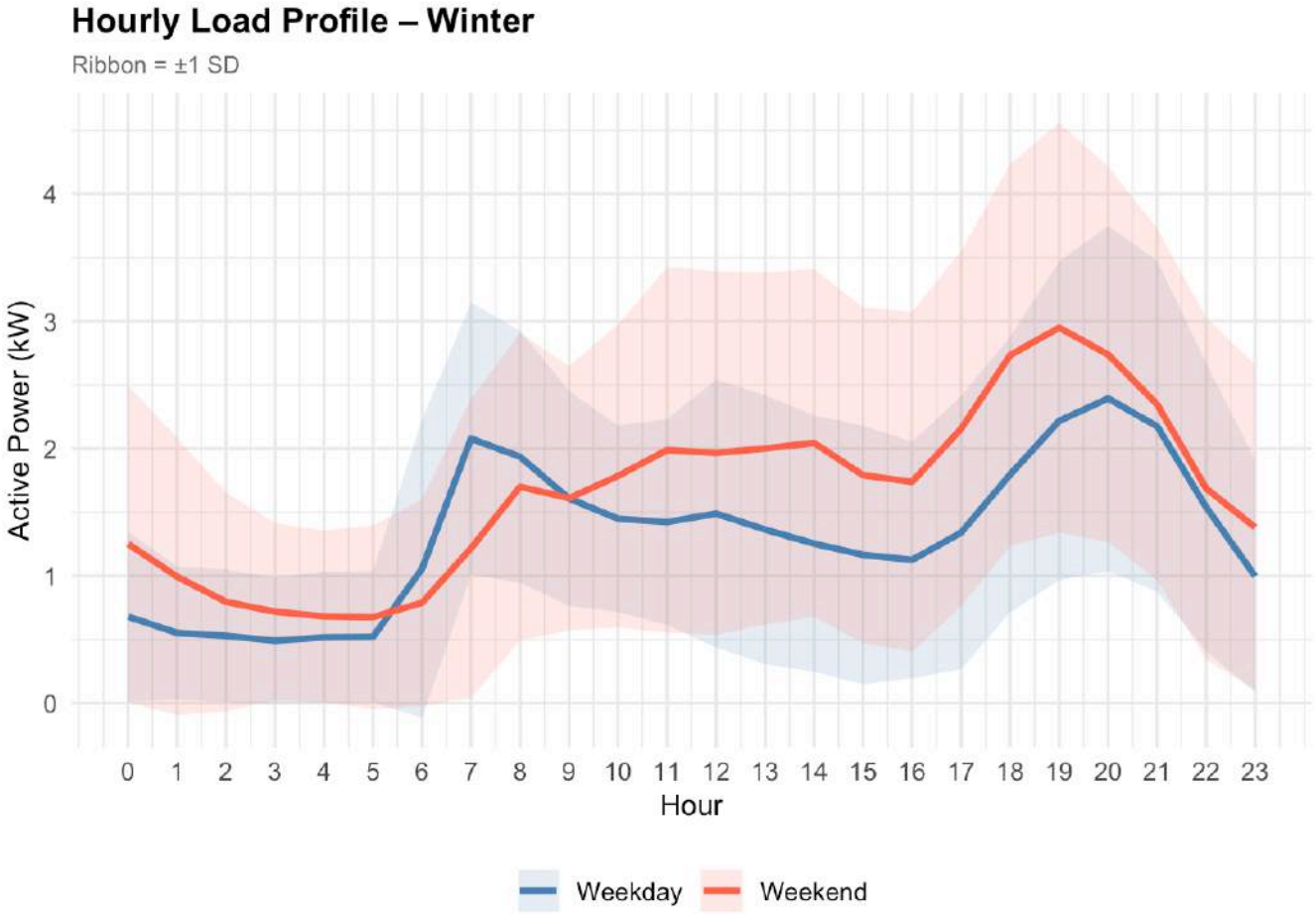
```
theme_load <- theme_minimal(base_size = 12) +
  theme(plot.title    = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9,  colour = "grey40"),
        axis.title    = element_text(size = 11),
        legend.position = "bottom")

# P1 — Daily energy with 7/30-day rolling averages
p1 <- ggplot(daily, aes(x = Date, y = total_kwh)) +
  geom_line(colour = "steelblue", alpha = 0.35, linewidth = 0.25) +
  geom_line(aes(y = rolling_avg_7),  colour = "darkorange", linewidth = 0.9) +
  geom_line(aes(y = rolling_avg_30), colour = "darkred",    linewidth = 1.1) +
  labs(title    = "Daily Household Energy Consumption",
       subtitle = "Blue=raw | Orange=7-day MA | Red=30-day MA",
       x = NULL, y = "Energy (kWh)") + theme_load

p1
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale rang
e
## (`geom_line()`).
```

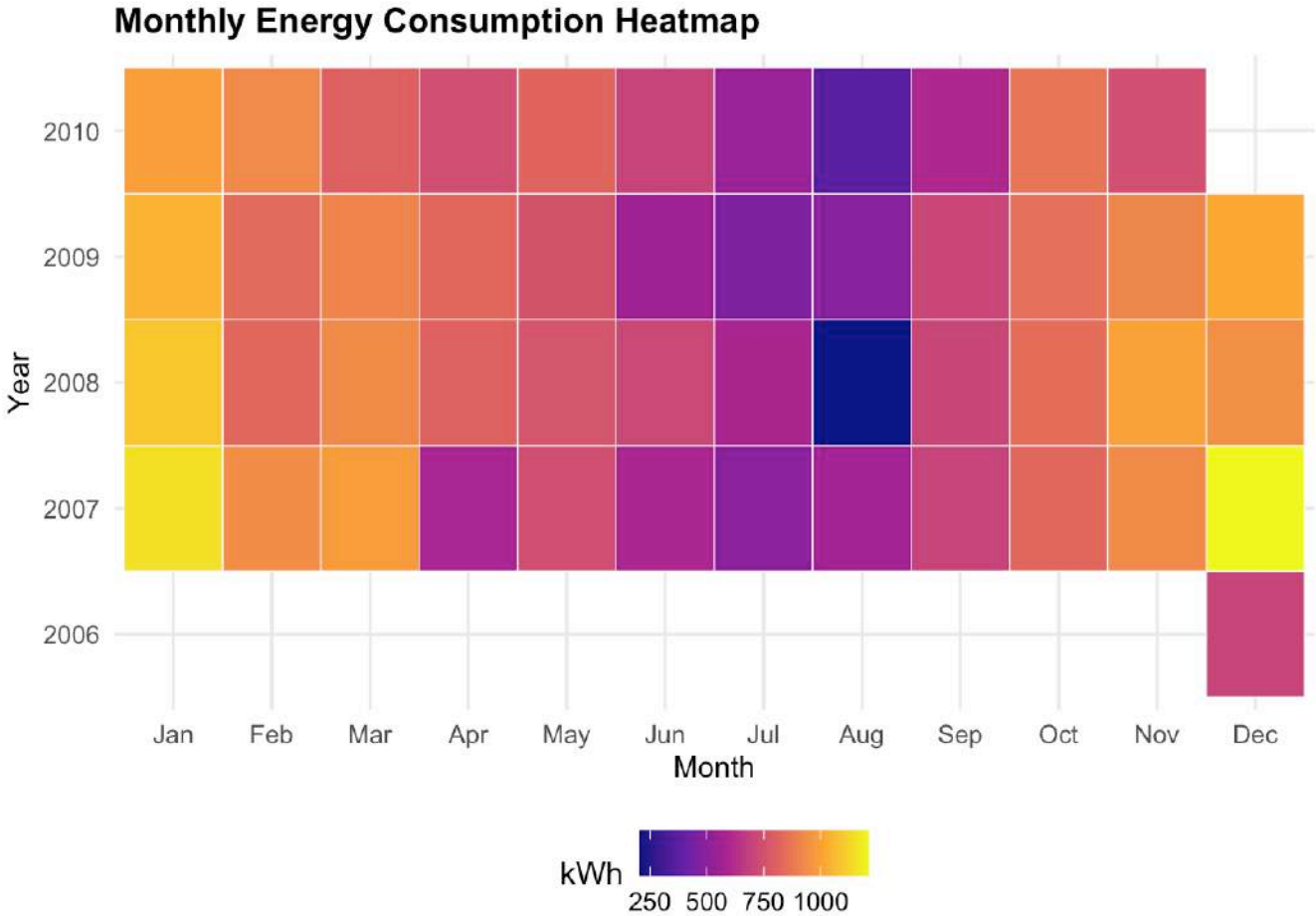
```
## Warning: Removed 29 rows containing missing values or values outside the scale ran
ge
## (`geom_line()`).
```



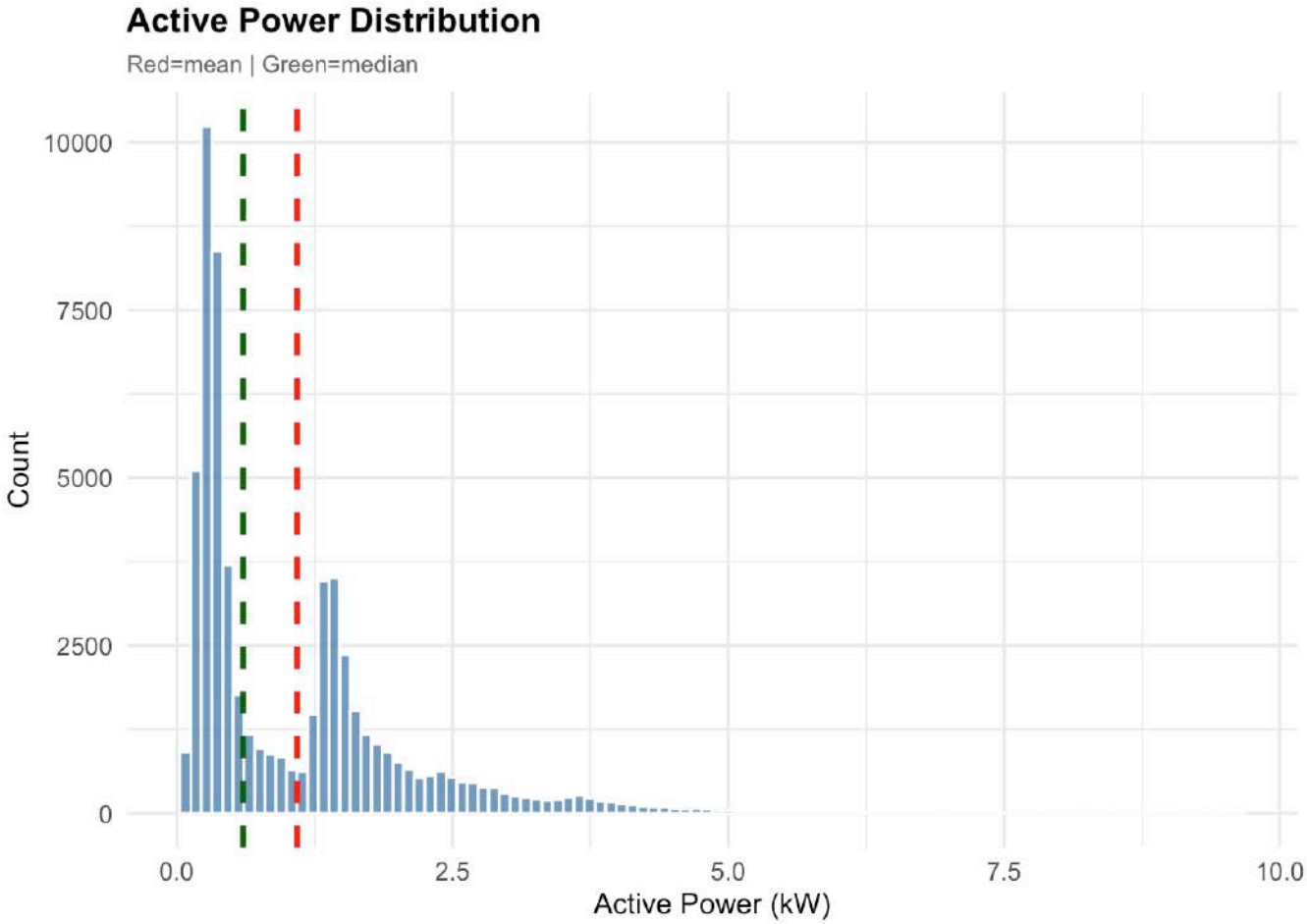

```
# P2 — Hourly winter load profile: weekday vs weekend
p2 <- ggplot(hourly_profile %>% filter(season == "Winter"),
             aes(x = hour, y = mean_power, colour = is_weekend, fill = is_weekend)) +
  geom_ribbon(aes(ymin = mean_power - sd_power, ymax = mean_power + sd_power),
              alpha = 0.15, colour = NA) +
  geom_line(linewidth = 1.2) +
  scale_x_continuous(breaks = 0:23) +
  scale_colour_manual(values = c("Weekday" = "steelblue", "Weekend" = "tomato")) +
  scale_fill_manual(values   = c("Weekday" = "steelblue", "Weekend" = "tomato")) +
  labs(title = "Hourly Load Profile — Winter", subtitle = "Ribbon = ±1 SD",
       x = "Hour", y = "Active Power (kW)", colour = NULL, fill = NULL) + theme_load

p2
```



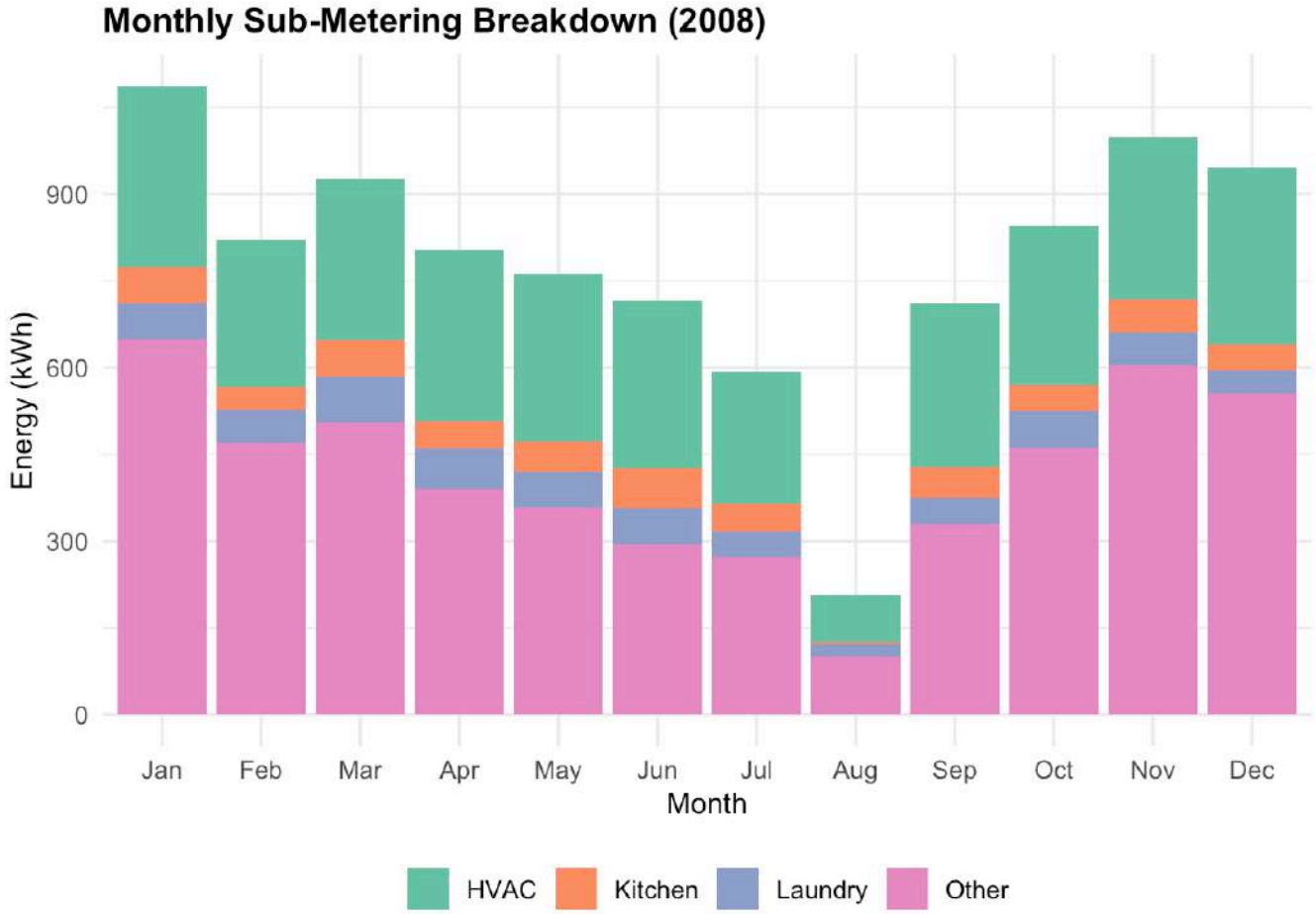

```
# P3 — Monthly energy heatmap
p3 <- ggplot(monthly, aes(x = month, y = factor(year), fill = total_kwh)) +
  geom_tile(colour = "white") +
  scale_fill_viridis_c(name = "kWh", option = "C") +
  labs(title = "Monthly Energy Consumption Heatmap",
       x = "Month", y = "Year") + theme_load

p3
```



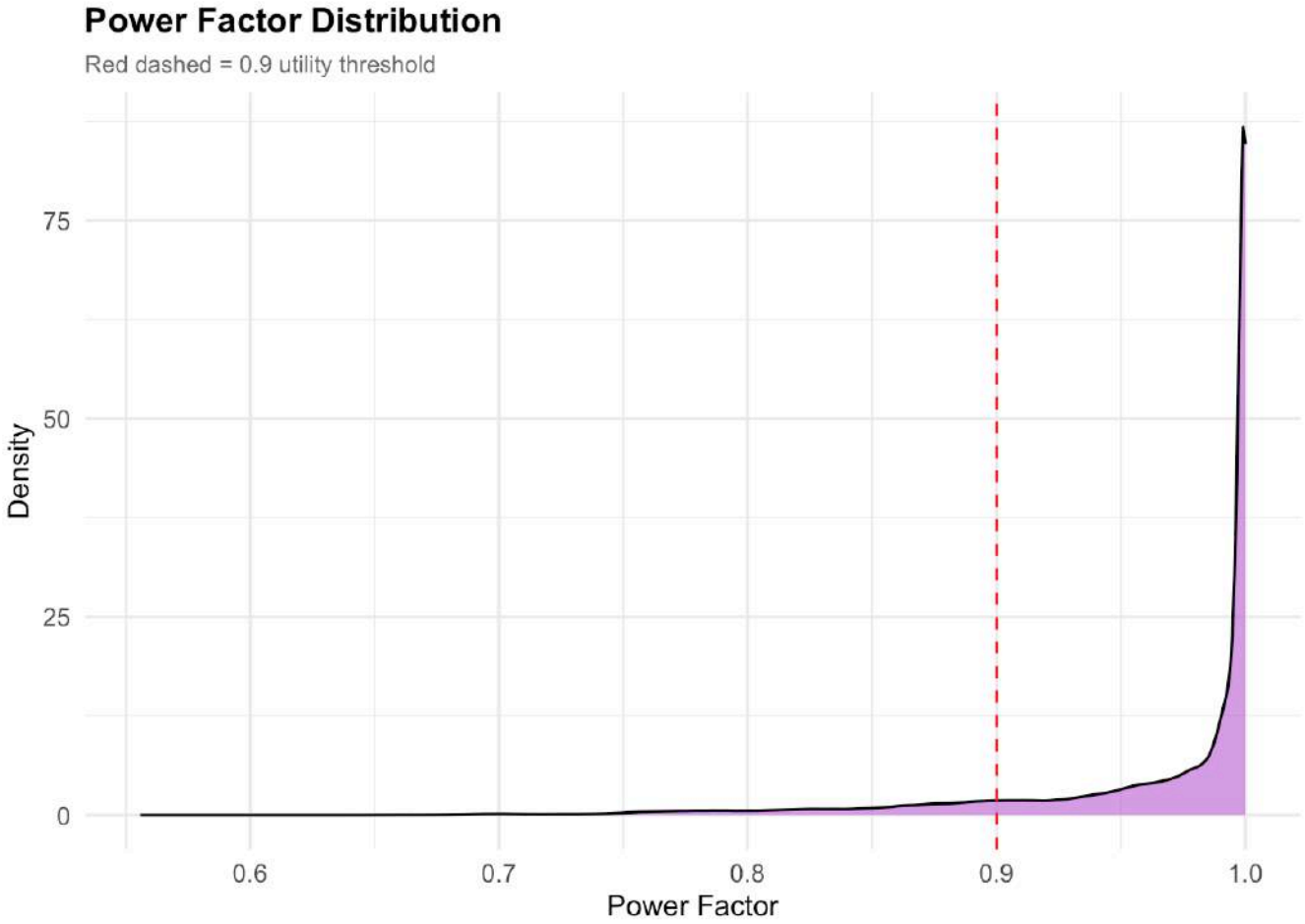

```
# P4 — Active power distribution with mean/median lines
p4 <- ggplot(df %>% sample_n(60000), aes(x = Global_active_power)) +
  geom_histogram(bins = 100, fill = "steelblue", colour = "white", alpha = 0.8) +
  geom_vline(xintercept = mean(df$Global_active_power, na.rm = TRUE),
             colour = "red",       linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = median(df$Global_active_power, na.rm = TRUE),
             colour = "darkgreen", linetype = "dashed", linewidth = 1) +
  labs(title    = "Active Power Distribution",
       subtitle = "Red=mean | Green=median",
       x = "Active Power (kW)", y = "Count") + theme_load

p4
```



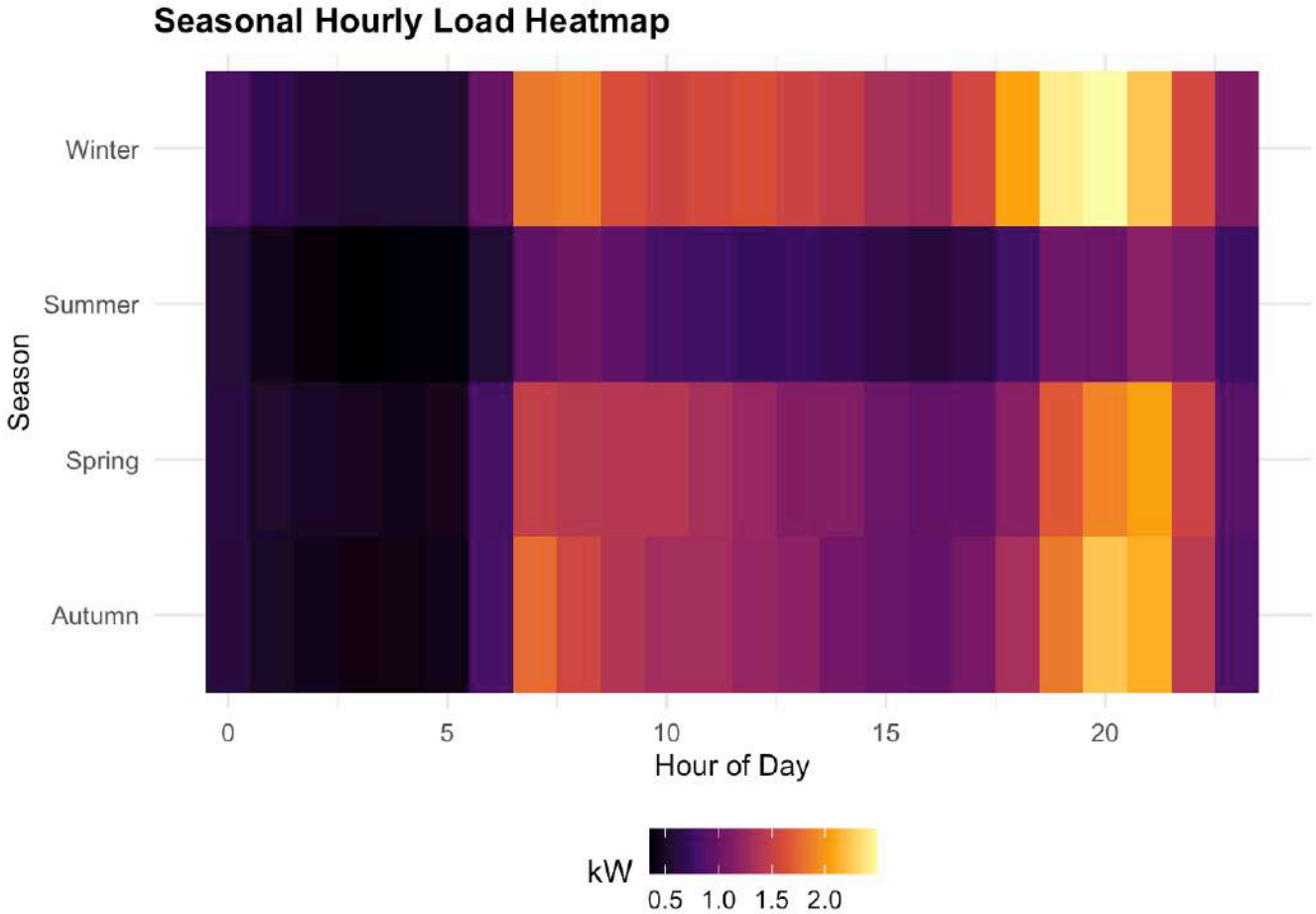

```
# P5 — Sub-metering stacked bar (2008)
sub_long <- df %>%
  filter(year == 2008) %>%
  group_by(month) %>%
  summarise(
    Kitchen = sum(Sub_metering_1,     na.rm = TRUE) / 1000,
    Laundry = sum(Sub_metering_2,     na.rm = TRUE) / 1000,
    HVAC    = sum(Sub_metering_3,     na.rm = TRUE) / 1000,
    Other   = sum(other_submetering,  na.rm = TRUE) / 1000,
    .groups = "drop"
  ) %>%
  pivot_longer(-month, names_to = "Category", values_to = "kWh")

p5 <- ggplot(sub_long, aes(x = month, y = kWh, fill = Category)) +
  geom_col(position = "stack") +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Monthly Sub-Metering Breakdown (2008)",
       x = "Month", y = "Energy (kWh)", fill = NULL) + theme_load

p5
```



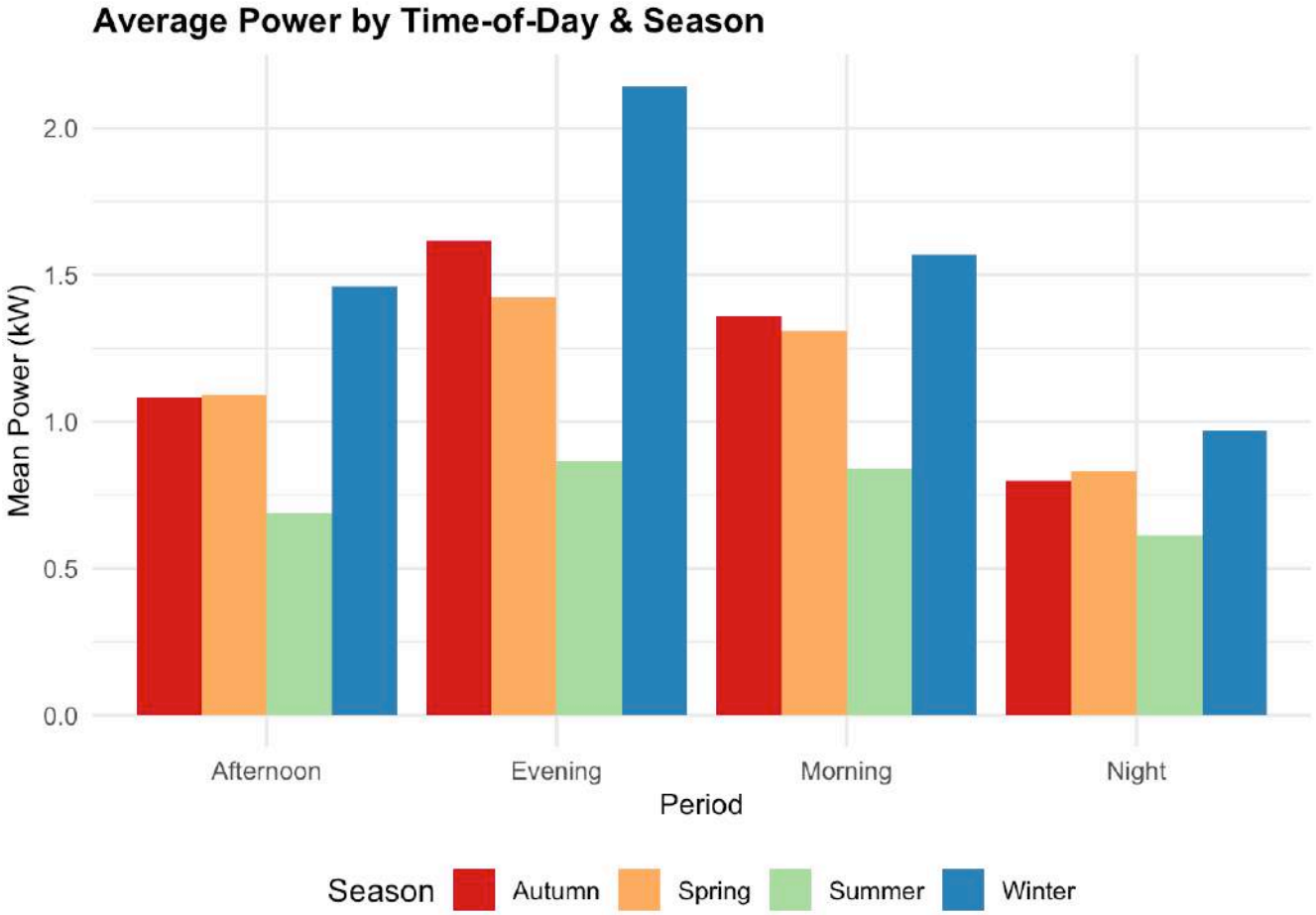

```
# P6 — Power factor density
p6 <- ggplot(df %>% filter(!is.na(power_factor)), aes(x = power_factor)) +
  geom_density(fill = "mediumorchid", alpha = 0.6) +
  geom_vline(xintercept = 0.9, colour = "red", linetype = "dashed") +
  labs(title    = "Power Factor Distribution",
       subtitle = "Red dashed = 0.9 utility threshold",
       x = "Power Factor", y = "Density") + theme_load
p6
```



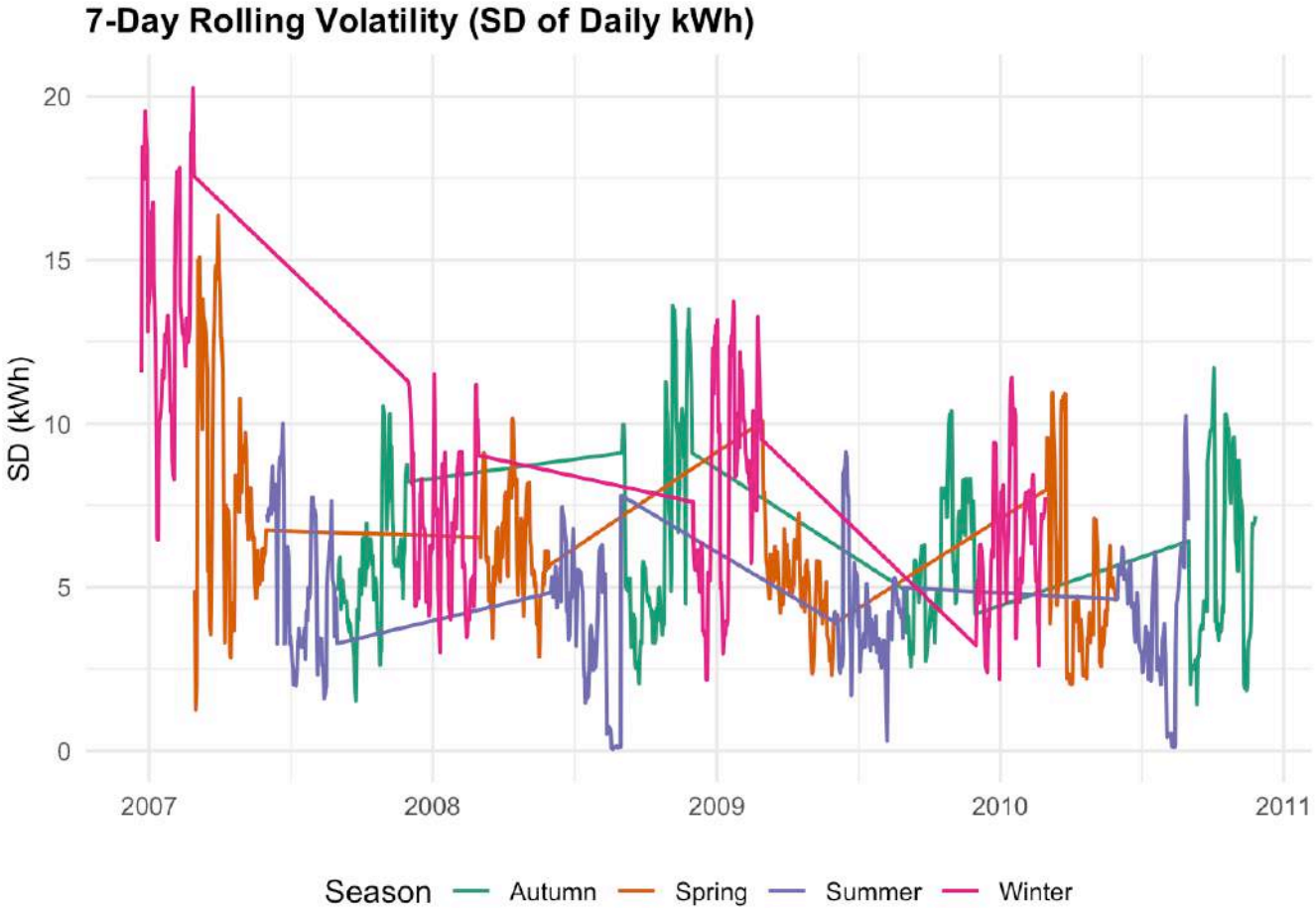

```
# P7 — Seasonal hourly heatmap
p7 <- ggplot(seasonal_hour, aes(x = hour, y = season, fill = mean_power)) +
  geom_tile() +
  scale_fill_viridis_c(name = "kW", option = "B") +
  labs(title = "Seasonal Hourly Load Heatmap",
       x = "Hour of Day", y = "Season") + theme_load

p7
```



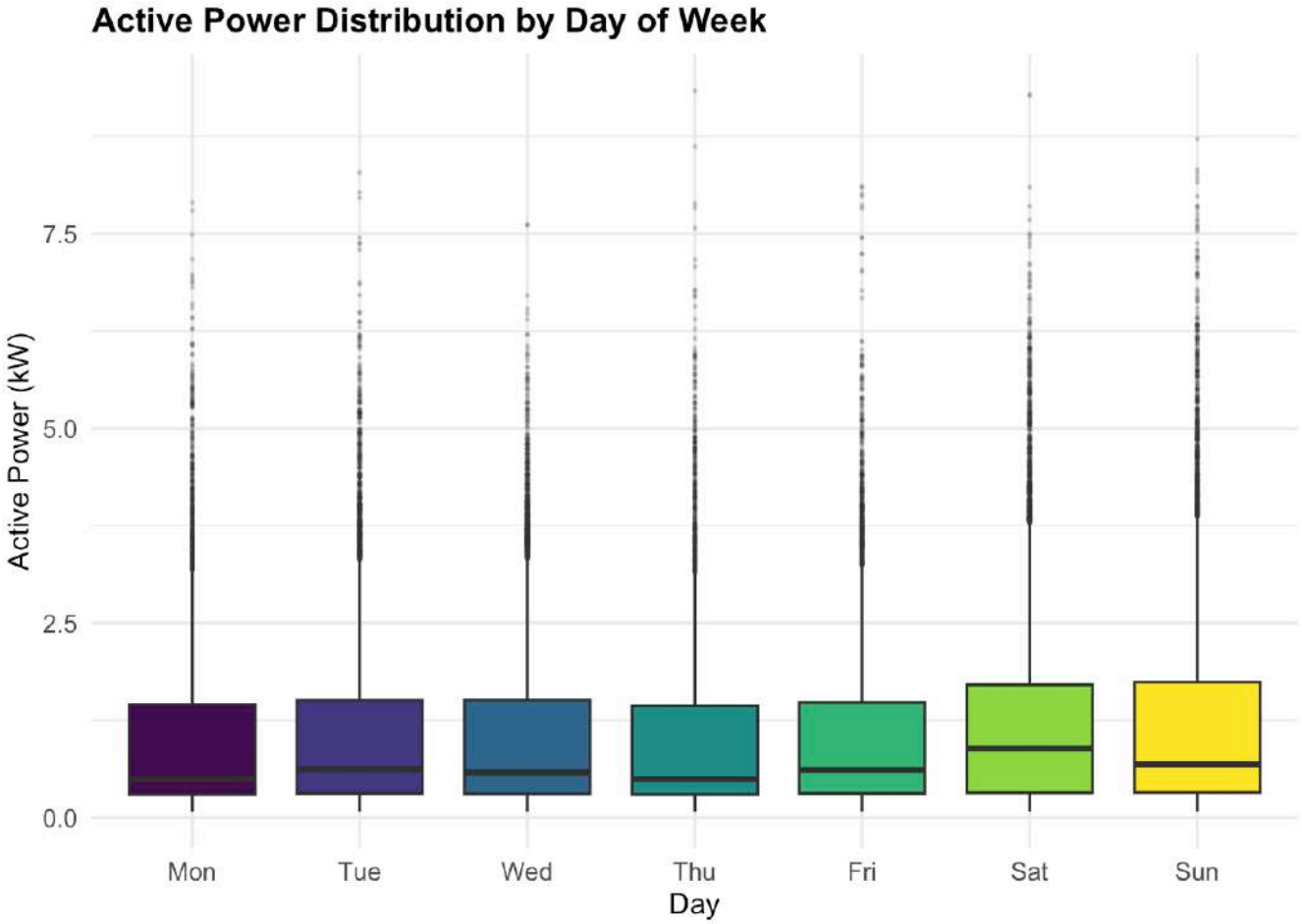

```
# P8 — Time-of-day mean power by season
p8 <- ggplot(tod_summary, aes(x = time_of_day, y = mean_power, fill = season)) +
  geom_col(position = "dodge") +
  scale_fill_brewer(palette = "Spectral") +
  labs(title = "Average Power by Time-of-Day & Season",
       x = "Period", y = "Mean Power (kW)", fill = "Season") + theme_load
p8
```



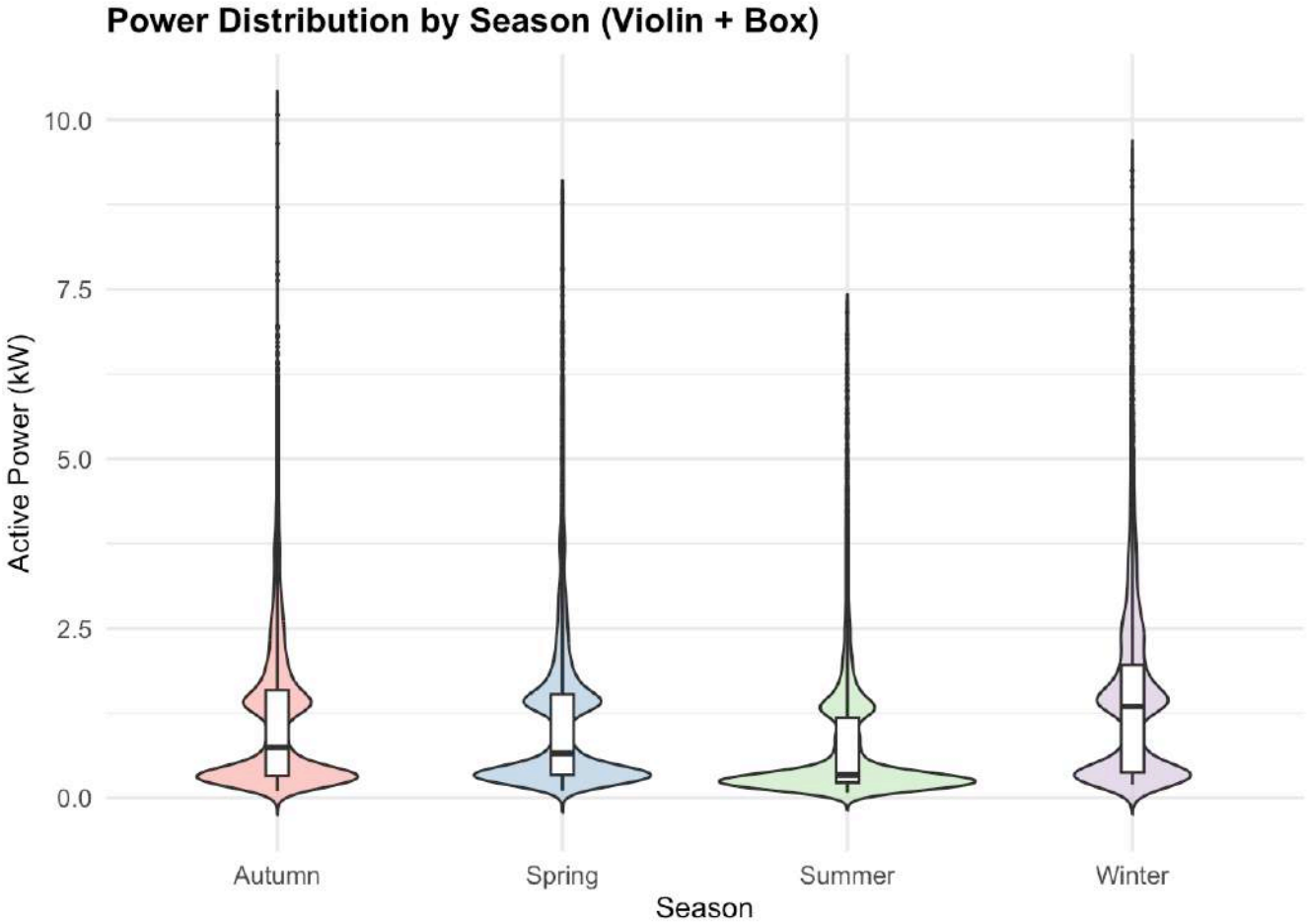

```
# P9 — 7-day rolling volatility (SD) coloured by season
p9 <- ggplot(daily %>% filter(!is.na(rolling_sd_7)),
             aes(x = Date, y = rolling_sd_7, colour = season)) +
  geom_line(linewidth = 0.7) +
  scale_colour_brewer(palette = "Dark2") +
  labs(title = "7-Day Rolling Volatility (SD of Daily kWh)",
       x = NULL, y = "SD (kWh)", colour = "Season") + theme_load

p9
```




```
# P10 — Boxplot by day of week
p10 <- ggplot(df %>% sample_n(80000),
              aes(x = weekday, y = Global_active_power, fill = weekday)) +
  geom_boxplot(outlier.size = 0.3, outlier.alpha = 0.2) +
  scale_fill_viridis_d() +
  labs(title = "Active Power Distribution by Day of Week",
       x = "Day", y = "Active Power (kW)") +
  theme_load + theme(legend.position = "none")

p10
```

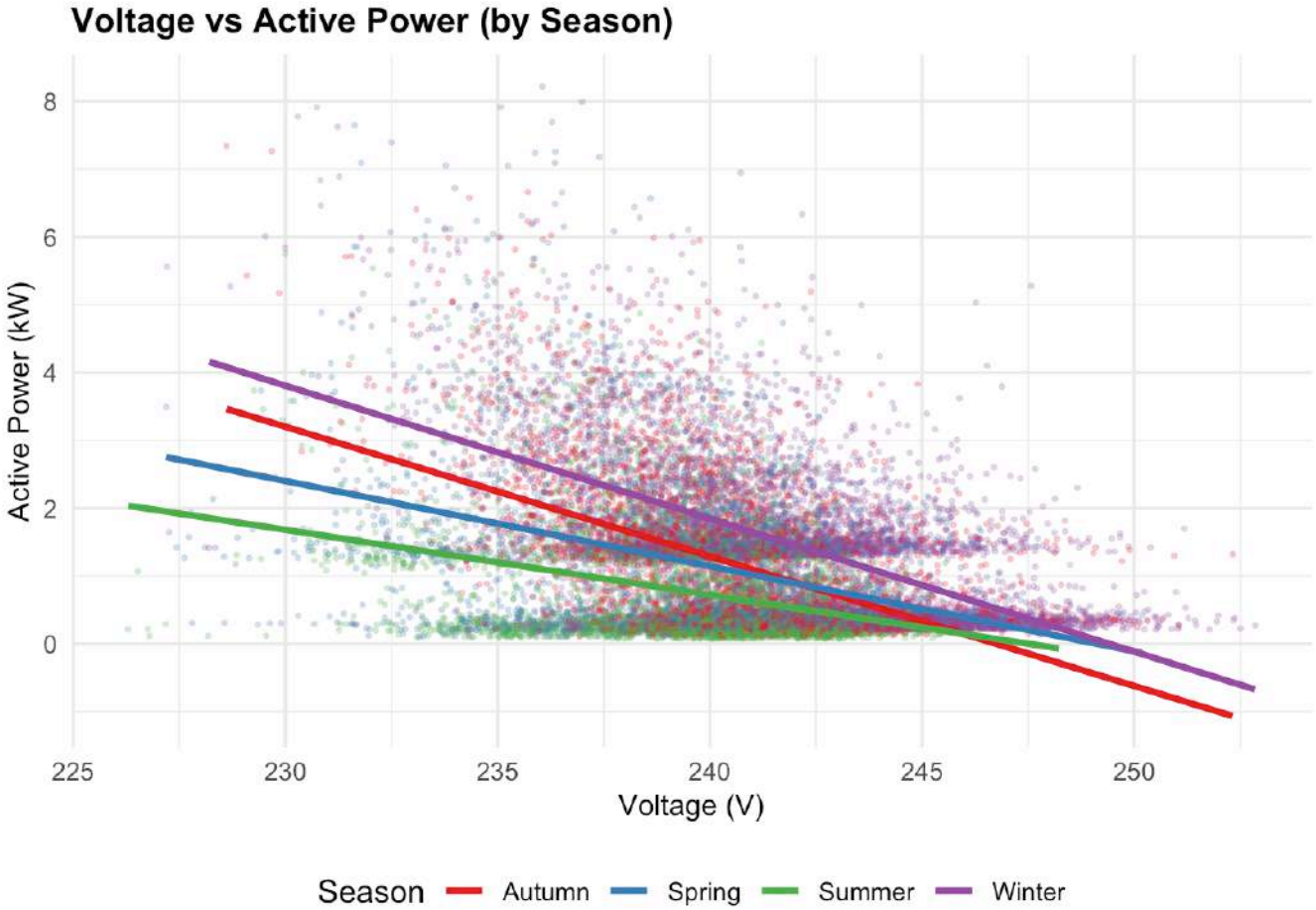



```
# P11 — Violin + boxplot by season
p11 <- ggplot(df %>% sample_n(80000),
              aes(x = season, y = Global_active_power, fill = season)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.08, fill = "white", outlier.size = 0.2) +
  scale_fill_brewer(palette = "Pastel1") +
  labs(title = "Power Distribution by Season (Violin + Box)",
       x = "Season", y = "Active Power (kW)") +
  theme_load + theme(legend.position = "none")

p11
```



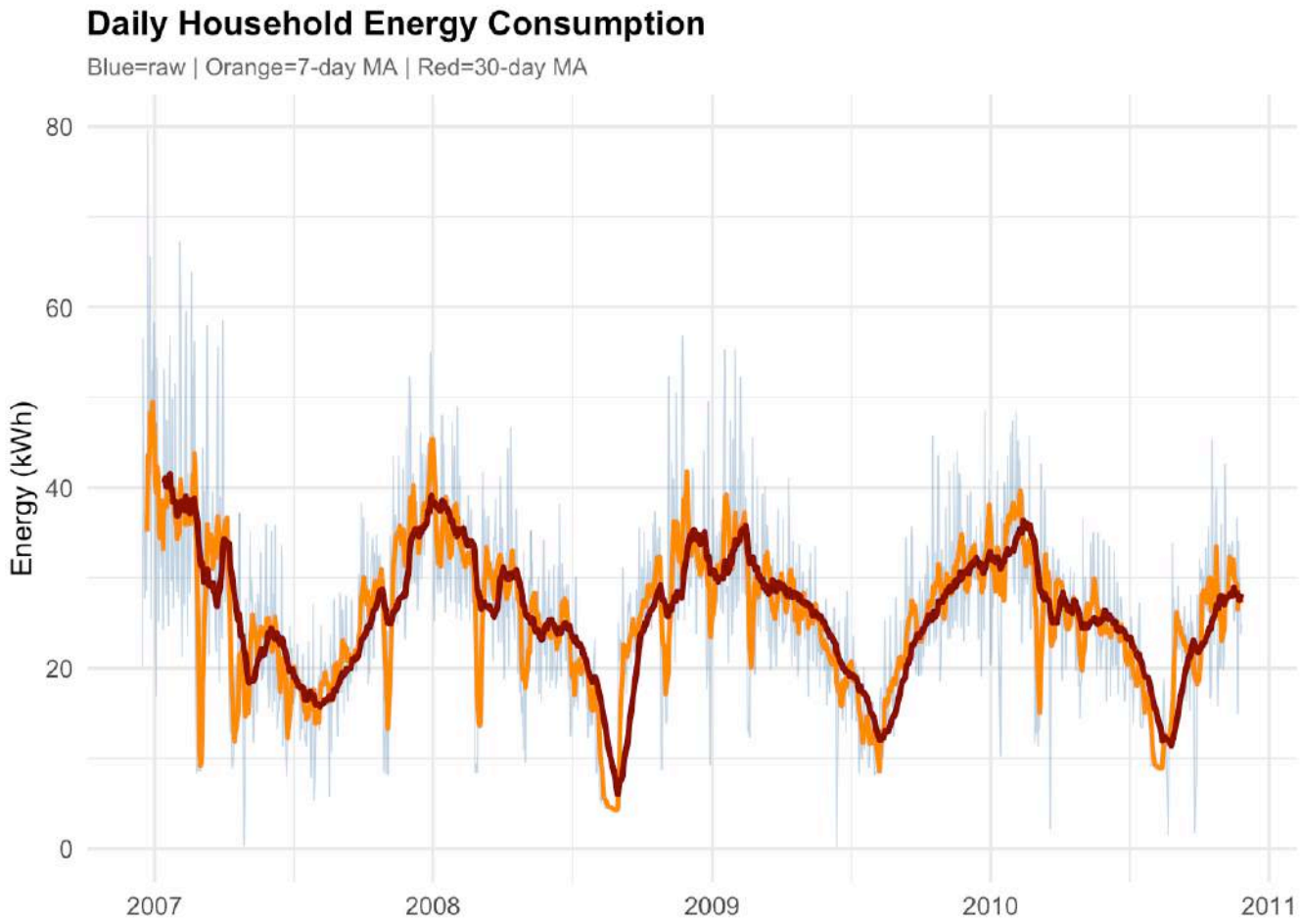

```
# P12 — Voltage vs active power with per-season regression lines
p12 <- ggplot(df %>% sample_n(20000),
              aes(x = Voltage, y = Global_active_power, colour = season)) +
  geom_point(alpha = 0.2, size = 0.5) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 1.2) +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "Voltage vs Active Power (by Season)",
       x = "Voltage (V)", y = "Active Power (kW)", colour = "Season") + theme_load

p12
```

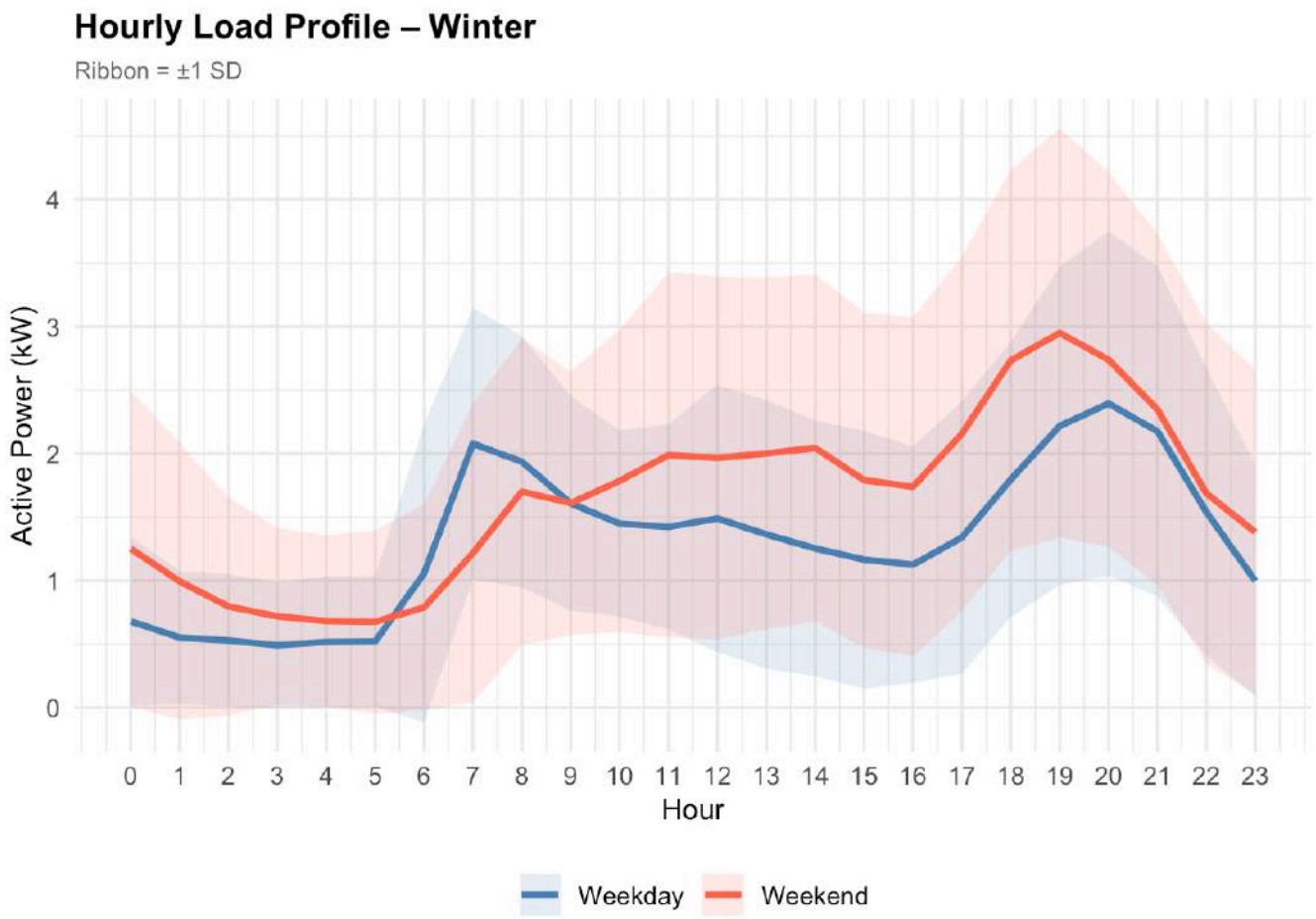
```
## `geom_smooth()` using formula = 'y ~ x'
```



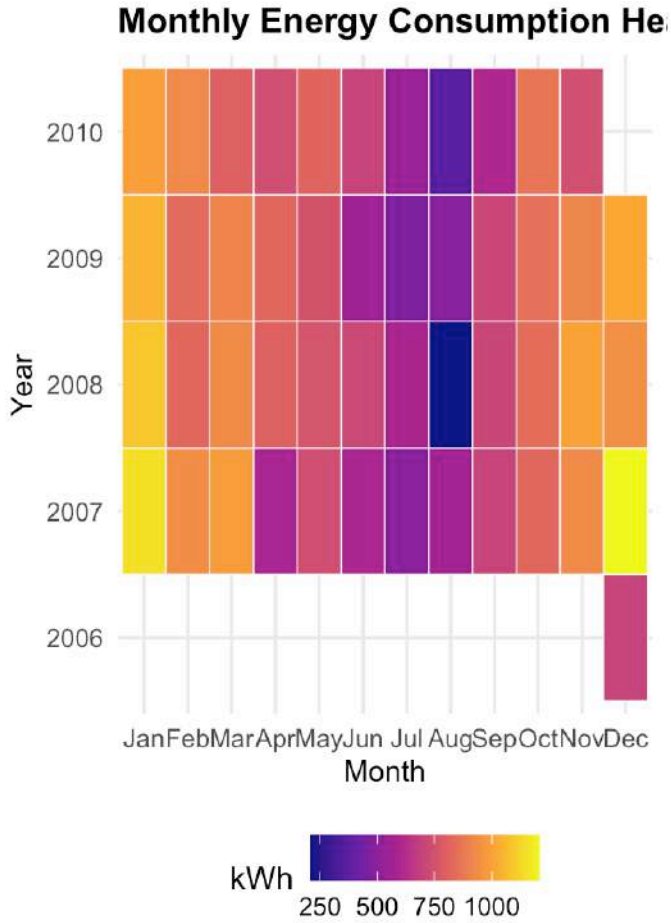

```
# Render all plots
print(p1); print(p2)
```

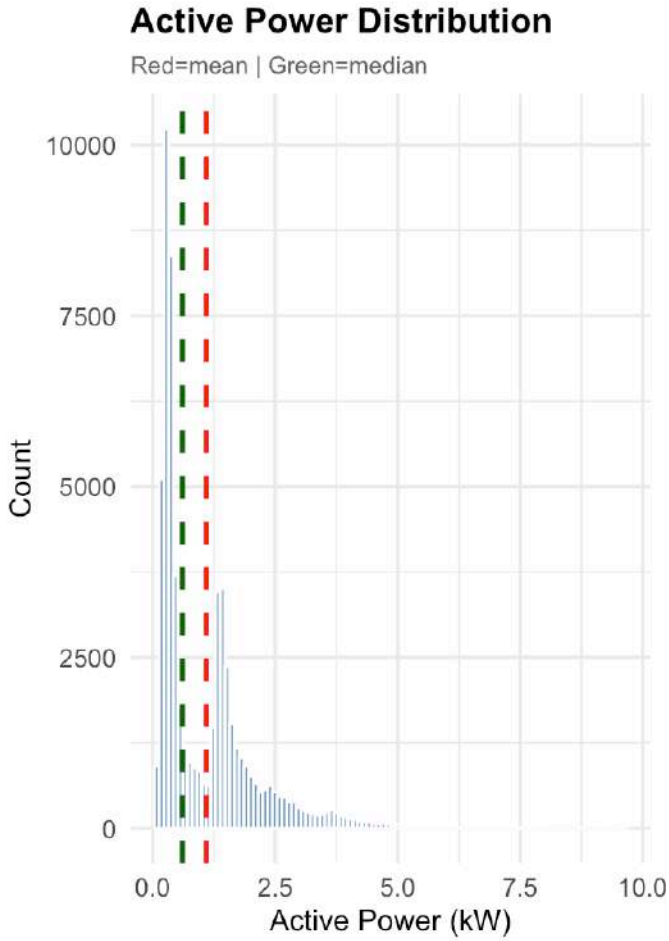
```
## Warning: Removed 6 rows containing missing values or values outside the scale rang
e
## (`geom_line()`).
## Removed 29 rows containing missing values or values outside the scale range
## (`geom_line()`).
```



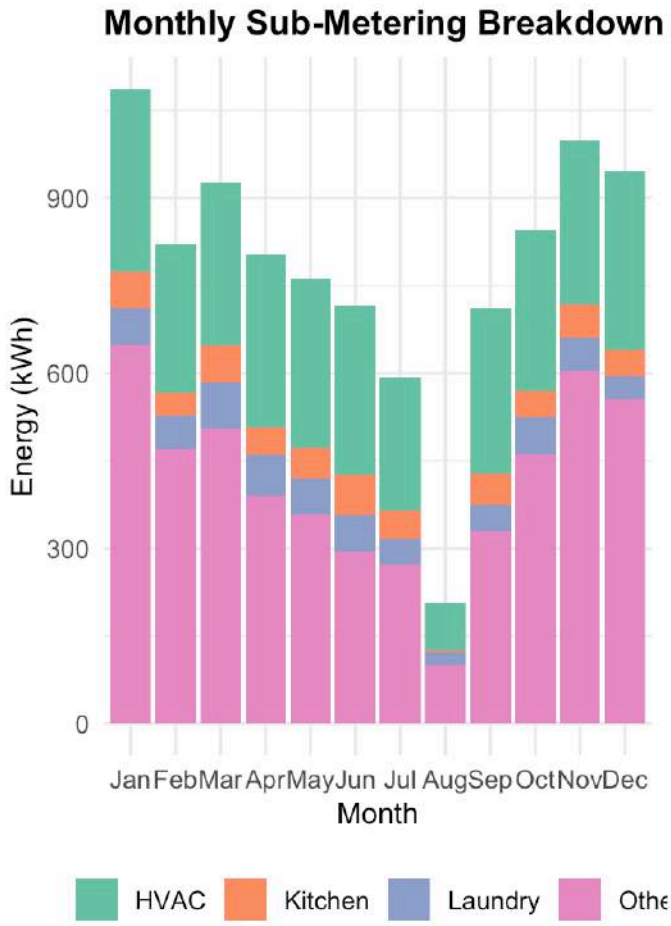


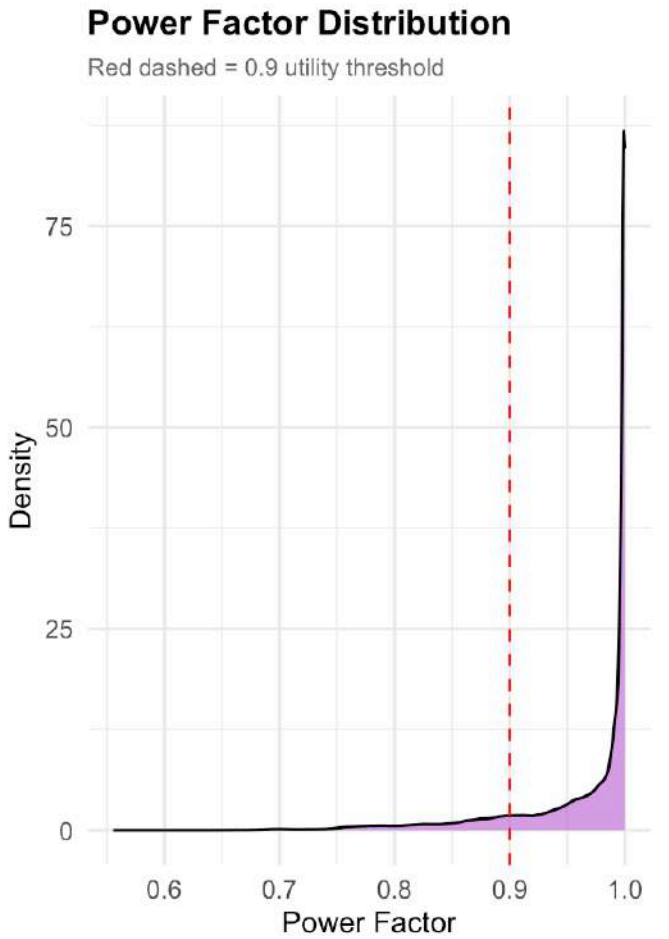

```
grid.arrange(p3, p4, ncol = 2)
```



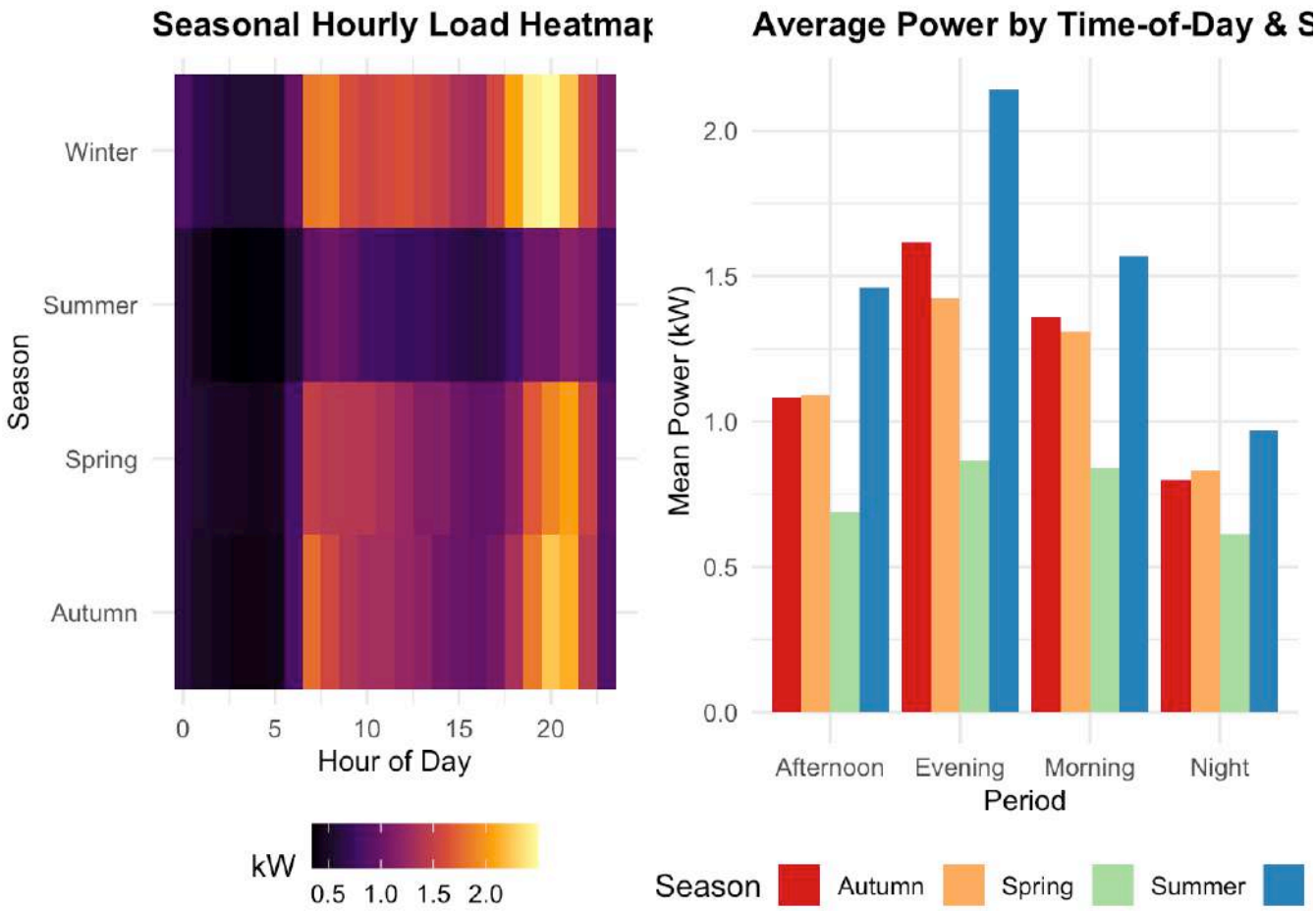



```
grid.arrange(p5, p6, ncol = 2)
```

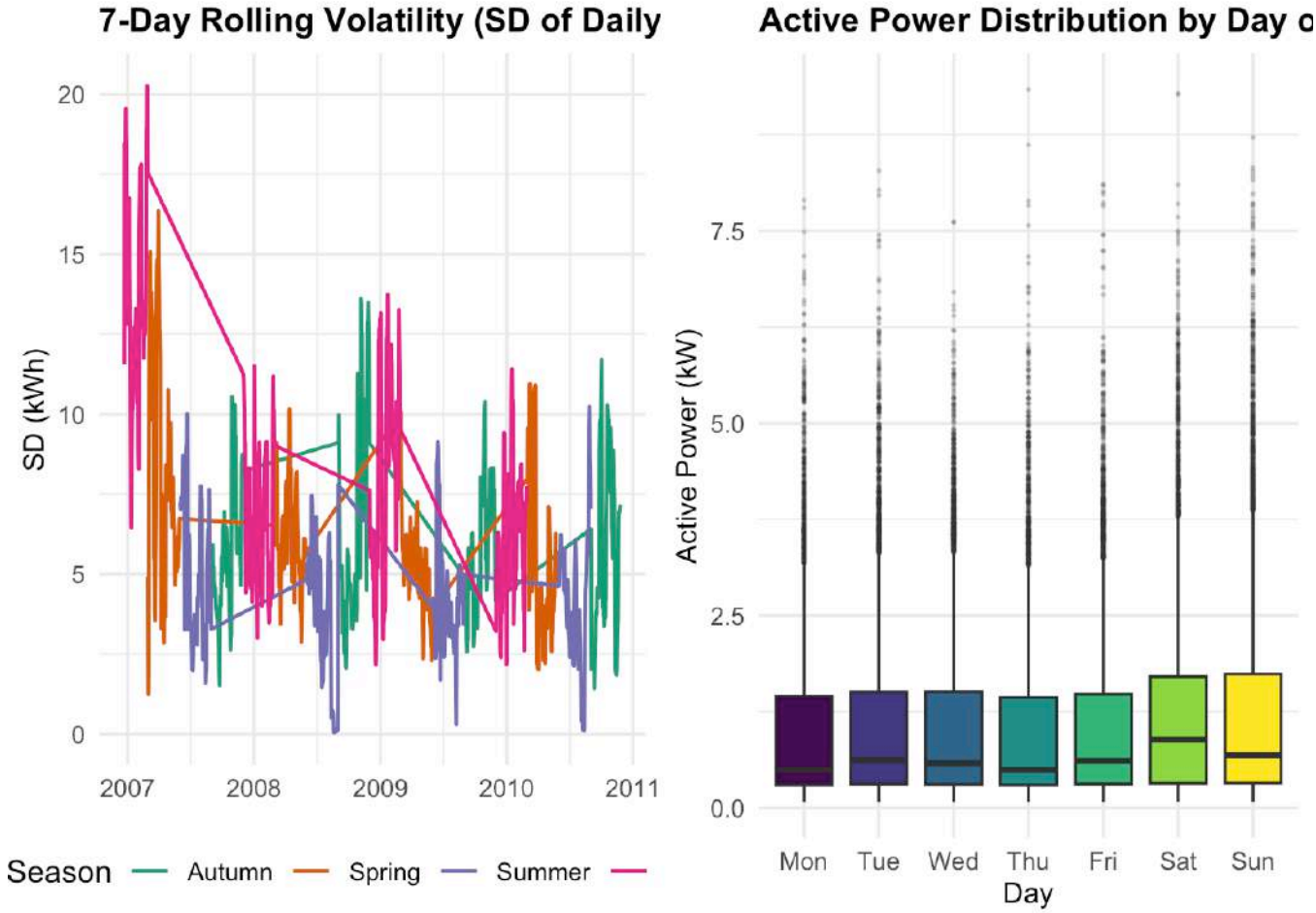




```
grid.arrange(p7, p8, ncol = 2)
```

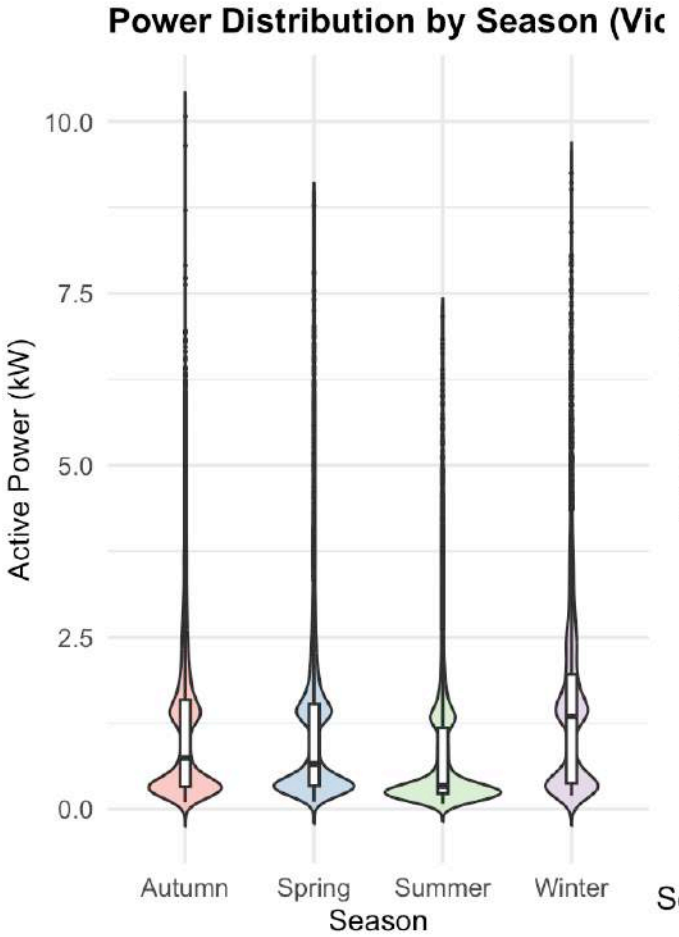


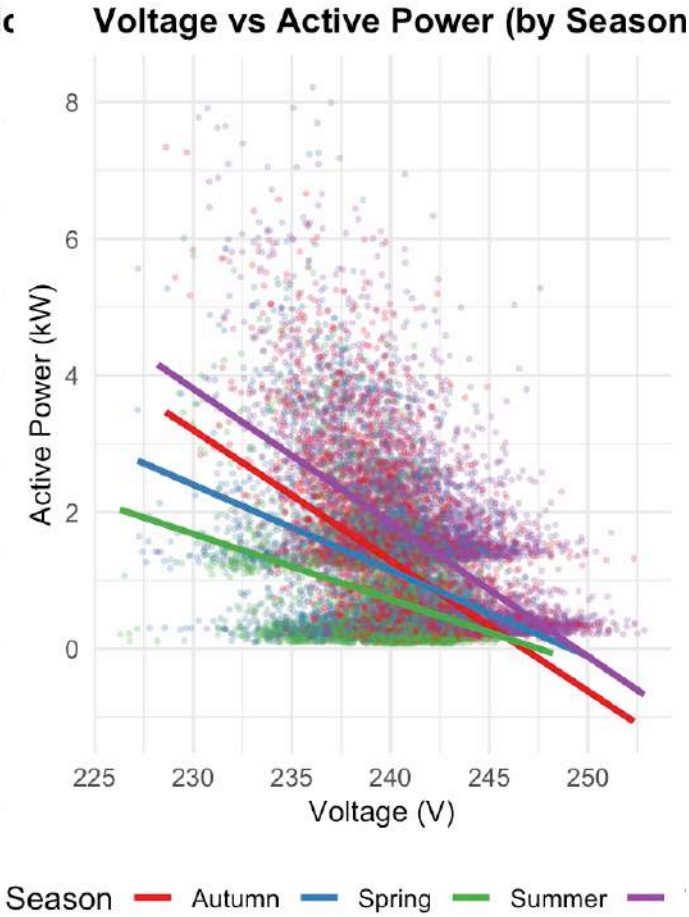

```
grid.arrange(p9, p10, ncol = 2)
```




```
grid.arrange(p11, p12, ncol = 2)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```






```
cat("✅ 12 visualisations rendered.\n")
```

```
## ✅ 12 visualisations rendered.
```

```
# ── 6. Correlation Analysis ──────────────────────────────────────────
cat("\n🔗 Section 6: Correlation & Pair Analysis\n")
```

```
##
## 🔗 Section 6: Correlation & Pair Analysis
```

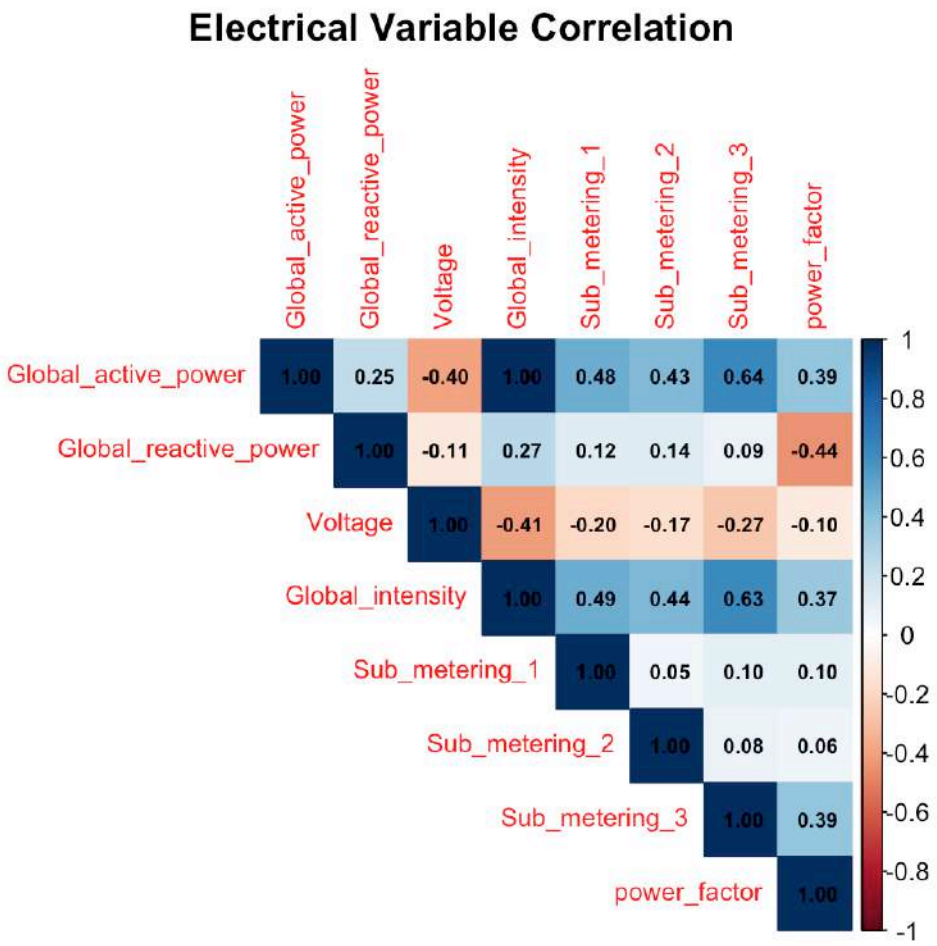
```
corr_df <- df %>%
  select(Global_active_power, Global_reactive_power,
         Voltage, Global_intensity,
         Sub_metering_1, Sub_metering_2, Sub_metering_3, power_factor) %>%
  drop_na()

corr_mat <- cor(corr_df)
cat("Full correlation matrix:\n")
```

```
## Full correlation matrix:
```

```
print(round(corr_mat, 3))
```

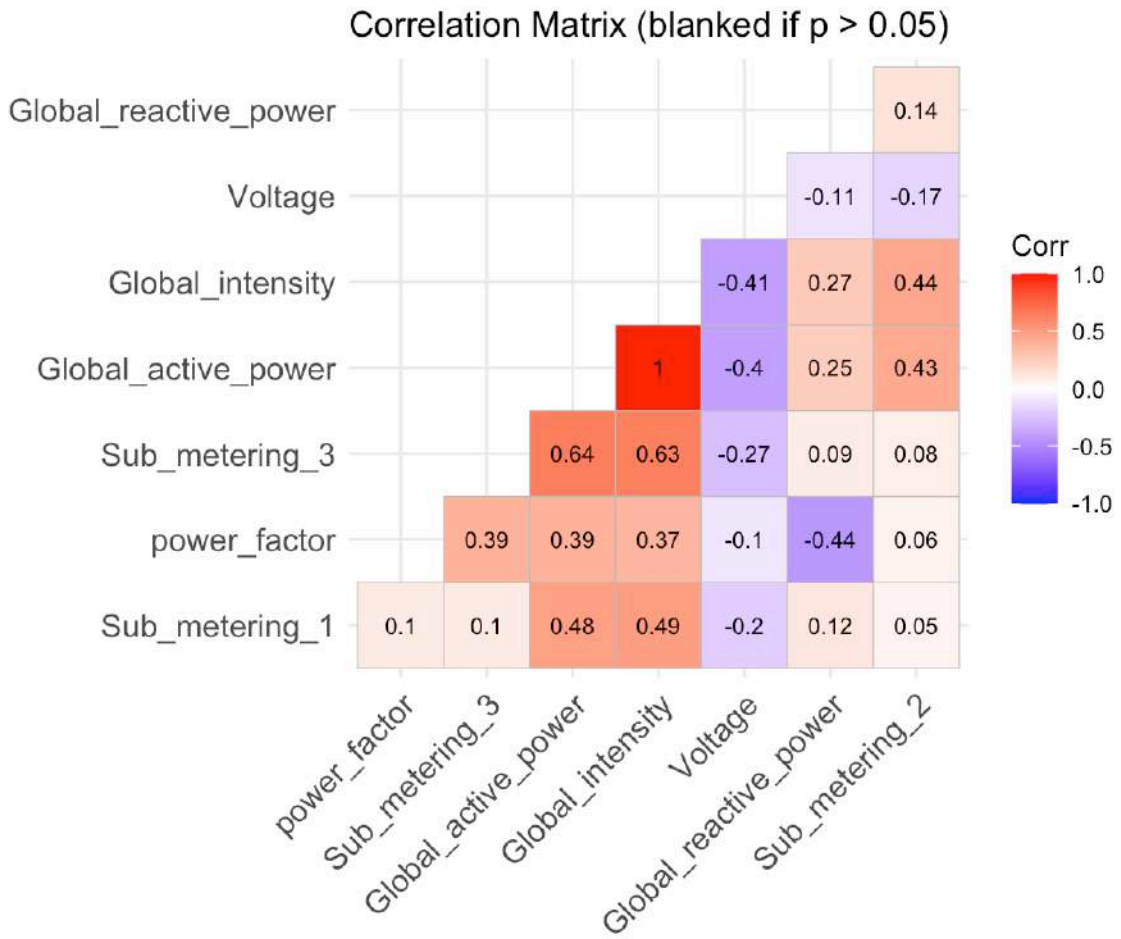
```
##                       Global_active_power Global_reactive_power Voltage
## Global_active_power                 1.000                 0.247  -0.400
## Global_reactive_power               0.247                 1.000  -0.112
## Voltage                            -0.400                -0.112   1.000
## Global_intensity                    0.999                 0.266  -0.411
## Sub_metering_1                      0.484                 0.123  -0.196
## Sub_metering_2                      0.435                 0.139  -0.167
## Sub_metering_3                      0.639                 0.090  -0.268
## power_factor                        0.385                -0.441  -0.101
##                       Global_intensity Sub_metering_1 Sub_metering_2
## Global_active_power              0.999          0.484          0.435
## Global_reactive_power            0.266          0.123          0.139
## Voltage                         -0.411         -0.196         -0.167
## Global_intensity                 1.000          0.489          0.440
## Sub_metering_1                   0.489          1.000          0.055
## Sub_metering_2                   0.440          0.055          1.000
## Sub_metering_3                   0.627          0.103          0.081
## power_factor                     0.366          0.104          0.063
##                       Sub_metering_3 power_factor
## Global_active_power            0.639        0.385
## Global_reactive_power          0.090       -0.441
## Voltage                       -0.268       -0.101
## Global_intensity               0.627        0.366
## Sub_metering_1                 0.103        0.104
## Sub_metering_2                 0.081        0.063
## Sub_metering_3                 1.000        0.386
## power_factor                   0.386        1.000
```

```
# Base corrplot
corrplot(corr_mat, method = "color", type = "upper",
         tl.cex = 0.8, addCoef.col = "black", number.cex = 0.65,
         title  = "Electrical Variable Correlation", mar = c(0, 0, 2, 0))
```

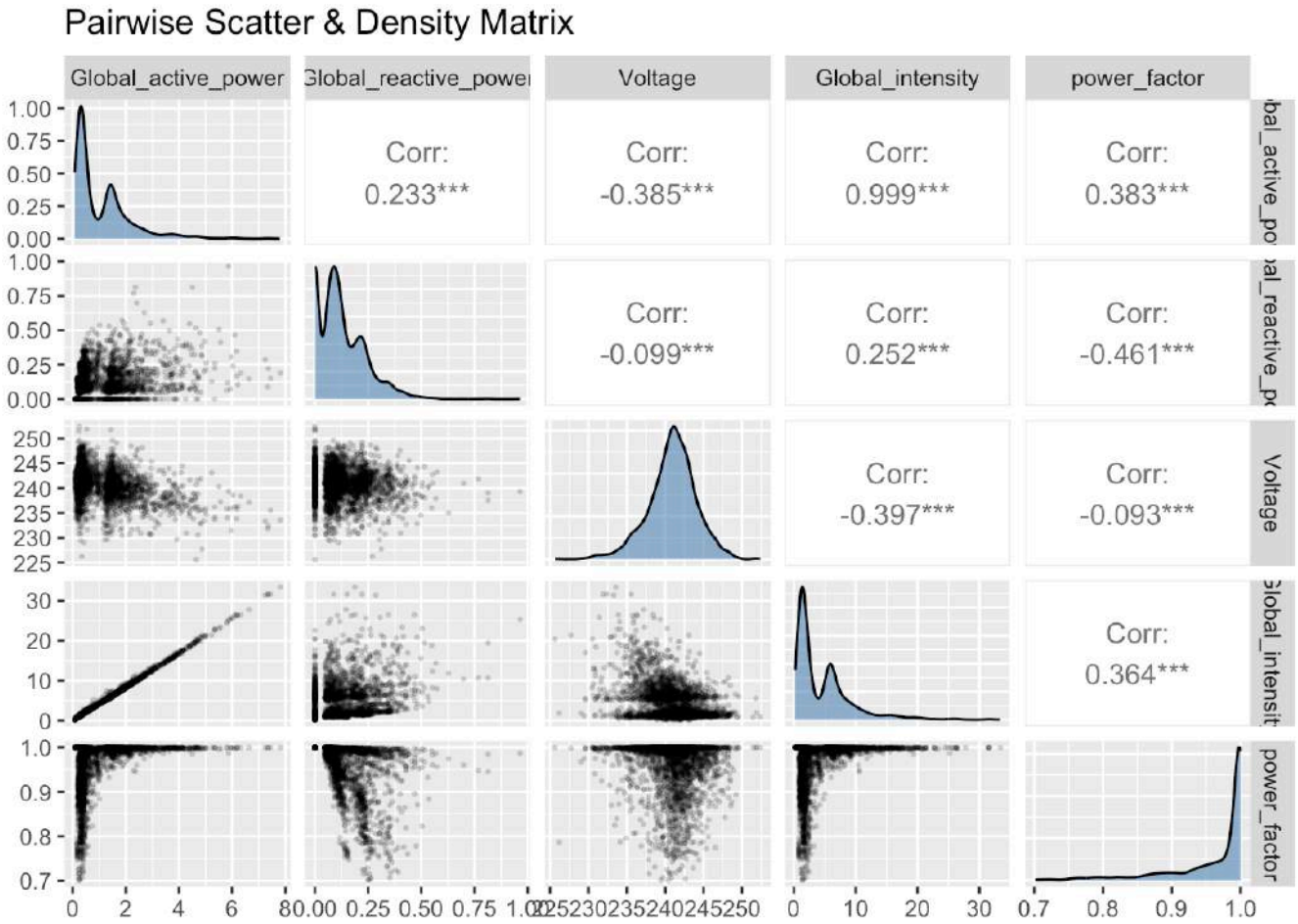



```
# ggcorrplot with p-value significance blanking
p_mat <- cor_pmat(corr_df)
ggcorrplot(corr_mat, hc.order = TRUE, type = "lower",
           lab = TRUE, lab_size = 3,
           p.mat = p_mat, sig.level = 0.05, insig = "blank",
           title = "Correlation Matrix (blanked if p > 0.05)")
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## ℹ Please use tidy evaluation idioms with `aes()`.
## ℹ See also `vignette("ggplot2-in-packages")` for more information.
## ℹ The deprecated feature was likely used in the ggcorrplot package.
##   Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



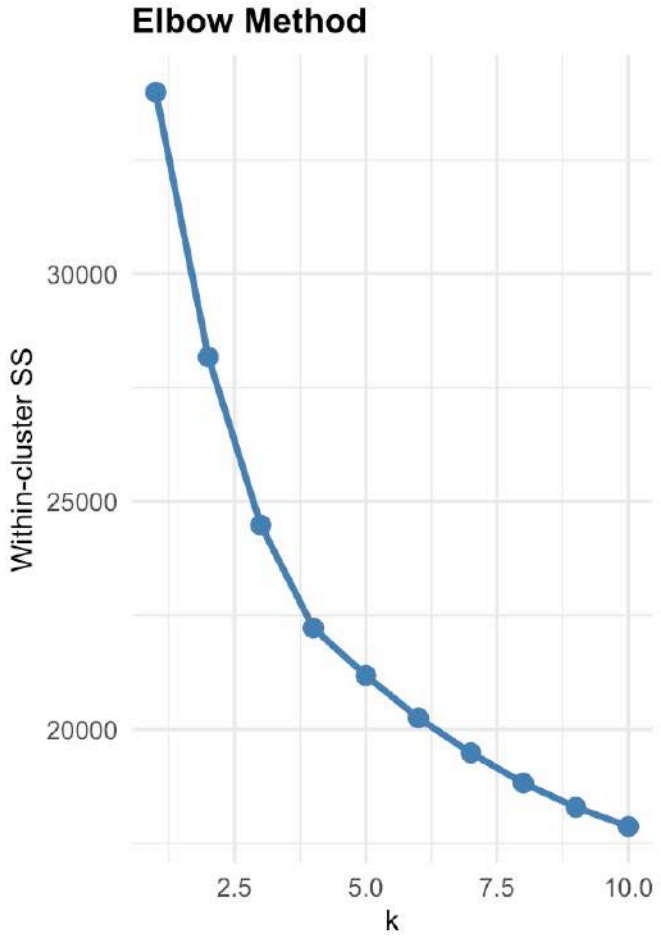

```
# GGpairs pairwise scatter/density matrix
ggpairs(
  df %>% sample_n(3000) %>%
    select(Global_active_power, Global_reactive_power,
           Voltage, Global_intensity, power_factor),
  title = "Pairwise Scatter & Density Matrix",
  lower = list(continuous = wrap("points", alpha = 0.15, size = 0.4)),
  diag  = list(continuous = wrap("densityDiag", fill = "steelblue", alpha = 0.6))
)
```




```
# ── 7. K-Means Clustering - FIXED silhouette ──────────────────────────────
cat("\n🔍 Section 7: K-Means Load Pattern Clustering\n")
```

```
##
## 🔍 Section 7: K-Means Load Pattern Clustering
```

```
hourly_wide <- df %>%
  group_by(Date, hour) %>%
  summarise(mean_power = mean(Global_active_power, na.rm = TRUE), .groups = "drop") %
>%
  pivot_wider(names_from = hour, values_from = mean_power, names_prefix = "h") %>%
  drop_na()

cm <- scale(as.matrix(hourly_wide[ , -1]))   # matrix, not data frame
n_rows <- nrow(cm)

# Elbow method
set.seed(42)
wss <- sapply(1:10, function(k) {
  kmeans(cm, centers = k, nstart = 25, iter.max = 50)$tot.withinss
})

# FIXED silhouette: sample SAME rows for both cluster vector & dist matrix
SIL_N <- min(500, n_rows)

set.seed(42)
sil_scores <- sapply(2:8, function(k) {
  km_full   <- kmeans(cm, centers = k, nstart = 25, iter.max = 50)
  samp_idx  <- sample(n_rows, SIL_N)
  sil_obj   <- silhouette(
    x    = km_full$cluster[samp_idx],
    dist = dist(cm[samp_idx, ])
  )
  mean(sil_obj[, 3])
})

# Plot elbow + silhouette
elbow_df <- data.frame(k = 1:10, wss = wss)
sil_df   <- data.frame(k = 2:8,  sil = sil_scores)

pe <- ggplot(elbow_df, aes(k, wss)) +
  geom_line(colour = "steelblue", linewidth = 1.2) +
  geom_point(size = 3, colour = "steelblue") +
  labs(title = "Elbow Method", x = "k", y = "Within-cluster SS") + theme_load

ps <- ggplot(sil_df, aes(k, sil)) +
  geom_line(colour = "tomato", linewidth = 1.2) +
  geom_point(size = 3, colour = "tomato") +
  labs(title = "Silhouette Score", x = "k", y = "Avg Silhouette Width") + theme_load

grid.arrange(pe, ps, ncol = 2)
```



```
best_k <- sil_df$k[which.max(sil_df$sil)]
cat(sprintf("Elbow suggests k ≈ 3. Silhouette best k = %d.\n", best_k))
```

```
## Elbow suggests k ≈ 3. Silhouette best k = 2.
```

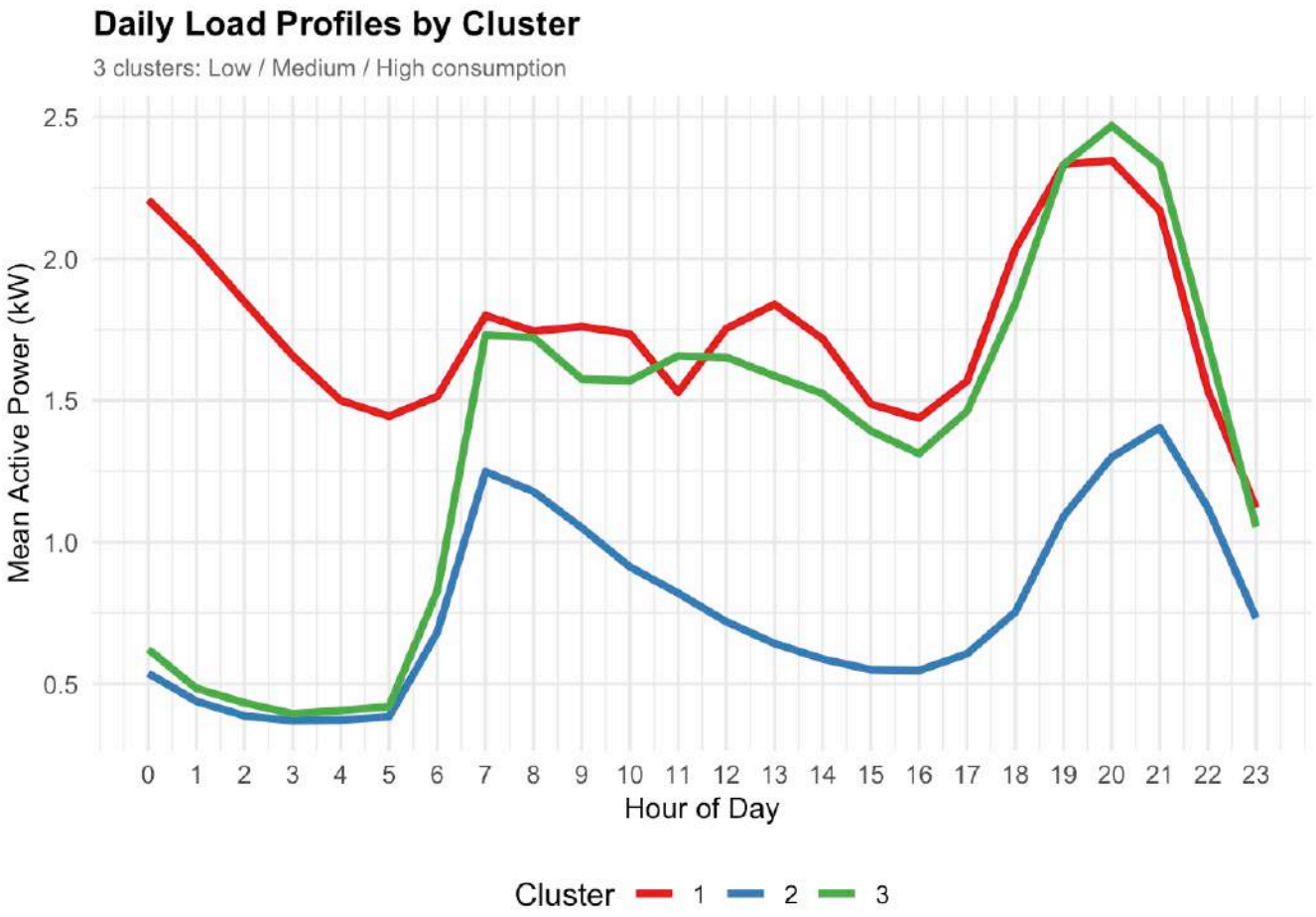
```
cat("Proceeding with k = 3 (Low / Medium / High load days).\n")
```

```
## Proceeding with k = 3 (Low / Medium / High load days).
```

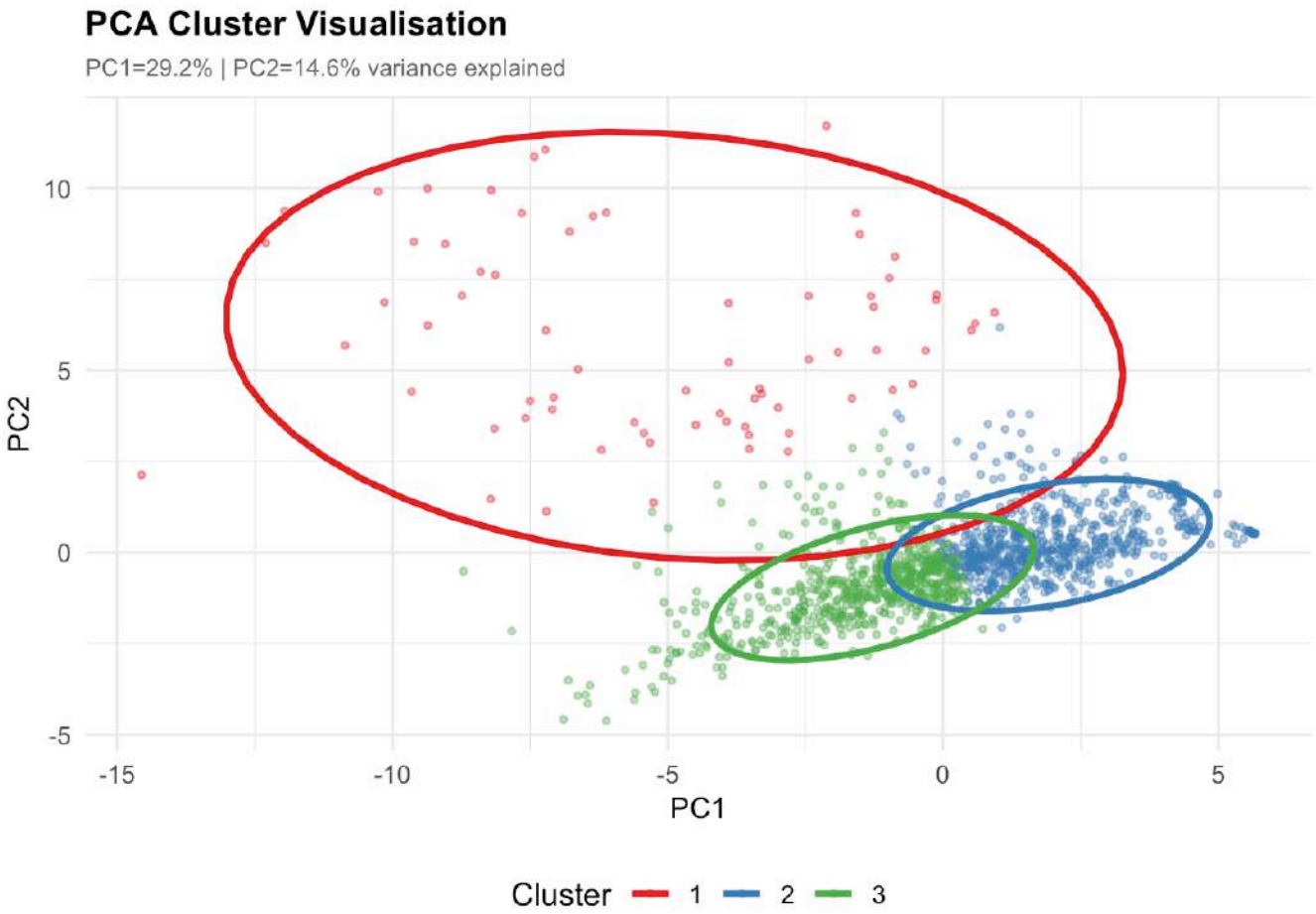
```
# Final k=3 model
set.seed(42)
km3 <- kmeans(cm, centers = 3, nstart = 50, iter.max = 100)
hourly_wide$cluster <- factor(km3$cluster)
cat(sprintf("Cluster sizes: %s\n",
            paste(names(table(km3$cluster)),
                  table(km3$cluster), sep = "=", collapse = " | ")))
```

```
## Cluster sizes: 1=70 | 2=683 | 3=664
```

```
# Cluster daily-profile plot
cluster_long <- hourly_wide %>%
  pivot_longer(starts_with("h"), names_to = "hour_str", values_to = "power") %>%
  mutate(hour = as.integer(sub("h", "", hour_str))) %>%
  group_by(cluster, hour) %>%
  summarise(mean_power = mean(power, na.rm = TRUE), .groups = "drop")

ggplot(cluster_long, aes(x = hour, y = mean_power,
                         colour = cluster, group = cluster)) +
  geom_line(linewidth = 1.4) +
  scale_x_continuous(breaks = 0:23) +
  scale_colour_brewer(palette = "Set1") +
  labs(title    = "Daily Load Profiles by Cluster",
       subtitle = "3 clusters: Low / Medium / High consumption",
       x = "Hour of Day", y = "Mean Active Power (kW)", colour = "Cluster") +
  theme_load
```



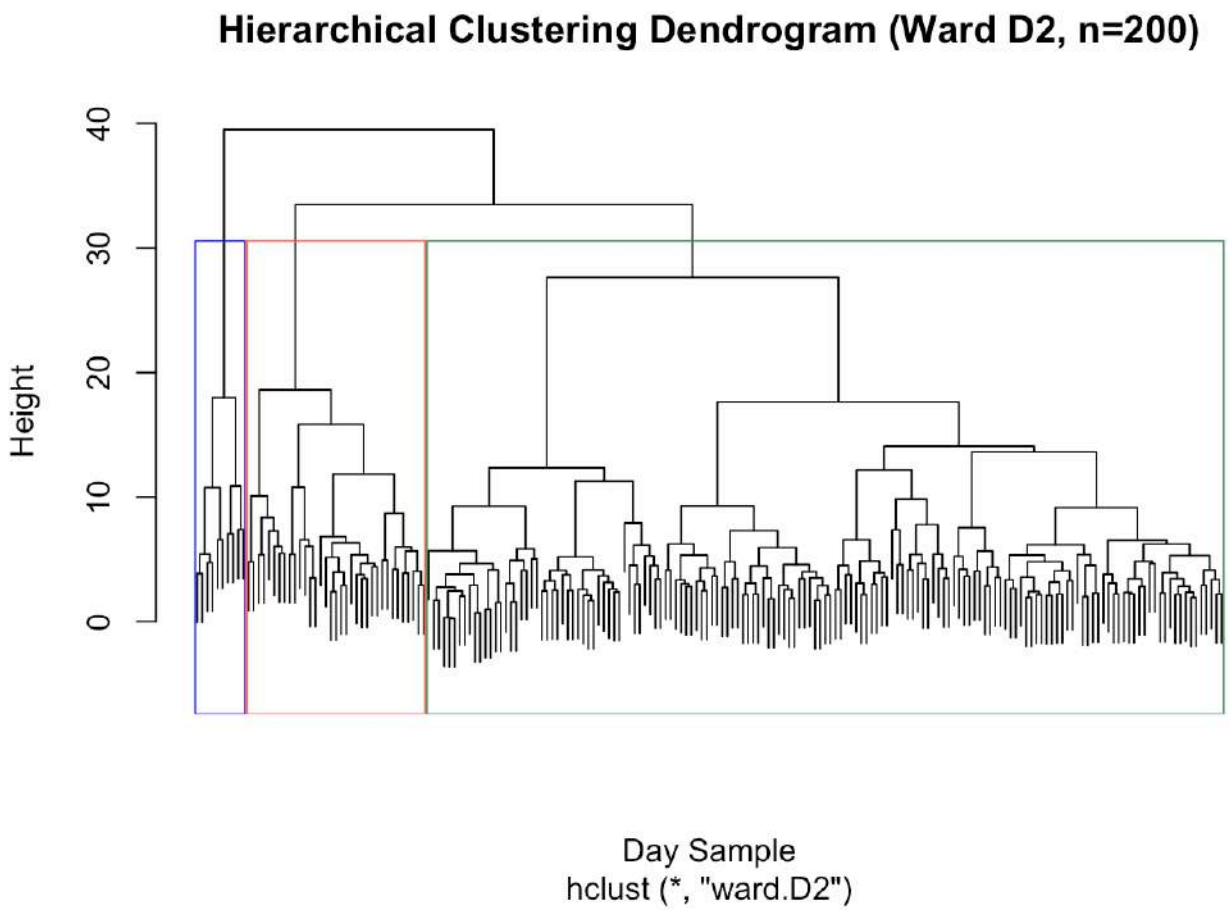
```
# PCA cluster visualisation
pca_res <- prcomp(cm, scale. = FALSE)
pca_df  <- data.frame(pca_res$x[, 1:2], cluster = hourly_wide$cluster)
pve     <- summary(pca_res)$importance[2, 1:2] * 100

ggplot(pca_df, aes(PC1, PC2, colour = cluster)) +
  geom_point(alpha = 0.4, size = 0.9) +
  stat_ellipse(linewidth = 1.2) +
  scale_colour_brewer(palette = "Set1") +
  labs(title    = "PCA Cluster Visualisation",
       subtitle = sprintf("PC1=%.1f%% | PC2=%.1f%% variance explained",
                          pve[1], pve[2]),
       colour = "Cluster") + theme_load
```




```
# ── 8. Hierarchical Clustering (dendrogram) ──────────────────────────────
cat("\n🌳 Section 8: Hierarchical Clustering\n")
```

```
##
## 🌳 Section 8: Hierarchical Clustering
```

```
set.seed(42)
hc_idx    <- sample(n_rows, 200)
hc_sample <- cm[hc_idx, ]
hc_dist   <- dist(hc_sample, method = "euclidean")
hc_model  <- hclust(hc_dist, method = "ward.D2")

plot(hc_model,
     labels = FALSE,
     main   = "Hierarchical Clustering Dendrogram (Ward D2, n=200)",
     xlab   = "Day Sample", ylab = "Height")
rect.hclust(hc_model, k = 3, border = c("blue", "tomato", "seagreen"))
```




```
cat("Dendrogram rendered (3 groups highlighted).\n")
```

```
## Dendrogram rendered (3 groups highlighted).
```

```
# ── 9. Time Series: Stationarity & Decomposition ──────────────────────────
cat("\n📈 Section 9: Time Series Analysis\n")
```

```
##
## 📈 Section 9: Time Series Analysis
```

```
daily_ts <- ts(daily$total_kwh,
               start     = c(year(min(daily$Date)), yday(min(daily$Date))),
               frequency = 365)

# ADF Test
adf_r <- adf.test(na.omit(daily_ts))
```

```
## Warning in adf.test(na.omit(daily_ts)): p-value smaller than printed p-value
```

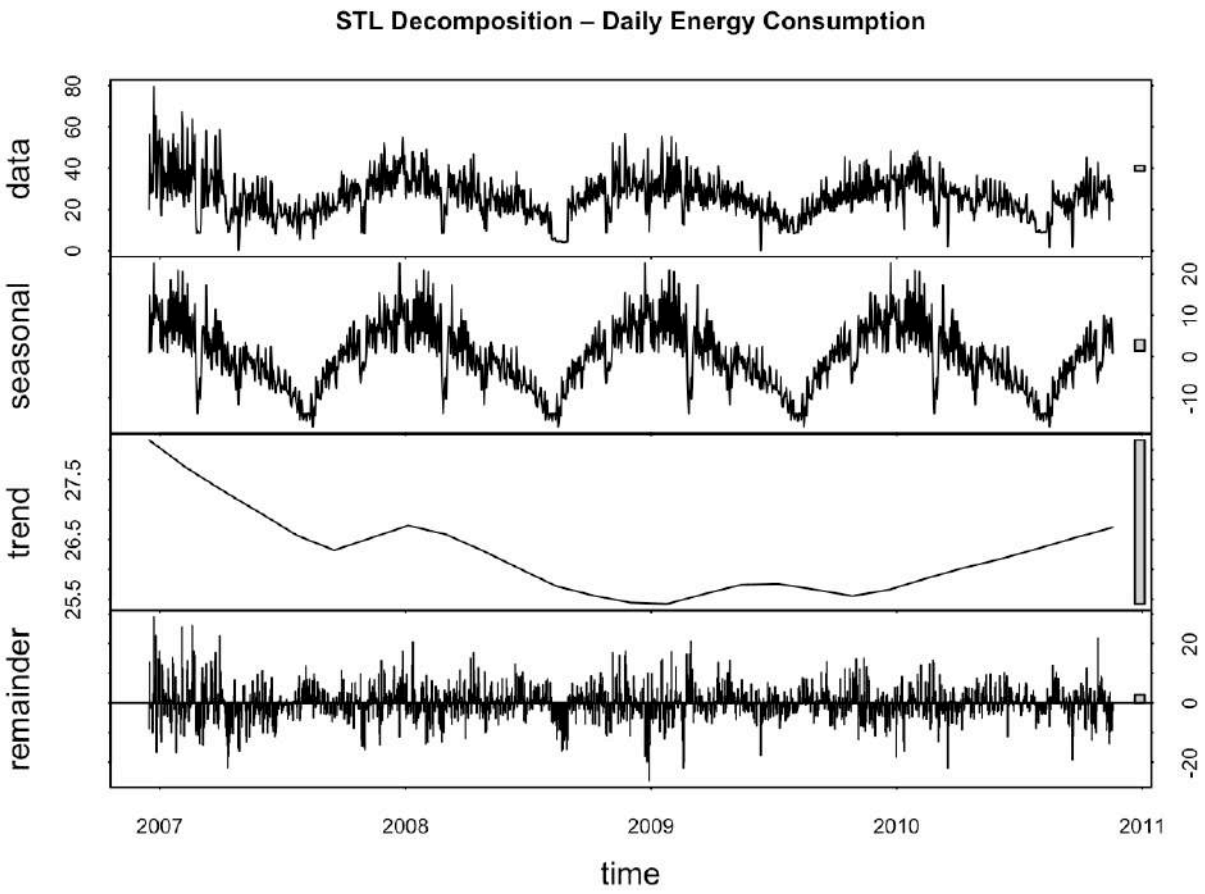
```
cat(sprintf("ADF:  stat=%.4f, p=%.4f → %s\n",
            adf_r$statistic, adf_r$p.value,
            ifelse(adf_r$p.value < 0.05, "STATIONARY ✅", "NON-STATIONARY ⚠️")))
```

```
## ADF:  stat=-5.1601, p=0.0100 → STATIONARY ✅
```

```
# KPSS Test
kpss_r <- kpss.test(na.omit(daily_ts))
cat(sprintf("KPSS: stat=%.4f, p=%.4f → %s\n",
            kpss_r$statistic, kpss_r$p.value,
            ifelse(kpss_r$p.value > 0.05, "STATIONARY ✅", "NON-STATIONARY ⚠️")))
```

```
## KPSS: stat=0.5249, p=0.0361 → NON-STATIONARY ⚠️
```

```
# STL decomposition
stl_d <- stl(na.approx(daily_ts), s.window = "periodic")
plot(stl_d, main = "STL Decomposition — Daily Energy Consumption")
```



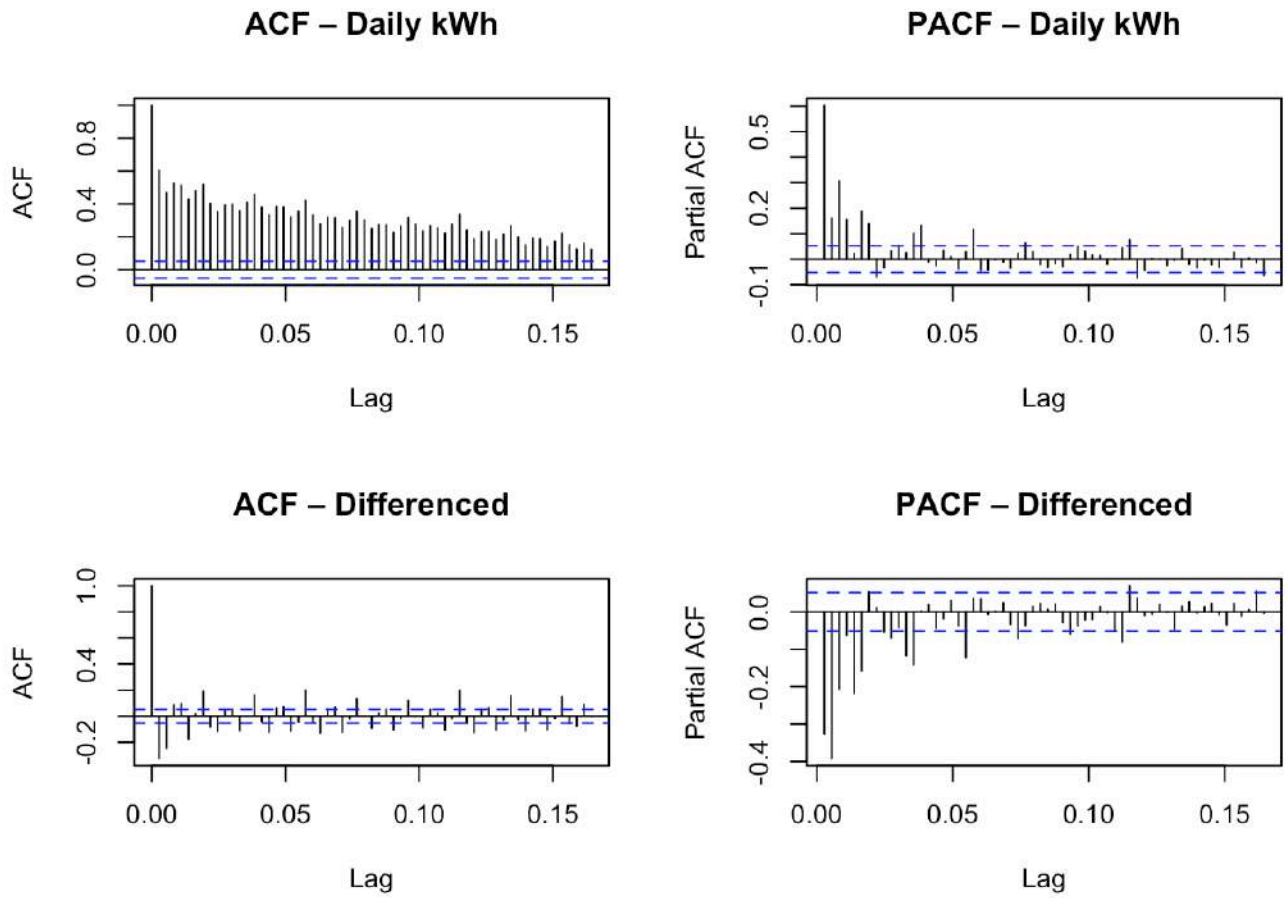

```
# Differenced series
diff_ts  <- diff(na.approx(daily_ts))
adf_diff <- adf.test(diff_ts)
```

```
## Warning in adf.test(diff_ts): p-value smaller than printed p-value
```

```
cat(sprintf("ADF (differenced): p=%.4f → %s\n",
            adf_diff$p.value,
            ifelse(adf_diff$p.value < 0.05, "STATIONARY ✅", "NON-STATIONARY ⚠️")))
```

```
## ADF (differenced): p=0.0100 → STATIONARY ✅
```

```
# ACF / PACF — 2×2 panel
par(mfrow = c(2, 2))
acf(na.omit(daily_ts),  lag.max = 60, main = "ACF — Daily kWh")
pacf(na.omit(daily_ts), lag.max = 60, main = "PACF — Daily kWh")
acf(diff_ts,            lag.max = 60, main = "ACF — Differenced")
pacf(diff_ts,           lag.max = 60, main = "PACF — Differenced")
```




```
par(mfrow = c(1, 1))

# ── 10. Change-Point Detection ──────────────────────────────────────────
# FIX: switched penalty from "BIC" to "MBIC" to prevent near-every-day
#      false detections on a high-frequency ts (frequency = 365).
cat("\n🔎 Section 10: Change-Point Detection (PELT / MBIC)\n")
```

```
##
## 🔎 Section 10: Change-Point Detection (PELT / MBIC)
```

```
ts_clean <- na.approx(daily_ts)

cpt_m <- cpt.mean(ts_clean, method = "PELT", penalty = "MBIC")
cpt_v <- cpt.var(ts_clean,  method = "PELT", penalty = "MBIC")

cat(sprintf("Mean change-points detected   : %d\n", length(cpts(cpt_m))))
```

```
## Mean change-points detected   : 588
```

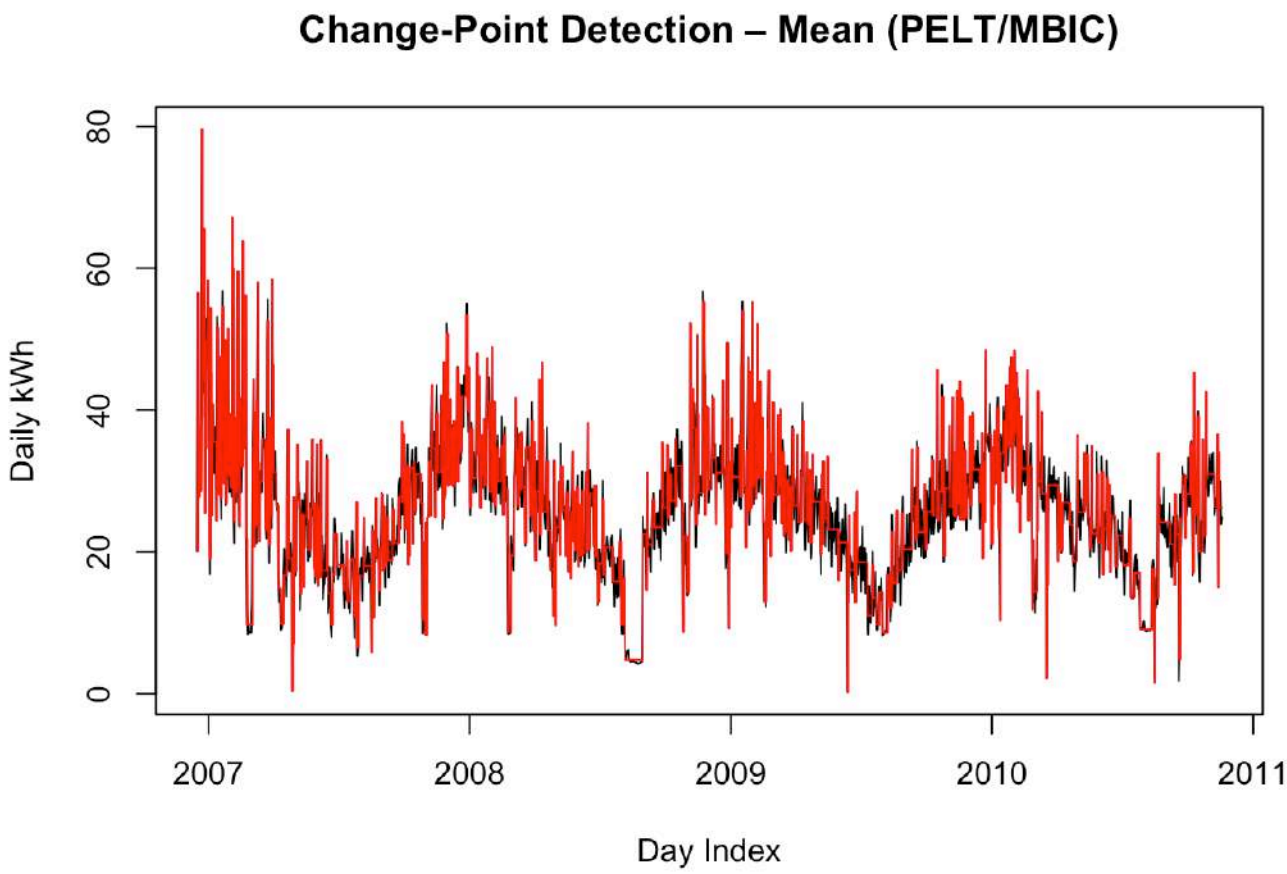
```
cat(sprintf("Mean change-points at index   : %s\n",
            paste(cpts(cpt_m), collapse = ", ")))
```

```
## Mean change-points at index   : 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 15, 16, 1
7, 19, 20, 21, 22, 23, 27, 28, 30, 31, 32, 33, 35, 37, 38, 39, 40, 42, 43, 44, 46, 4
7, 49, 50, 51, 53, 54, 56, 57, 58, 59, 60, 61, 64, 65, 66, 67, 68, 69, 70, 78, 79, 8
0, 81, 82, 84, 85, 86, 87, 88, 91, 96, 98, 100, 102, 103, 105, 106, 107, 111, 114, 11
7, 121, 126, 127, 128, 133, 134, 135, 136, 139, 140, 141, 144, 146, 149, 150, 154, 15
5, 159, 161, 162, 163, 164, 165, 167, 168, 169, 170, 172, 173, 174, 181, 183, 187, 19
0, 192, 196, 205, 209, 210, 215, 219, 221, 223, 224, 226, 233, 234, 244, 245, 247, 24
8, 250, 251, 254, 256, 258, 260, 261, 262, 264, 267, 273, 275, 282, 286, 287, 289, 29
0, 293, 295, 296, 297, 298, 302, 303, 304, 306, 313, 315, 318, 319, 320, 321, 322, 32
4, 326, 327, 328, 329, 333, 334, 337, 341, 342, 343, 345, 346, 348, 349, 351, 352, 35
4, 355, 356, 358, 359, 360, 363, 365, 366, 367, 370, 376, 378, 380, 381, 383, 385, 39
1, 393, 394, 395, 396, 397, 398, 401, 402, 404, 405, 406, 409, 412, 413, 414, 415, 41
6, 418, 421, 425, 428, 429, 431, 434, 435, 439, 443, 445, 446, 448, 450, 451, 453, 45
4, 458, 460, 465, 467, 468, 470, 471, 472, 473, 474, 478, 479, 481, 482, 483, 484, 48
8, 491, 494, 498, 499, 500, 501, 502, 506, 507, 508, 511, 515, 516, 519, 520, 522, 52
3, 525, 526, 527, 528, 531, 533, 534, 536, 538, 540, 544, 545, 546, 547, 549, 554, 55
5, 556, 558, 559, 560, 562, 566, 568, 571, 581, 590, 593, 596, 598, 623, 628, 629, 63
0, 636, 639, 650, 651, 657, 658, 660, 663, 668, 669, 670, 678, 679, 681, 685, 688, 68
9, 690, 693, 694, 697, 699, 700, 701, 707, 709, 710, 711, 712, 713, 715, 719, 720, 72
2, 725, 726, 734, 735, 736, 738, 741, 742, 743, 744, 745, 746, 747, 748, 758, 759, 76
1, 762, 764, 767, 768, 770, 771, 773, 774, 775, 776, 777, 780, 781, 782, 783, 784, 78
5, 787, 788, 790, 791, 793, 796, 797, 798, 799, 800, 801, 803, 804, 805, 806, 812, 81
3, 815, 816, 817, 818, 820, 821, 822, 825, 827, 828, 830, 831, 832, 834, 839, 840, 84
4, 846, 848, 852, 853, 856, 857, 860, 861, 871, 873, 874, 875, 876, 881, 882, 896, 89
7, 909, 910, 911, 912, 919, 920, 921, 922, 923, 937, 943, 945, 949, 953, 958, 965, 96
8, 971, 973, 978, 979, 985, 987, 999, 1000, 1004, 1006, 1008, 1017, 1018, 1020, 1027,
1029, 1034, 1035, 1037, 1041, 1043, 1046, 1051, 1052, 1056, 1057, 1059, 1060, 1062, 1
063, 1064, 1066, 1067, 1068, 1069, 1071, 1072, 1074, 1076, 1080, 1081, 1083, 1085, 10
94, 1097, 1098, 1099, 1101, 1102, 1103, 1110, 1112, 1113, 1114, 1115, 1118, 1120, 112
2, 1123, 1127, 1130, 1131, 1132, 1135, 1136, 1138, 1139, 1141, 1142, 1143, 1145, 114
6, 1147, 1149, 1150, 1151, 1152, 1153, 1160, 1161, 1163, 1167, 1168, 1169, 1174, 117
5, 1176, 1180, 1181, 1187, 1188, 1189, 1203, 1204, 1207, 1209, 1220, 1224, 1230, 123
1, 1240, 1241, 1243, 1245, 1248, 1250, 1251, 1257, 1259, 1264, 1266, 1267, 1271, 127
3, 1275, 1276, 1277, 1283, 1287, 1294, 1302, 1305, 1310, 1318, 1335, 1338, 1339, 134
3, 1344, 1360, 1366, 1367, 1368, 1372, 1374, 1377, 1381, 1382, 1388, 1390, 1391, 139
3, 1394, 1396, 1398, 1400, 1405, 1406, 1410, 1411, 1422, 1426, 1427, 1428, 1429
```

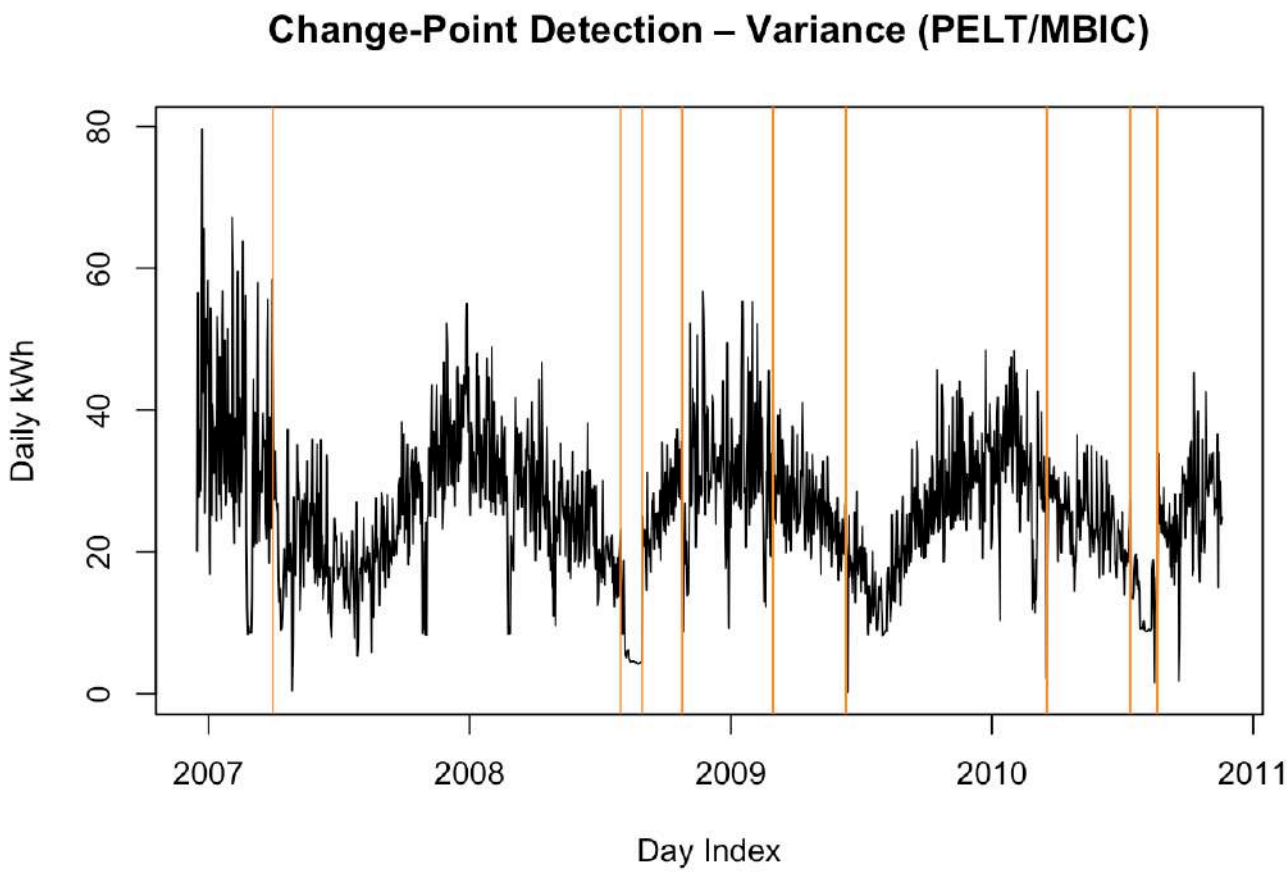
```
cat(sprintf("Variance change-points at index: %s\n",
            paste(cpts(cpt_v), collapse = ", ")))
```

```
## Variance change-points at index: 107, 593, 623, 679, 806, 908, 1189, 1305, 1343
```

```
plot(cpt_m,
     main = "Change-Point Detection — Mean (PELT/MBIC)",
     xlab = "Day Index", ylab = "Daily kWh", cpt.col = "red")
```




```
plot(cpt_v,
     main = "Change-Point Detection – Variance (PELT/MBIC)",
     xlab = "Day Index", ylab = "Daily kWh", cpt.col = "darkorange")
```




```
# ── 11. Anomaly Detection ──────────────────────────────────────────────
cat("\n🚨 Section 11: Anomaly Detection (Z-score & IQR)\n")
```

```
##
## 🚨 Section 11: Anomaly Detection (Z-score & IQR)
```

```
q1   <- quantile(daily$total_kwh, 0.25, na.rm = TRUE)
q3   <- quantile(daily$total_kwh, 0.75, na.rm = TRUE)
iqr  <- q3 - q1
mu   <- mean(daily$total_kwh, na.rm = TRUE)
sgm  <- sd(daily$total_kwh,   na.rm = TRUE)

anom_df <- daily %>%
  mutate(
    z_score     = (total_kwh - mu) / sgm,
    anomaly_z   = abs(z_score) > 3,
    anomaly_iqr = total_kwh < (q1 - 1.5 * iqr) | total_kwh > (q3 + 1.5 * iqr)
  )

cat(sprintf("Z-score anomalies (|z|>3) : %d days (%.1f%%)\n",
            sum(anom_df$anomaly_z),
            sum(anom_df$anomaly_z) / nrow(anom_df) * 100))
```

```
## Z-score anomalies (|z|>3) : 12 days (0.8%)
```

```
cat(sprintf("IQR anomalies             : %d days (%.1f%%)\n",
            sum(anom_df$anomaly_iqr),
            sum(anom_df$anomaly_iqr) / nrow(anom_df) * 100))
```

```
## IQR anomalies             : 34 days (2.4%)
```

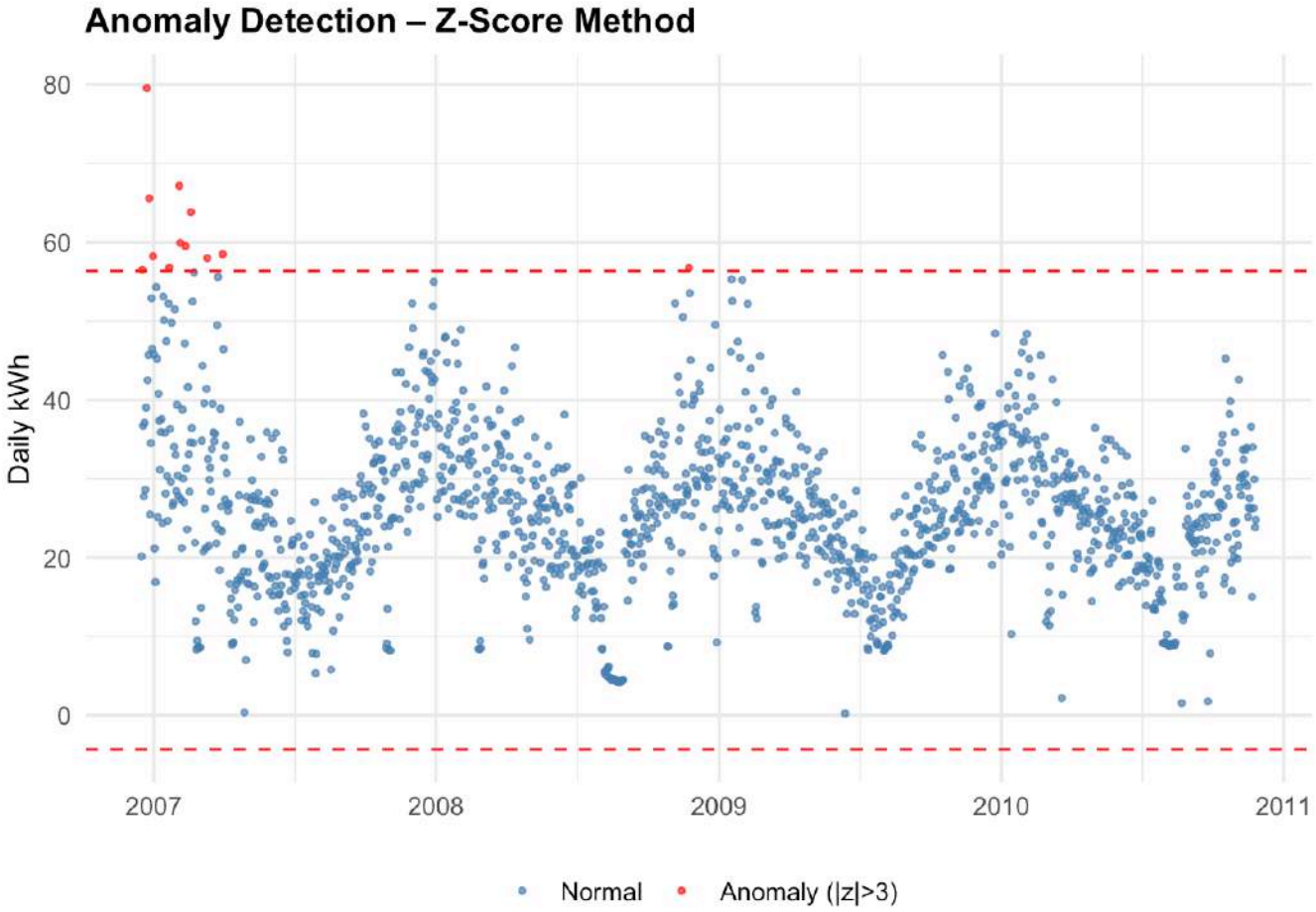
```
cat("\nTop 10 anomalous days (Z-score):\n")
```

```
##
## Top 10 anomalous days (Z-score):
```

```
print(anom_df %>% filter(anomaly_z) %>%
        arrange(desc(abs(z_score))) %>%
        select(Date, total_kwh, z_score) %>% head(10))
```

```
## # A tibble: 10 × 3
##    Date       total_kwh z_score
##    <date>         <dbl>   <dbl>
##  1 2006-12-23      79.6    5.29
##  2 2007-02-03      67.2    4.07
##  3 2006-12-26      65.6    3.91
##  4 2007-02-18      63.8    3.74
##  5 2007-02-04      59.9    3.35
##  6 2007-02-11      59.5    3.31
##  7 2007-03-31      58.5    3.21
##  8 2006-12-31      58.2    3.19
##  9 2007-03-11      58.0    3.16
## 10 2007-01-21      56.8    3.04
```

```
ggplot(anom_df, aes(x = Date, y = total_kwh, colour = anomaly_z)) +
  geom_point(size = 0.8, alpha = 0.7) +
  geom_hline(yintercept = mu + 3 * sgm, linetype = "dashed", colour = "red") +
  geom_hline(yintercept = mu - 3 * sgm, linetype = "dashed", colour = "red") +
  scale_colour_manual(values = c("FALSE" = "steelblue", "TRUE" = "red"),
                      labels = c("Normal", "Anomaly (|z|>3)")) +
  labs(title = "Anomaly Detection — Z-Score Method",
       x = NULL, y = "Daily kWh", colour = NULL) + theme_load
```




```
# ── 12. ARIMA Forecasting ──────────────────────────────────────────────
cat("\n🔮 Section 12: ARIMA Forecasting\n")
```

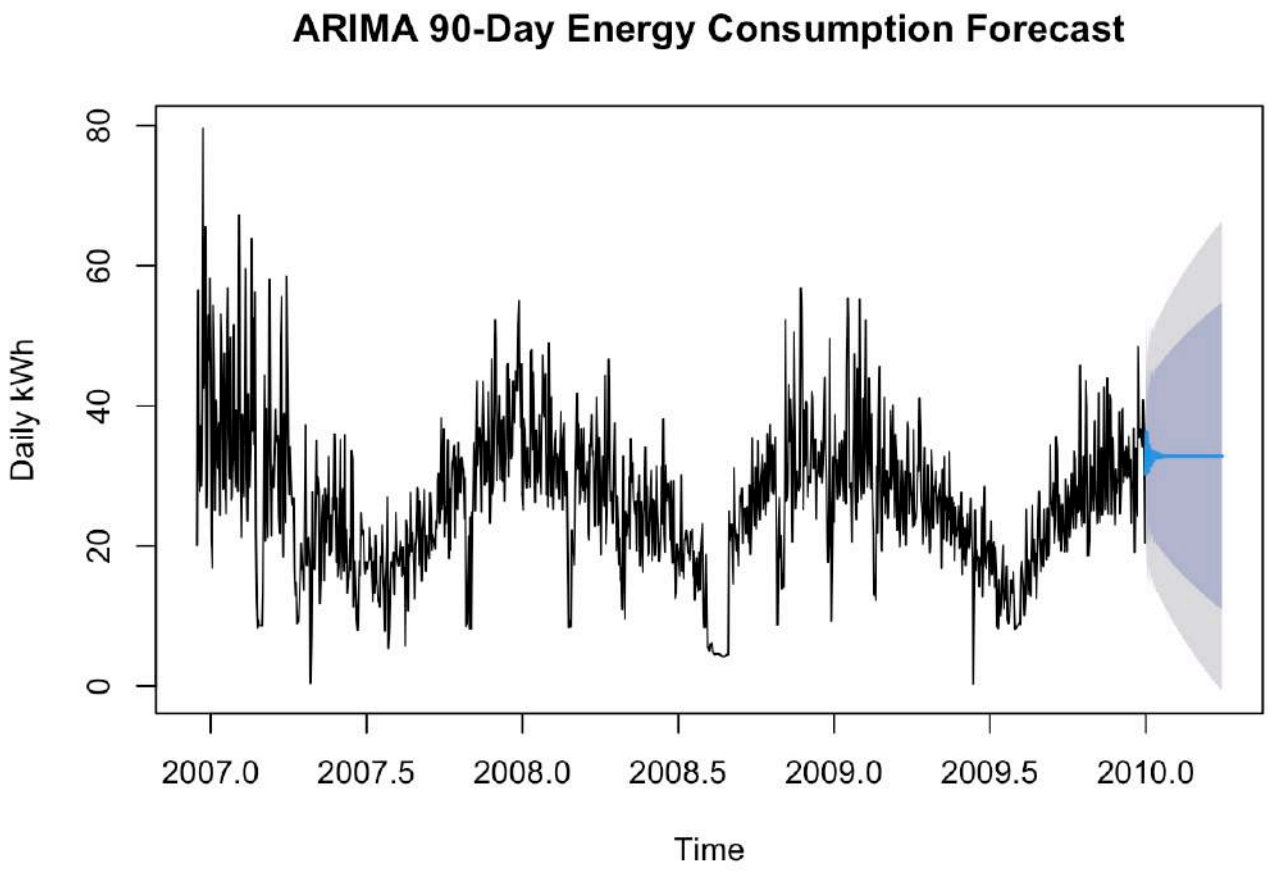
```
##
## 🔮 Section 12: ARIMA Forecasting
```

```
train_ts <- window(daily_ts, end   = c(2009, 365))
test_ts  <- window(daily_ts, start = c(2010, 1))

arima_m  <- auto.arima(na.approx(train_ts),
                       seasonal      = TRUE,
                       stepwise      = FALSE,
                       approximation = FALSE)
cat("Best ARIMA model:\n"); print(arima_m)
```

```
## Best ARIMA model:
```

```
## Series: na.approx(train_ts)
## ARIMA(2,1,3)
##
## Coefficients:
##           ar1      ar2      ma1     ma2      ma3
##       -0.5940  -0.6775  -0.0410  0.0022  -0.4838
## s.e.   0.1099   0.0768   0.1254  0.1394   0.0511
##
## sigma^2 = 58.25:  log likelihood = -3829.03
## AIC=7670.07   AICc=7670.15   BIC=7700.14
```

```
arima_fc <- forecast(arima_m, h = 90)
plot(arima_fc,
     main = "ARIMA 90-Day Energy Consumption Forecast",
     xlab = "Time", ylab = "Daily kWh")
```

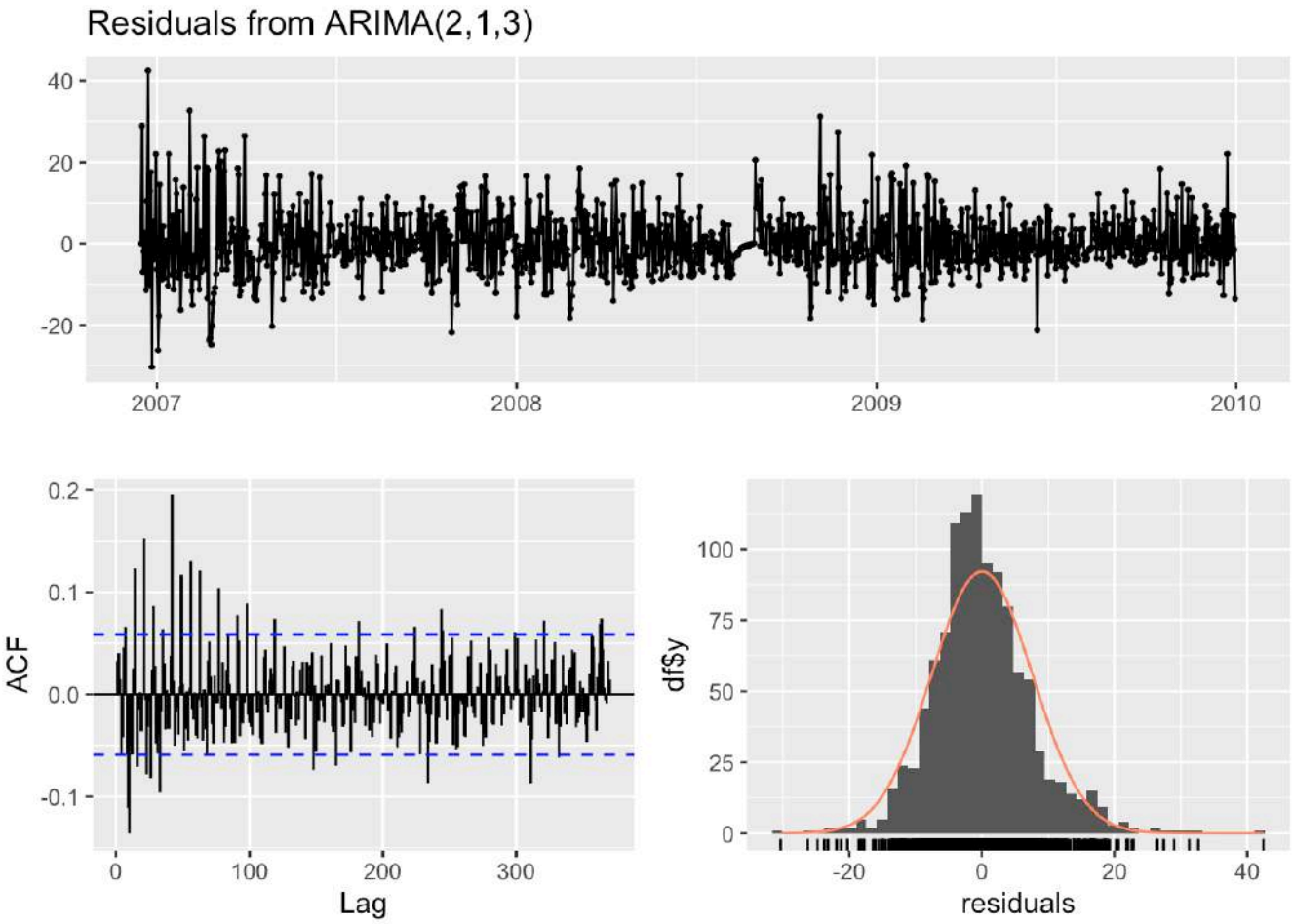



```
acc_a <- forecast::accuracy(arima_fc, na.approx(test_ts))
cat("\nARIMA Forecast Accuracy:\n"); print(round(acc_a, 4))
```

```
##
## ARIMA Forecast Accuracy:
```

```
##                   ME   RMSE    MAE      MPE    MAPE   MASE   ACF1 Theil's U
## Training set  0.0076 7.6116 5.6413 -21.4255 37.6980 0.6729 0.0328        NA
## Test set     -2.2496 8.8595 6.5528 -32.5925 42.9525 0.7816 0.4112    1.2367
```

```
checkresiduals(arima_m)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,1,3)
## Q* = 498.56, df = 217, p-value < 2.2e-16
##
## Model df: 5.   Total lags used: 222
```

```
lb_p <- Box.test(residuals(arima_m), lag = 20, type = "Ljung-Box")$p.value
cat(sprintf("Ljung-Box p=%.4f → %s\n", lb_p,
            ifelse(lb_p > 0.05, "Residuals are white noise ✅",
                   "Residual autocorrelation detected ⚠️")))
```

```
## Ljung-Box p=0.0000 → Residual autocorrelation detected ⚠️
```

```
# ── 13. ETS Forecasting ──────────────────────────────────────────────
# FIX: replaced ets() with stlf() so seasonality is captured correctly.
#      ets() silently drops seasonality when frequency > 24.
cat("\n🔮 Section 13: ETS (Exponential Smoothing State Space) Forecasting\n")
```

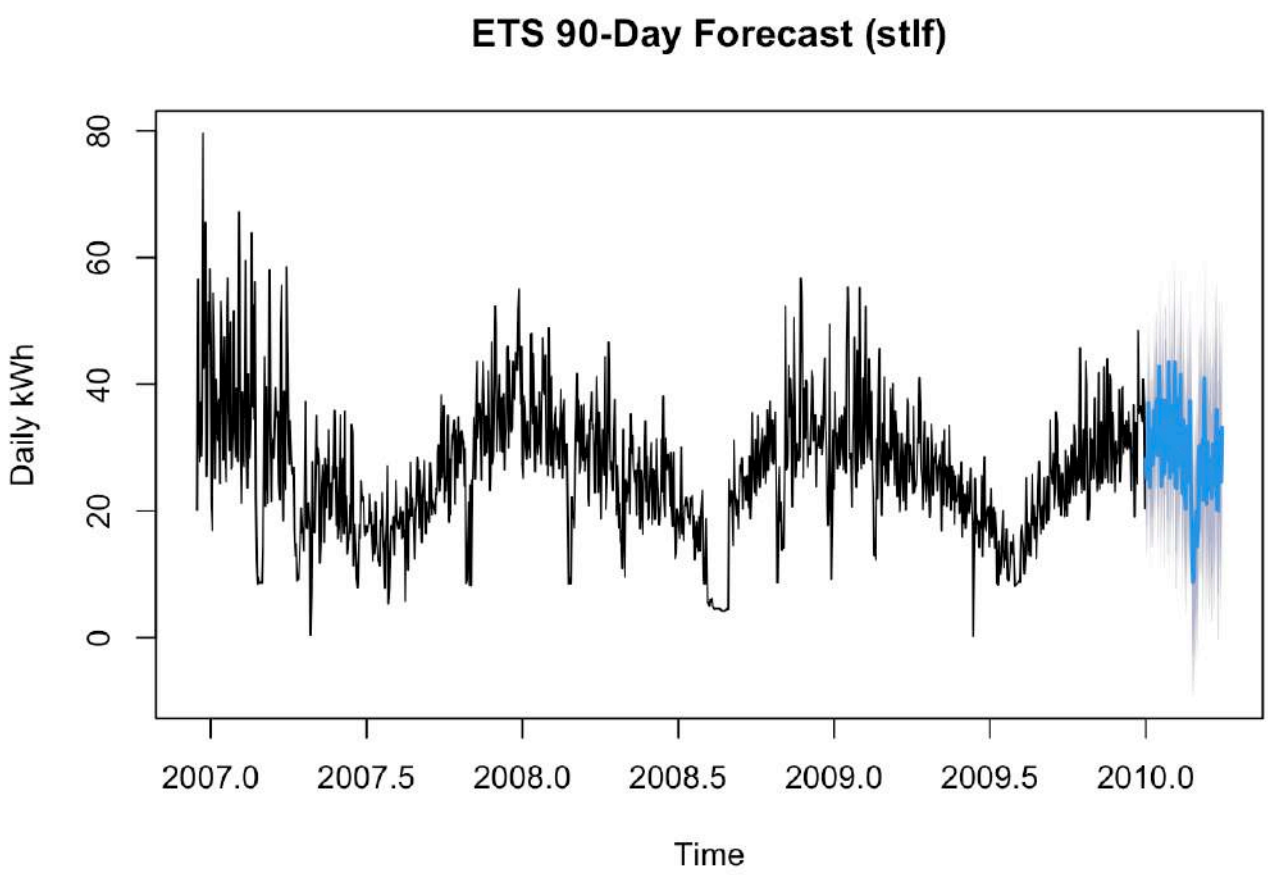
```
##
## 🔮 Section 13: ETS (Exponential Smoothing State Space) Forecasting
```

```
ets_fc <- stlf(na.approx(train_ts), h = 90)   # STL decomposition + ETS
ets_m  <- ets_fc$model                          # underlying ETS model object
cat("ETS model (via stlf):\n"); print(ets_m)
```

```
## ETS model (via stlf):
```

```
## ETS(A,N,N)
##
## Call:
## ets(y = na.interp(x), model = etsmodel, allow.multiplicative.trend = allow.multipl
icative.trend)
##
##   Smoothing parameters:
##     alpha = 0.148
##
##   Initial states:
##     l = 30.289
##
##   sigma:  6.1688
##
##      AIC     AICc      BIC
## 11838.41 11838.43 11853.45
```

```
plot(ets_fc, main = "ETS 90-Day Forecast (stlf)", xlab = "Time", ylab = "Daily kWh")
```



```
acc_e <- forecast::accuracy(ets_fc, na.approx(test_ts))
cat("\nETS Accuracy:\n"); print(round(acc_e, 4))
```

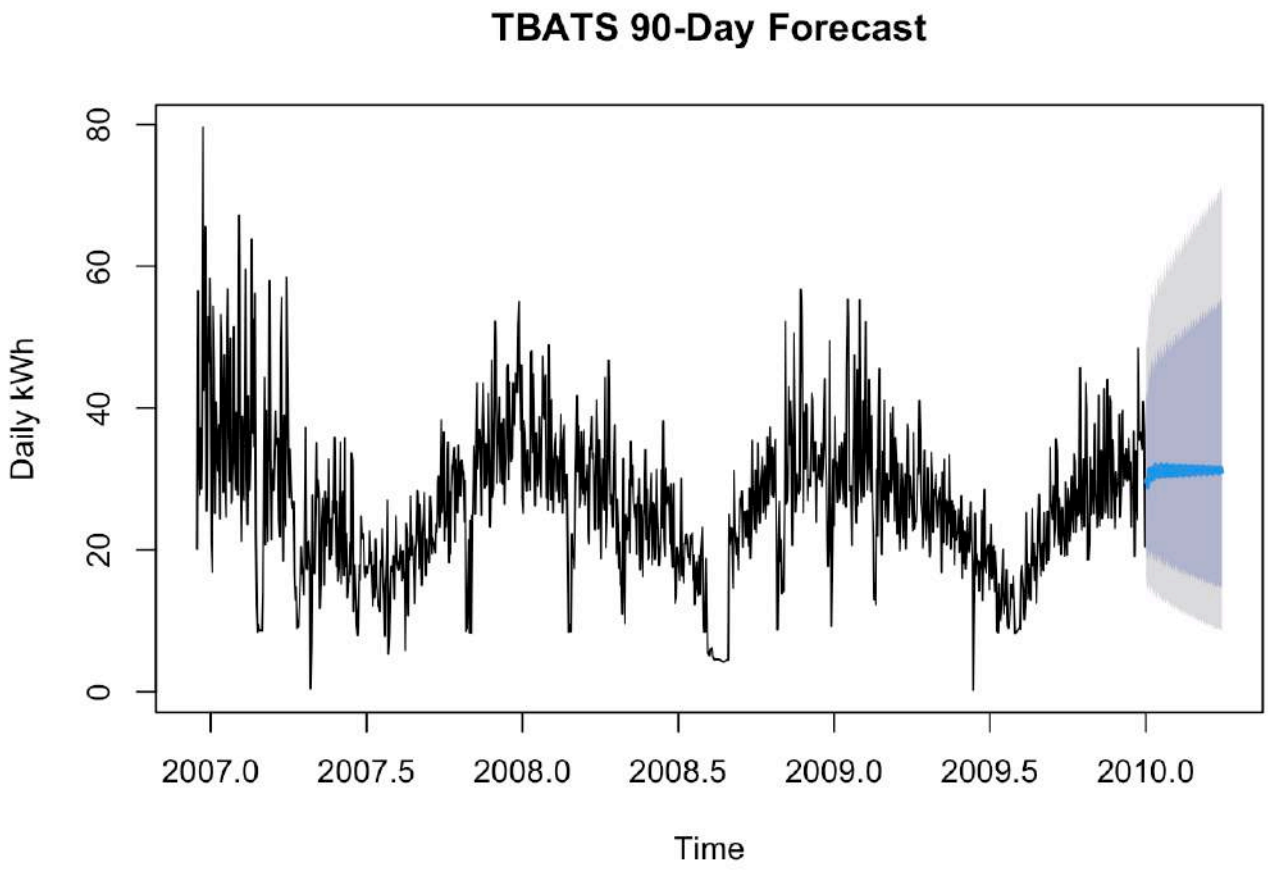
```
##
## ETS Accuracy:
```

```
##                   ME   RMSE    MAE      MPE    MAPE   MASE   ACF1 Theil's U
## Training set -0.0452 6.1633 4.6393 -15.2003 29.3905 0.5534 0.1759        NA
## Test set      2.0941 8.3797 6.1135 -10.2434 34.0594 0.7292 0.1538     0.588
```

```
# ── 14. TBATS Forecasting (handles complex seasonality) ──────────────────────
cat("\n🔮 Section 14: TBATS Forecast\n")
```

```
##
## 🔮 Section 14: TBATS Forecast
```

```
tbats_m  <- tbats(na.approx(train_ts))
tbats_fc <- forecast(tbats_m, h = 90)
plot(tbats_fc, main = "TBATS 90-Day Forecast", xlab = "Time", ylab = "Daily kWh")
```




```
acc_tb <- forecast::accuracy(tbats_fc, na.approx(test_ts))
cat("\nTBATS Accuracy:\n"); print(round(acc_tb, 4))
```

```
##
## TBATS Accuracy:
```

```
##                   ME   RMSE    MAE      MPE    MAPE   MASE    ACF1 Theil's U
## Training set  0.4519 7.4637 5.4409 -19.6990 37.0032 0.6490 -0.0199        NA
## Test set     -0.5002 8.6780 6.3387 -25.5567 40.0207 0.7561  0.3968     1.154
```

```
# ── 15. Model Comparison ──────────────────────────────────────────────
cat("\n📊 Section 15: Forecasting Model Comparison\n")
```

```
##
## 📊 Section 15: Forecasting Model Comparison
```

```
comp_df <- data.frame(
  Model = c("ARIMA", "ETS", "TBATS"),
  RMSE  = c(acc_a["Test set", "RMSE"],
            acc_e["Test set", "RMSE"],
            acc_tb["Test set", "RMSE"]),
  MAE   = c(acc_a["Test set", "MAE"],
            acc_e["Test set", "MAE"],
            acc_tb["Test set", "MAE"]),
  MAPE  = c(acc_a["Test set", "MAPE"],
            acc_e["Test set", "MAPE"],
            acc_tb["Test set", "MAPE"])
)
cat("Model Comparison Table:\n"); print(comp_df)
```

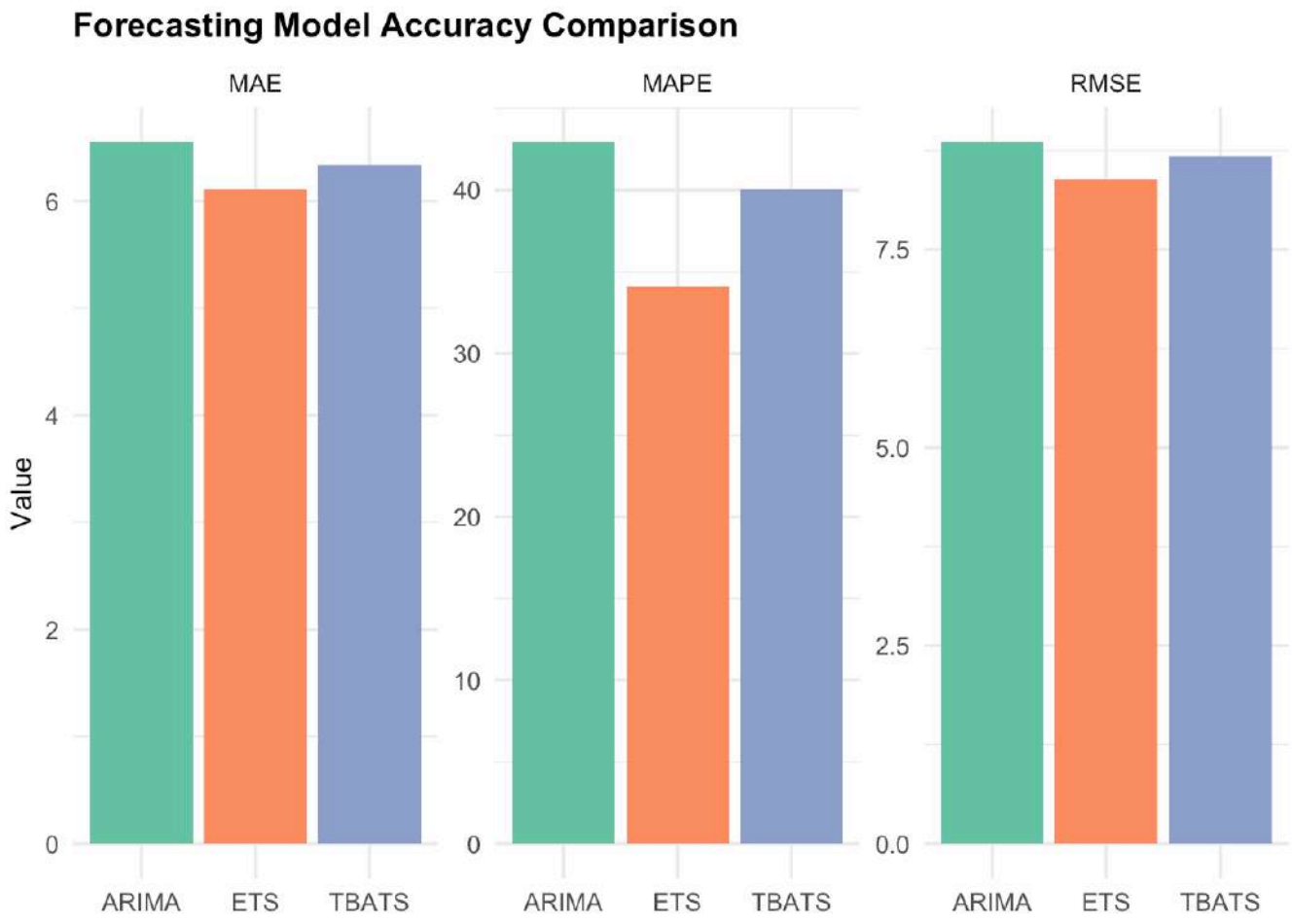
```
## Model Comparison Table:
```

```
##   Model     RMSE      MAE     MAPE
## 1 ARIMA 8.859476 6.552842 42.95253
## 2   ETS 8.379670 6.113496 34.05938
## 3 TBATS 8.678015 6.338669 40.02067
```

```
cat(sprintf("\n🏆 Best model by MAPE: %s\n",
            comp_df$Model[which.min(comp_df$MAPE)]))
```

```
##
## 🏆 Best model by MAPE: ETS
```

```
comp_long <- comp_df %>%
  pivot_longer(-Model, names_to = "Metric", values_to = "Value")

ggplot(comp_long, aes(x = Model, y = Value, fill = Model)) +
  geom_col() +
  facet_wrap(~Metric, scales = "free_y") +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Forecasting Model Accuracy Comparison",
       x = NULL, y = "Value") +
  theme_load + theme(legend.position = "none")
```




```
# ── 16. Random Forest Regression ──────────────────────────────────────
cat("\n🌲 Section 16: Random Forest Load Prediction\n")
```

```
##
## 🌲 Section 16: Random Forest Load Prediction
```

```
rf_df <- daily %>%
  mutate(
    dow     = as.integer(wday(Date, week_start = 1)),
    month_n = as.integer(month(Date)),
    week_n  = week(Date),
    lag1    = dplyr::lag(total_kwh, 1),
    lag7    = dplyr::lag(total_kwh, 7),
    lag30   = dplyr::lag(total_kwh, 30)
  ) %>%
  select(total_kwh, dow, month_n, week_n, day_of_year,
         lag1, lag7, lag30, mean_voltage, mean_pf, mean_react) %>%
  drop_na()

set.seed(42)
train_idx  <- sample(seq_len(nrow(rf_df)), size = 0.80 * nrow(rf_df))
rf_train   <- rf_df[ train_idx, ]
rf_test    <- rf_df[-train_idx, ]

rf_model <- randomForest(
  total_kwh ~ ., data = rf_train,
  ntree = 300, mtry = 4, importance = TRUE
)
cat("Random Forest summary:\n"); print(rf_model)
```

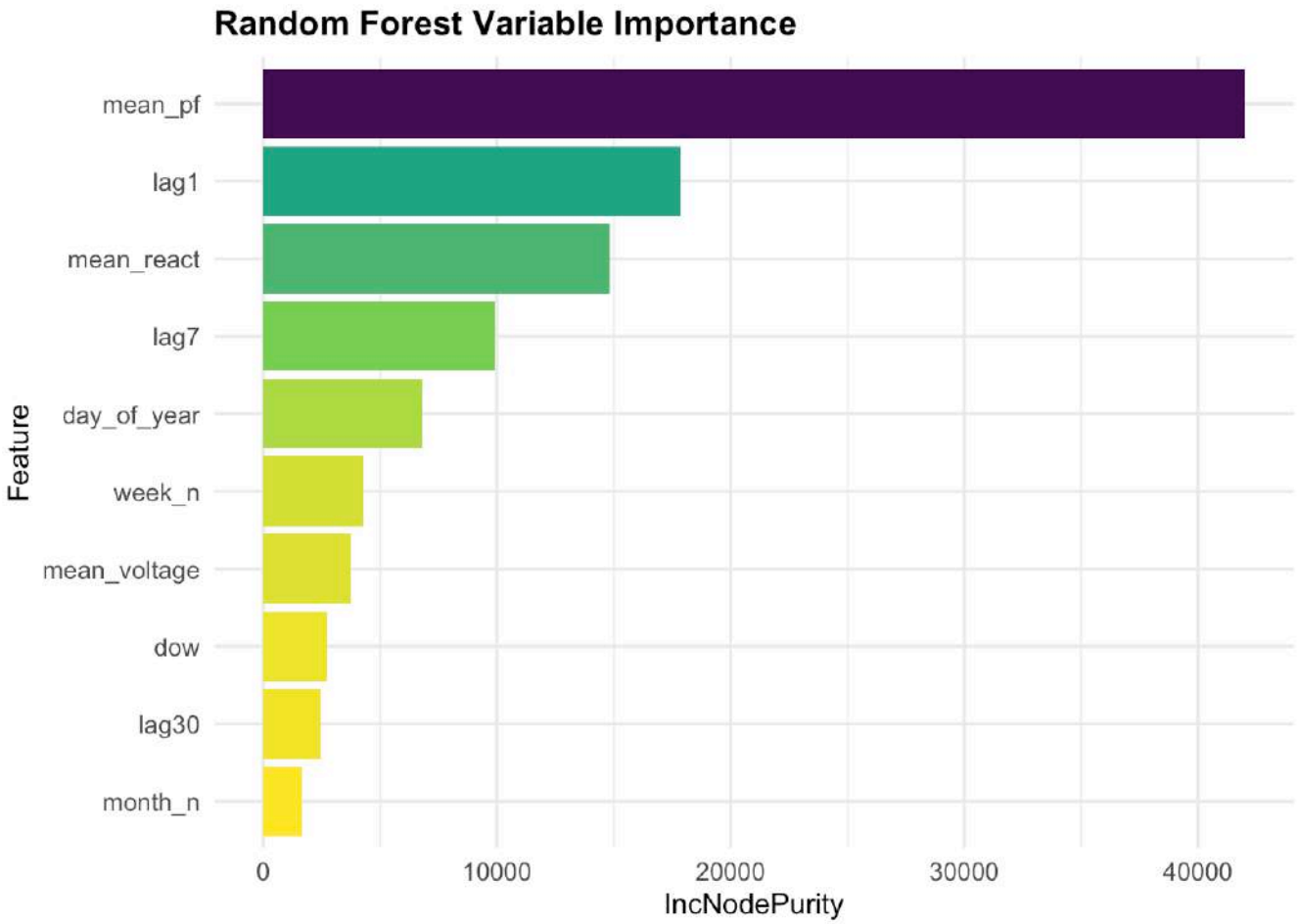
```
## Random Forest summary:
```

```
##
## Call:
##  randomForest(formula = total_kwh ~ ., data = rf_train, ntree = 300,      mtry =
4, importance = TRUE)
##                Type of random forest: regression
##                      Number of trees: 300
## No. of variables tried at each split: 4
##
##           Mean of squared residuals: 19.21356
##                     % Var explained: 80.05
```

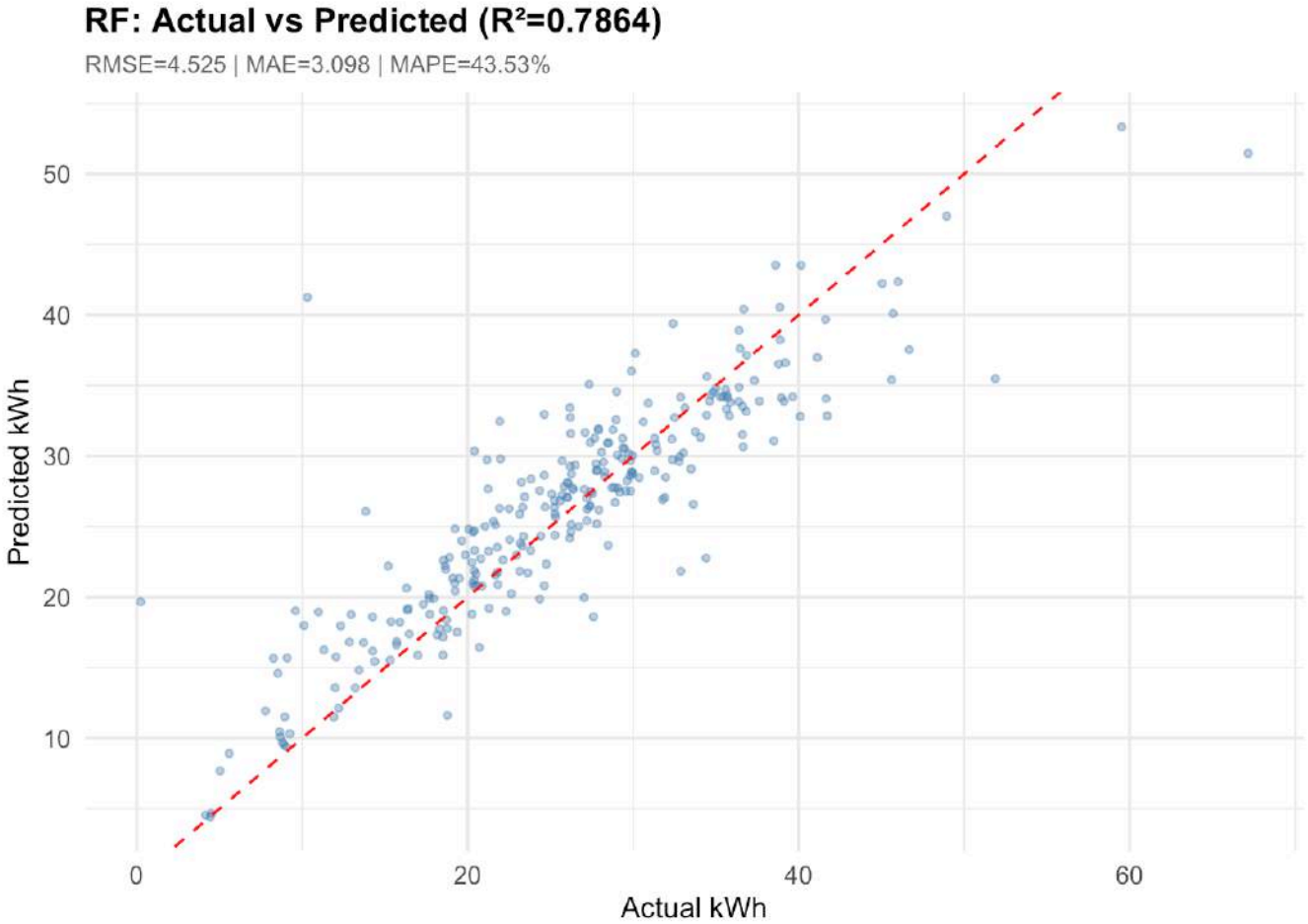
```
rf_pred <- predict(rf_model, rf_test)
rf_rmse <- Metrics::rmse(rf_test$total_kwh, rf_pred)
rf_mae  <- Metrics::mae(rf_test$total_kwh,  rf_pred)
rf_mape <- mean(abs((rf_test$total_kwh - rf_pred) /
                      rf_test$total_kwh)) * 100
rf_r2   <- cor(rf_test$total_kwh, rf_pred)^2

cat(sprintf("RF → RMSE=%.3f | MAE=%.3f | MAPE=%.2f%% | R²=%.4f\n",
            rf_rmse, rf_mae, rf_mape, rf_r2))
```

```
## RF → RMSE=4.525 | MAE=3.098 | MAPE=43.53% | R²=0.7864
```

```
# Variable importance
imp_df <- data.frame(
  Variable  = rownames(importance(rf_model)),
  IncPurity = importance(rf_model)[, "IncNodePurity"]
) %>% arrange(desc(IncPurity))

ggplot(imp_df, aes(x = reorder(Variable, IncPurity),
                   y = IncPurity, fill = IncPurity)) +
  geom_col() + coord_flip() +
  scale_fill_viridis_c(direction = -1) +
  labs(title = "Random Forest Variable Importance",
       x = "Feature", y = "IncNodePurity") +
  theme_load + theme(legend.position = "none")
```




```
# Actual vs Predicted
ggplot(data.frame(Actual = rf_test$total_kwh, Predicted = rf_pred),
       aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.4, size = 1, colour = "steelblue") +
  geom_abline(slope = 1, intercept = 0, colour = "red", linetype = "dashed") +
  labs(title    = sprintf("RF: Actual vs Predicted (R²=%.4f)", rf_r2),
       subtitle = sprintf("RMSE=%.3f | MAE=%.3f | MAPE=%.2f%%",
                          rf_rmse, rf_mae, rf_mape),
       x = "Actual kWh", y = "Predicted kWh") + theme_load
```




```
# ── 17. Multiple Linear Regression ──────────────────────────────────────
cat("\n📐 Section 17: Multiple Linear Regression\n")
```

```
##
## 📐 Section 17: Multiple Linear Regression
```

```
lm_df <- daily %>%
  mutate(
    dow     = as.integer(wday(Date, week_start = 1)),
    month_n = as.integer(month(Date)),
    lag1    = dplyr::lag(total_kwh, 1),
    lag7    = dplyr::lag(total_kwh, 7)
  ) %>%
  select(total_kwh, dow, month_n, day_of_year,
         lag1, lag7, mean_voltage, mean_pf, mean_react) %>%
  drop_na()

lm_idx   <- sample(seq_len(nrow(lm_df)), size = 0.80 * nrow(lm_df))
lm_model <- lm(total_kwh ~ ., data = lm_df[lm_idx, ])
cat("\nLinear Regression Summary:\n"); print(summary(lm_model))
```

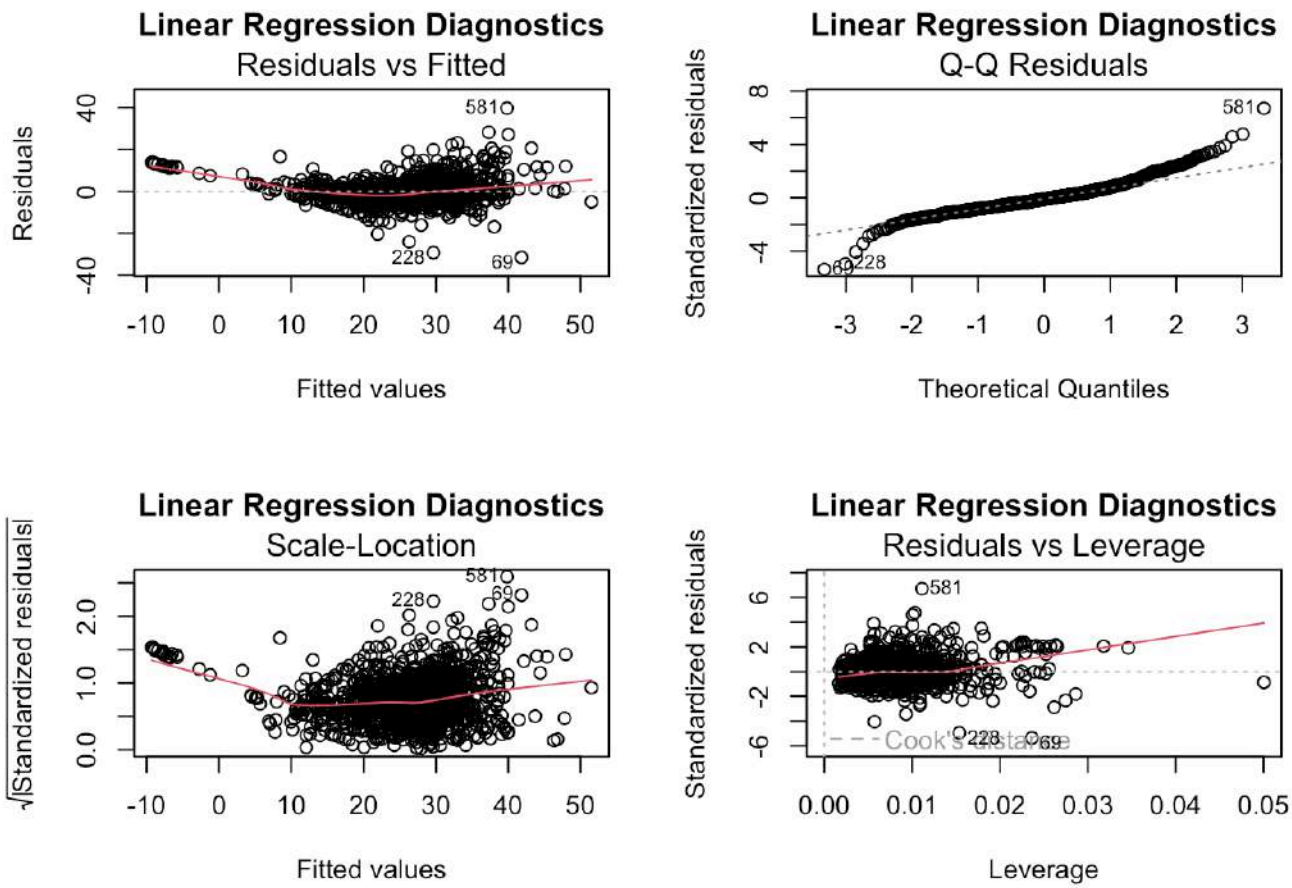
```
##
## Linear Regression Summary:
```

```
##
## Call:
## lm(formula = total_kwh ~ ., data = lm_df[lm_idx, ])
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -31.525  -3.535  -0.530   2.708  39.740
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -161.05329   23.18485  -6.946 6.31e-12 ***
## dow             0.29835    0.09367   3.185  0.00149 **
## month_n        -0.33534    0.61967  -0.541  0.58850
## day_of_year     0.01211    0.02027   0.597  0.55044
## lag1            0.25354    0.02327  10.896  < 2e-16 ***
## lag7            0.12925    0.02206   5.858 6.14e-09 ***
## mean_voltage   -0.19586    0.08956  -2.187  0.02895 *
## mean_pf       217.79442    8.87414  24.543  < 2e-16 ***
## mean_react    106.86476    6.01124  17.777  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.952 on 1131 degrees of freedom
## Multiple R-squared:  0.6469, Adjusted R-squared:  0.6444
## F-statistic:   259 on 8 and 1131 DF,  p-value: < 2.2e-16
```

```
lm_pred   <- predict(lm_model, lm_df[-lm_idx, ])
# FIX: pull() extracts a plain numeric vector from the tibble column;
#      the original lm_df[-lm_idx, "total_kwh"] returned a 1-column tibble
#      which caused Metrics::rmse() to return NA silently.
lm_actual <- lm_df[-lm_idx, ] %>% pull(total_kwh)
lm_rmse   <- Metrics::rmse(lm_actual, lm_pred)
lm_r2     <- summary(lm_model)$r.squared
cat(sprintf("LM → RMSE=%.3f | R²=%.4f\n", lm_rmse, lm_r2))
```

```
## LM → RMSE=6.168 | R²=0.6469
```

```
par(mfrow = c(2, 2))
plot(lm_model, main = "Linear Regression Diagnostics")
```




```
par(mfrow = c(1, 1))

# ── 18. Load Duration Curve ──────────────────────────────────────────────
cat("\n📉 Section 18: Load Duration Curve\n")
```

```
##
## 📉 Section 18: Load Duration Curve
```

```
ldc_df <- df %>%
  group_by(hour_block = floor_date(datetime, "hour")) %>%
  summarise(mean_power = mean(Global_active_power, na.rm = TRUE), .groups = "drop") %
>%
  arrange(desc(mean_power)) %>%
  mutate(pct_time = row_number() / n() * 100)

base_load <- quantile(ldc_df$mean_power, 0.10)
avg_load  <- mean(ldc_df$mean_power, na.rm = TRUE)
pk_load   <- quantile(ldc_df$mean_power, 0.90)

cat(sprintf("Base load  (10th pctile): %.3f kW\n", base_load))
```

```
## Base load  (10th pctile): 0.275 kW
```

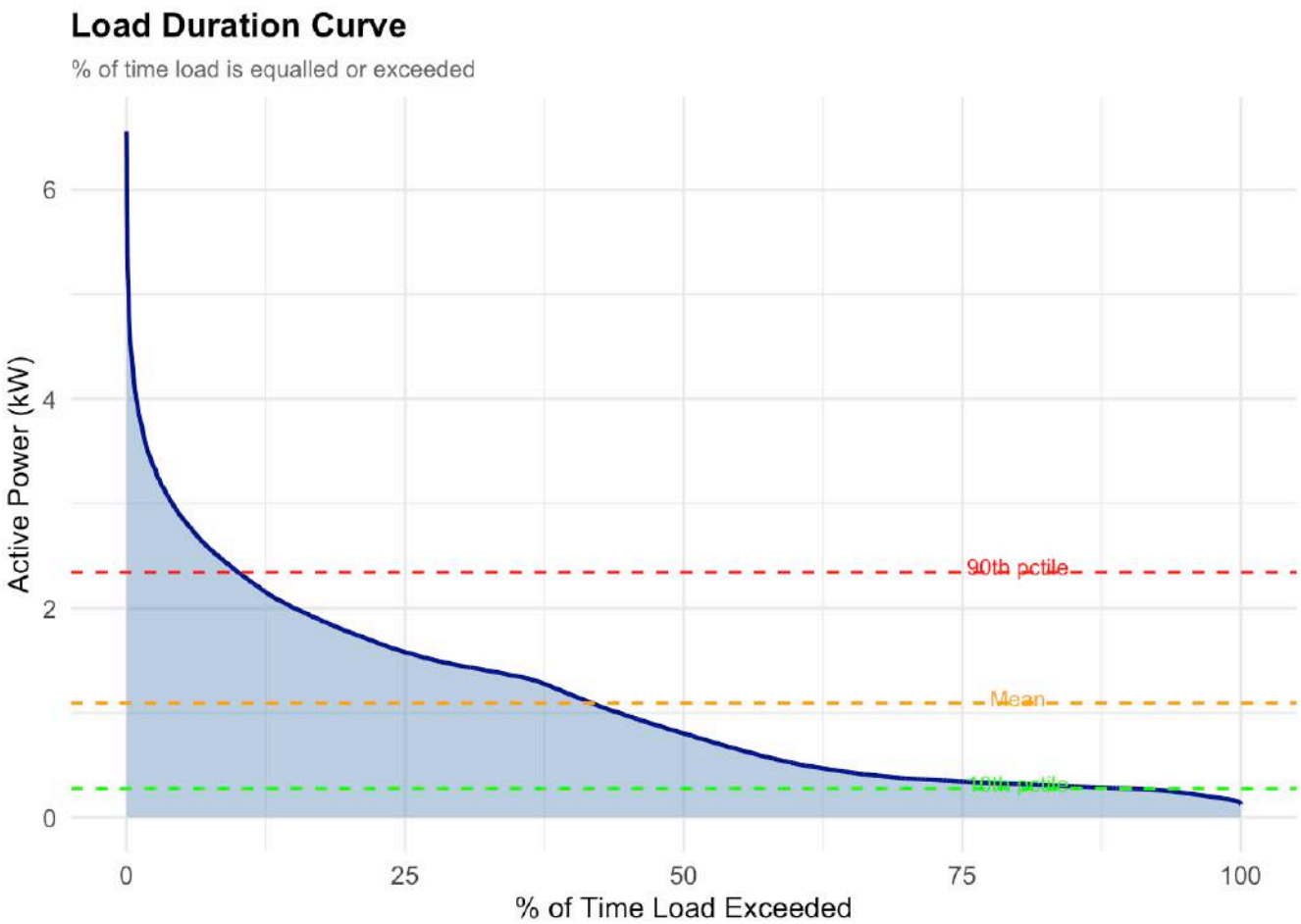
```
cat(sprintf("Average load            : %.3f kW\n", avg_load))
```

```
## Average load            : 1.092 kW
```

```
cat(sprintf("Peak load  (90th pctile): %.3f kW\n", pk_load))
```

```
## Peak load  (90th pctile): 2.345 kW
```

```
ggplot(ldc_df, aes(x = pct_time, y = mean_power)) +
  geom_area(fill = "steelblue", alpha = 0.4) +
  geom_line(colour = "darkblue", linewidth = 0.8) +
  geom_hline(yintercept = pk_load,   colour = "red",    linetype = "dashed") +
  geom_hline(yintercept = avg_load,  colour = "orange", linetype = "dashed") +
  geom_hline(yintercept = base_load, colour = "green",  linetype = "dashed") +
  annotate("text", x = 80, y = pk_load   + 0.05, label = "90th pctile", colour = "re
d",    size = 3) +
  annotate("text", x = 80, y = avg_load  + 0.05, label = "Mean",        colour = "ora
nge", size = 3) +
  annotate("text", x = 80, y = base_load + 0.05, label = "10th pctile", colour = "gre
en",  size = 3) +
  labs(title    = "Load Duration Curve",
       subtitle = "% of time load is equalled or exceeded",
       x = "% of Time Load Exceeded", y = "Active Power (kW)") + theme_load
```




```
# ── 19. Energy Cost Analysis ──────────────────────────────────────────
cat("\n💰 Section 19: Electricity Cost Analysis\n")
```

```
##
## 💰 Section 19: Electricity Cost Analysis
```

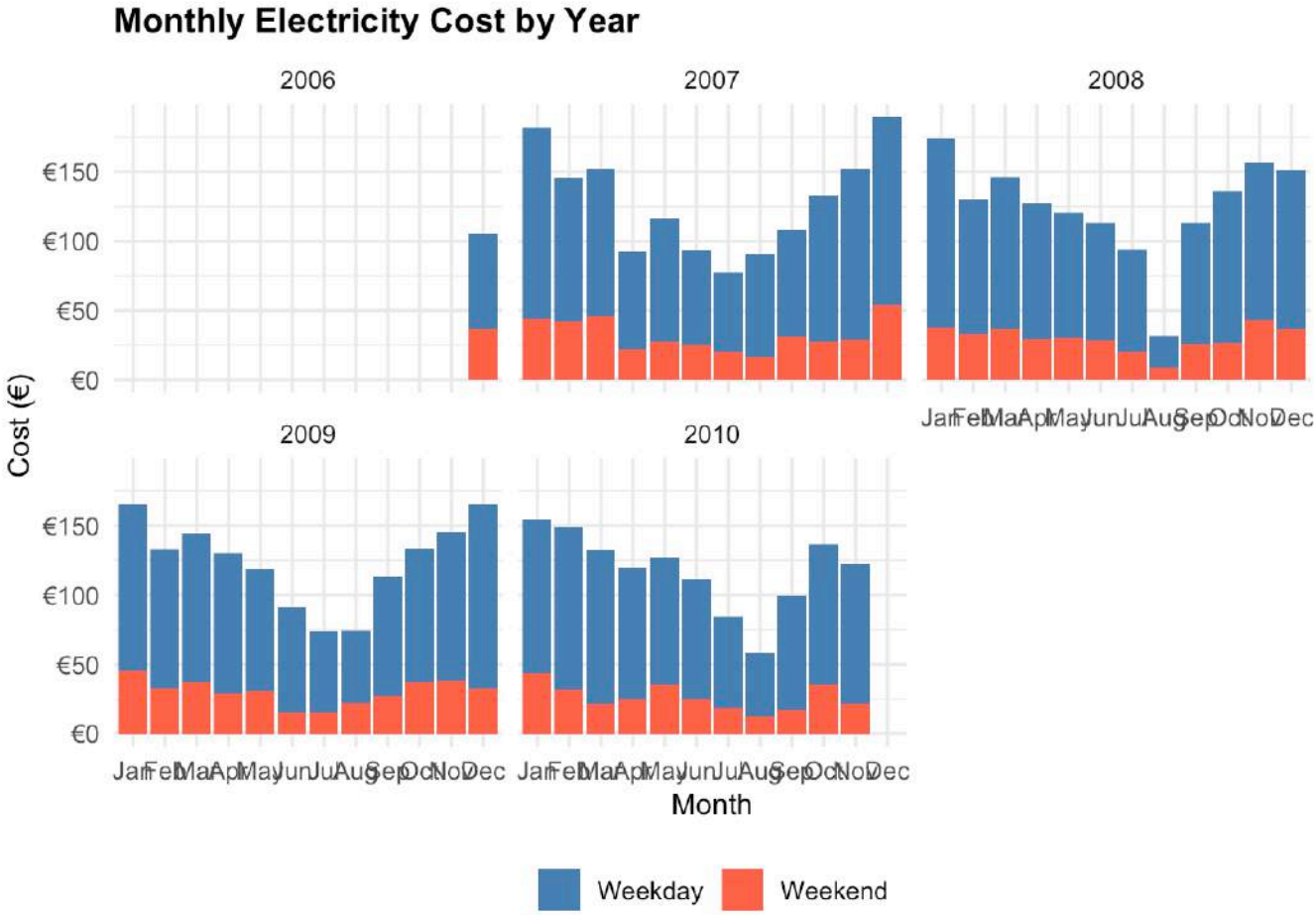
```
# French EDF tariff approximations (€/kWh)
OFF_PEAK <- 0.1470   # Night (22:00–06:00)
PEAK     <- 0.1841   # Day
WKND     <- 0.1200   # Weekend flat

cost_df <- df %>%
  mutate(rate = case_when(
    is_weekend == "Weekend" ~ WKND,
    hour >= 22 | hour < 6  ~ OFF_PEAK,
    TRUE                   ~ PEAK
  ),
  cost_eur = energy_kwh * rate) %>%
  group_by(year, month, is_weekend) %>%
  summarise(total_kwh  = sum(energy_kwh,  na.rm = TRUE),
            total_cost = sum(cost_eur,     na.rm = TRUE), .groups = "drop")

annual_cost <- cost_df %>%
  group_by(year) %>%
  summarise(annual_kwh  = round(sum(total_kwh),  1),
            annual_cost = round(sum(total_cost),  2), .groups = "drop")

cat("Annual Energy & Cost Summary:\n")
```

```
## Annual Energy & Cost Summary:
```

```
print(annual_cost)
```

```
## # A tibble: 5 × 3
##    year annual_kwh annual_cost
##   <dbl>      <dbl>       <dbl>
## 1  2006       697.        106.
## 2  2007      9712.       1533.
## 3  2008      9415.       1495.
## 4  2009      9372.       1488.
## 5  2010      8088.       1293.
```

```
ggplot(cost_df, aes(x = month, y = total_cost, fill = is_weekend)) +
  geom_col(position = "stack") +
  facet_wrap(~year) +
  scale_fill_manual(values = c("Weekday" = "steelblue", "Weekend" = "tomato")) +
  scale_y_continuous(labels = label_dollar(prefix = "€")) +
  labs(title = "Monthly Electricity Cost by Year",
       x = "Month", y = "Cost (€)", fill = NULL) + theme_load
```




```
# ── 20. Peak Load Detection ──────────────────────────────────────────────
cat("\n⚡ Section 20: Peak Load Detection & Analysis\n")
```

```
##
## ⚡ Section 20: Peak Load Detection & Analysis
```

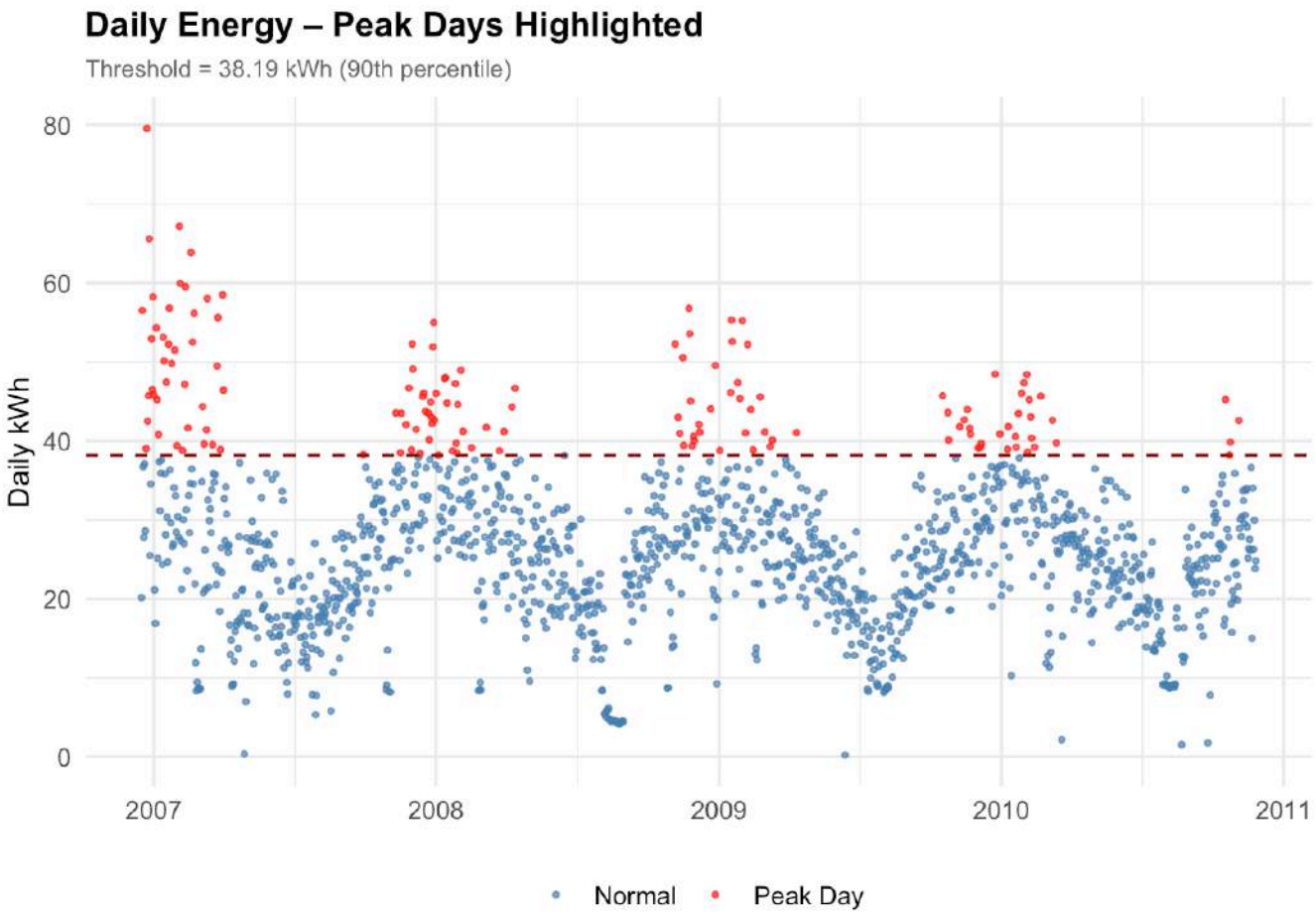
```
p90   <- quantile(daily$total_kwh, 0.90, na.rm = TRUE)
daily <- daily %>% mutate(is_peak = total_kwh >= p90)

peak_by_month <- daily %>%
  group_by(month = month(Date, label = TRUE)) %>%
  summarise(
    peak_days  = sum(is_peak, na.rm = TRUE),
    total_days = n(),
    peak_pct   = round(100 * peak_days / total_days, 1),
    .groups = "drop"
  )
cat("Peak day frequency (top 10% consumption) by month:\n")
```

```
## Peak day frequency (top 10% consumption) by month:
```

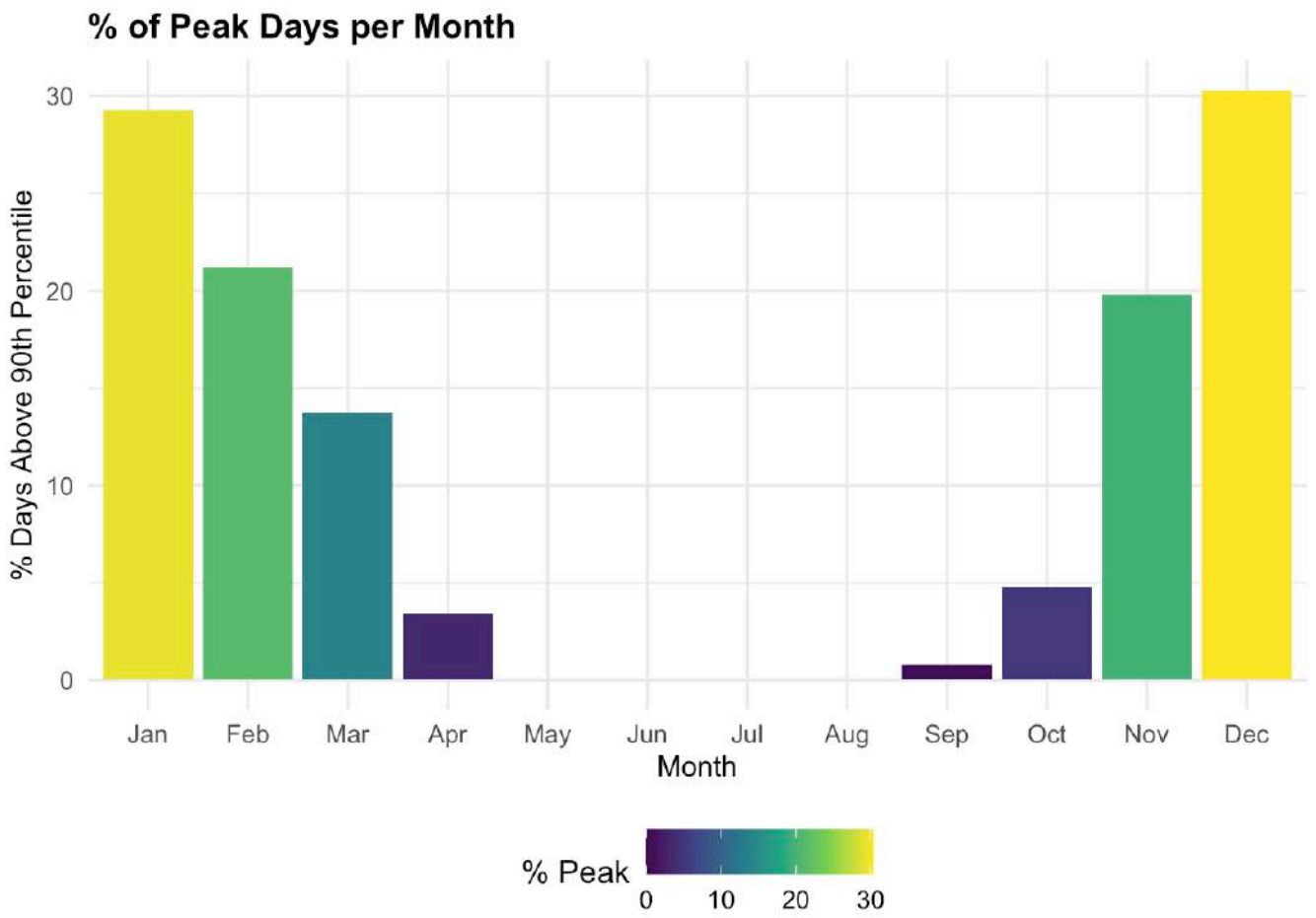
```
print(peak_by_month)
```

```
## # A tibble: 12 × 4
##    month peak_days total_days peak_pct
##    <ord>     <int>      <int>    <dbl>
##  1 Jan          36        123     29.3
##  2 Feb          24        113     21.2
##  3 Mar          17        124     13.7
##  4 Apr           4        119      3.4
##  5 May           0        124      0
##  6 Jun           0        119      0
##  7 Jul           0        124      0
##  8 Aug           0        120      0
##  9 Sep           1        118      0.8
## 10 Oct           6        124      4.8
## 11 Nov          23        116     19.8
## 12 Dec          33        109     30.3
```

```
ggplot(daily, aes(x = Date, y = total_kwh, colour = is_peak)) +
  geom_point(size = 0.7, alpha = 0.7) +
  geom_hline(yintercept = p90, linetype = "dashed", colour = "darkred") +
  scale_colour_manual(values = c("FALSE" = "steelblue", "TRUE" = "red"),
                      labels = c("Normal", "Peak Day")) +
  labs(title    = "Daily Energy — Peak Days Highlighted",
       subtitle = sprintf("Threshold = %.2f kWh (90th percentile)", p90),
       x = NULL, y = "Daily kWh", colour = NULL) + theme_load
```



```
ggplot(peak_by_month, aes(x = month, y = peak_pct, fill = peak_pct)) +
  geom_col() +
  scale_fill_viridis_c(name = "% Peak") +
  labs(title = "% of Peak Days per Month",
       x = "Month", y = "% Days Above 90th Percentile") + theme_load
```




```
# ── 21. Power System Sizing Recommendations ──────────────────────────────
cat("\n🏗️  Section 21: Automated Power System Sizing\n")
```

```
##
## 🏗️  Section 21: Automated Power System Sizing
```

```
peak_kw   <- max(df$Global_active_power,  na.rm = TRUE)
p95_kw    <- quantile(df$Global_active_power, 0.95, na.rm = TRUE)
avg_kw    <- mean(df$Global_active_power, na.rm = TRUE)
ann_kwh   <- mean(daily$total_kwh, na.rm = TRUE) * 365
load_fac  <- avg_kw / peak_kw
diversity <- 1.25

gen_kva     <- round(peak_kw * diversity / 0.80, 2)
gen_kva_p95 <- round(p95_kw  * diversity / 0.80, 2)
trafo_kva   <- round(peak_kw * 1.20, 2)
cable_230   <- round((peak_kw * 1000) / 230  * 1.25, 2)
cable_415   <- round((peak_kw * 1000) / 415  * 1.25, 2)
batt_kwh    <- round(avg_kw  * 4, 2)
solar_kwp   <- round(ann_kwh / (365 * 4.5), 2)

sizing_tbl <- data.frame(
  Component  = c("Generator (peak sizing)",
                 "Generator (P95 sizing)",
                 "Transformer",
                 "Cable — 230V single-phase",
                 "Cable — 415V three-phase",
                 "Battery Storage (4-hr backup)",
                 "Solar PV (self-consumption)"),
  Value      = c(paste(gen_kva,     "kVA"),
                 paste(gen_kva_p95, "kVA"),
                 paste(trafo_kva,   "kVA"),
                 paste(cable_230,   "A"),
                 paste(cable_415,   "A"),
                 paste(batt_kwh,    "kWh"),
                 paste(solar_kwp,   "kWp")),
  Engineering_Standard = c(
    "IEC 60034 — Peak × 1.25 diversity ÷ 0.8 PF",
    "IEC 60034 — P95 × 1.25 diversity ÷ 0.8 PF",
    "IEEE C57.91 — Peak × 1.20 safety margin",
    "NEC 230.79 — Peak kW ÷ 230V × 1.25",
    "IEC 60228 — Peak kW ÷ 415V × 1.25",
    "IEEE 1625 — Avg load × 4 h",
    "IEC 61724 — Annual kWh ÷ (365 d × 4.5 h)"
  )
)

cat("\n────────────────────────────────────────────────────────────────────────────────\n")
```

```
##
## ──────────────────────────────────────────────────────────────────────────────
```

```
cat("               AUTOMATED POWER SYSTEM SIZING REPORT\n")
```

```
##                AUTOMATED POWER SYSTEM SIZING REPORT
```

```
cat("────────────────────────────────────────────────────────────────────────────────\n")
```

```
## ────────────────────────────────────────────────────────────────────────────
```

```
cat(sprintf("  Peak Load (absolute)  : %.4f kW\n",  peak_kw))
```

```
##   Peak Load (absolute)  : 11.1220 kW
```

```
cat(sprintf("  P95 Load              : %.4f kW\n",  p95_kw))
```

```
##   P95 Load              : 3.2640 kW
```

```
cat(sprintf("  Average Load          : %.4f kW\n",  avg_kw))
```

```
##   Average Load          : 1.0916 kW
```

```
cat(sprintf("  Load Factor           : %.3f (%.1f%%)\n", load_fac, load_fac * 100))
```

```
##   Load Factor           : 0.098 (9.8%)
```

```
cat(sprintf("  Estimated Annual kWh  : %.1f kWh\n", ann_kwh))
```

```
##   Estimated Annual kWh  : 9496.6 kWh
```

```
cat(sprintf("  Diversity Factor      : %.2f\n\n",   diversity))
```

```
##   Diversity Factor      : 1.25
```

```
print(sizing_tbl, row.names = FALSE)
```

```
##                      Component     Value
##        Generator (peak sizing) 17.38 kVA
##         Generator (P95 sizing)   5.1 kVA
##                     Transformer 13.35 kVA
##      Cable — 230V single-phase   60.45 A
##       Cable — 415V three-phase    33.5 A
##  Battery Storage (4-hr backup)  4.37 kWh
##    Solar PV (self-consumption)  5.78 kWp
##                        Engineering_Standard
##  IEC 60034 — Peak × 1.25 diversity ÷ 0.8 PF
##   IEC 60034 — P95 × 1.25 diversity ÷ 0.8 PF
##     IEEE C57.91 — Peak × 1.20 safety margin
##          NEC 230.79 — Peak kW ÷ 230V × 1.25
##           IEC 60228 — Peak kW ÷ 415V × 1.25
##                  IEEE 1625 — Avg load × 4 h
##    IEC 61724 — Annual kWh ÷ (365 d × 4.5 h)
```

```
# ── 22. Final Summary ──────────────────────────────────────────────────────
cat("\n━━━
```

```
cat(sprintf("  LM    RMSE           : %.3f | R²=%.4f\n",   lm_rmse, lm_r2))
```

```
##   LM    RMSE           : 6.168 | R²=0.6469
```

```
cat(sprintf("  Best Forecaster      : %s\n",
            comp_df$Model[which.min(comp_df$MAPE)]))
```

```
##   Best Forecaster      : ETS
```

```
cat(sprintf("  Z-score Anomalies    : %d days\n",  sum(anom_df$anomaly_z)))
```

```
##   Z-score Anomalies    : 12 days
```

```
cat(sprintf("  IQR Anomalies        : %d days\n",  sum(anom_df$anomaly_iqr)))
```

```
##   IQR Anomalies        : 34 days
```

```
cat(sprintf("  Mean Change-Points   : %d detected\n", length(cpts(cpt_m))))
```

```
##   Mean Change-Points   : 588 detected
```

```
cat(sprintf("  Peak Threshold       : %.2f kWh/day (90th pctile)\n", p90))
```

```
##   Peak Threshold       : 38.19 kWh/day (90th pctile)
```

```
cat(sprintf("  Generator Size       : %.2f kVA\n", gen_kva))
```

```
##   Generator Size       : 17.38 kVA
```

```
cat(sprintf("  Transformer Size     : %.2f kVA\n", trafo_kva))
```

```
##   Transformer Size     : 13.35 kVA
```

```
cat(sprintf("  Battery Storage      : %.2f kWh\n", batt_kwh))
```

```
##   Battery Storage      : 4.37 kWh
```

```
cat(sprintf("  Solar PV             : %.2f kWp\n", solar_kwp))
```

```
##   Solar PV             : 5.78 kWp
```